Strongly polynomial local-connectivity-preserving complete splitting-off in hypergraphs

Kristof Berczi, Karthekeyan Chandrasekaran, Tamas Kiraly, Shubhang Kulkarni

arxiv:2307.08555v4; CC BY 4.0

Editorial note. The single primary target is thm:complete-splitting-off. Whole author mathematical body and support retained; inputs/includes and author bbl mechanically expanded. Only text-font and XeLaTeX wrapping adapted; native original files preserved.

Editorial note. Unavailable unused package loads omitted; the standard integer symbol Z from proba is bound to mathbb Z explicitly.

Allowed standard prerequisites (editorial).

- Standard algorithmic and mathematical background cited by the authors. All source-local constructions, proofs, appendices and references are retained; no new mathematical assumptions added.

Abstract

The splitting-off operation in undirected graphs is a fundamental reduction operation that detaches all edges incident to a given vertex and adds new edges between the neighbors of that vertex while preserving their degrees. Lovász [47, 49] and Mader [50] showed the existence of this operation while preserving global and local connectivities respectively in graphs under certain conditions. These results have far-reaching applications in graph algorithms literature [3, 9, 10, 14, 19, 24, 25, 26, 27, 28, 31, 32, 34, 35, 37, 40, 42, 43, 48, 50, 51, 52, 53]. In this work, we introduce a splitting-off operation in hypergraphs. We show that there exists a local connectivity preserving complete splitting-off in hypergraphs and give a strongly polynomial-time algorithm to compute it in weighted hypergraphs. We illustrate the usefulness of our splitting-off operation in hypergraphs by showing two applications: (1) we give a constructive characterization of k-hyperedge-connected hypergraphs and (2) we give an alternate proof of an approximate min-max relation for max Steiner rooted-connected orientation of graphs and hypergraphs (due to Király and Lau [40]). Our proof of the approximate min-max relation for graphs circumvents the Nash-Williams' strong orientation theorem and uses tools developed for hypergraphs.

Contents

1 Introduction 1
- 1.1 Proof Technique for Theorem 1.2 . . . 8
- 1.2 Proof Technique for Theorems 1.3 and 1.4 . . . 8
- 1.3 Proof Technique for Theorem 1.1 . . . 9
- 1.4 Proof Technique for Theorem 1.6 . . . 11

2 Preliminaries 14

3 Local Connectivity Preserving Complete h-Splitting-Off 15

4 Constructive Characterization of k-Hyperedge-Connected Hypergraphs 16

5 Steiner Rooted Connected Orientation of Graphs and Hypergraphs 18

6 Weak to Strong Cover in Strongly Polynomial Time 21
- 6.1 Weak to Strong Cover Algorithm . . . 22
- 6.2 Properties of Tight Sets . . . 23
- 6.3 Termination and Partial Correctness of the Algorithm . . . 25
- 6.4 Number of Recursive Calls and Hypergraph Support Size . . . 31
- 6.5 Run-time of the Algorithm . . . 33

7 Conclusion 36

A An Example for Weak To Strong Cover with Quadratic New Hyperedges 41

B Projection of Laminar Family 41

C Optimization Problems using Function Maximization Oracle 42
- C.1 p-`max-wc-Oracle` using p-`max-sc-Oracle` . . . 43
- C.2 Skew-supermodularity of a function p . . . 44
- C.3 Computing the Family of Maximal (p, H, w)-Tight Sets . . . 45

1 Introduction

The splitting-off operation in undirected graphs is a simple yet powerful operation in graph theory. It is a reduction operation that detaches all edges incident to a given vertex and adds new edges between the neighbors of that vertex while preserving their degrees. Equivalently, it enables a vertex to exit the network by informing its neighbors how to reconfigure the lost links among themselves in order to preserve their degrees. Lovász [47, 49] introduced the splitting-off operation and showed the existence of the operation to preserve global edge-connectivity under certain conditions. Mader [50] showed the existence of the splitting-off operation to preserve local edge-connectivity (i.e., all pairwise edge-connectivities) under certain conditions. Both Lovász's and Mader's results also admit strongly polynomial-time algorithms [27, 29, 51]. Owing to the inductive nature of the splitting-off operation, Lovász's and Mader's results have enabled fundamental results in graph theory as well as efficient algorithms and min-max relations for numerous graph optimization problems. In fact, Mader [50] illustrated the power of his local edge-connectivity preserving splitting-off result by deriving Nash-Williams' strong orientation theorem [54] (also see Frank's exposition of this derivation [25]). Subsequently, the splitting-off operation has been used to give a constructive characterization of k-edge-connected graphs [27] and to address problems in edge-connectivity augmentation [3, 24, 26, 27, 51], graph orientation [28, 40], minimum cuts enumeration [32, 34, 52], network design [14, 19, 31, 37, 48], tree packing [9, 35, 42, 43], congruency-constrained cuts [53], and approximation algorithms for TSP [10, 31]. Designing fast algorithms for global/local edge-connectivity preserving splitting-off remains an active area of research (e.g., see recent works [11, 12, 13, 44]) due to these far-reaching applications. In this work, we introduce a splitting-off operation in hypergraphs, show the existence of local-connectivity preserving splitting-off operation, design a strongly polynomial-time algorithm to compute it in weighted hypergraphs, and illustrate its usefulness by showing two applications.

Hypergraphs. Hypergraphs offer a richer and more accurate model than graphs for several applications. Consequently, hypergraphs have found applications in several modern areas (e.g., see [46, 56, 59, 62]) and these applications have, in turn, driven exciting recent progress in algorithms for hypergraph optimization problems [2, 5, 15, 16, 17, 20, 21, 23, 30, 33, 36, 38, 39, 41, 45, 57, 60]. A hypergraph $G = (V, E)$ consists of a finite set V of vertices and a set E of hyperedges, where every hyperedge $e \in E$ is a subset of V. We will denote a hypergraph $G = (V, E)$ with hyperedge weights $w : E \to \mathbb{Z}_+$ by the tuple (G, w) and use G_w to denote the unweighted multi-hypergraph over vertex set V containing $w(e)$ copies of every hyperedge $e \in E$. Throughout this work, we will be interested only in hypergraphs with positive integral weights and for algorithmic problems where the input/output is a hypergraph, we will require that the weights are represented in binary. If all hyperedges have size at most 2, then the hyperedges are known as edges and we call such a hypergraph as a graph. We emphasize a subtle but important distinction between hypergraphs and graphs: the number of hyperedges in a hypergraph could be exponential in the number of vertices. This is in sharp contrast to graphs where the number of edges is at most the square of the number of vertices. Consequently, in hypergraph network design problems where the goal is to construct a hypergraph with certain properties, one needs to be mindful of the number of hyperedges in the hypergraph returned by the algorithm. Recent works in hypergraph algorithms literature have focused on this issue in the context of cut/spectral sparsification of hypergraphs [5, 20, 21, 36, 38, 39, 41, 45, 57, 60].

Notation. Let $(G = (V, E), w : E \to \mathbb{Z}_+)$ be a hypergraph. For $X \subseteq V$, let $\delta_G(X) := \{e \in E : e \cap X \neq \emptyset, e \setminus X \neq \emptyset\}$. We define the cut function $d_{(G,w)} : 2^V \to \mathbb{Z}_{\geq 0}$ by $d_{(G,w)}(X) := \sum_{e \in \delta_G(X)} w(e)$ for every $X \subseteq V$. For a vertex $v \in V$, we use $d_{(G,w)}(v)$ to denote $d_{(G,w)}(\{v\})$. We define the degree of a vertex v to be the sum of the weights of hyperedges containing v—we note that the degree of a vertex is not necessarily equal to $d_{(G,w)}(v)$ since we could have $\{v\}$ itself as a hyperedge (i.e., a singleton hyperedge that contains only the vertex v). For distinct vertices $u, v \in V$, let $\lambda_{(G,w)}(u, v) := \min\{d_{(G,w)}(X) : u \in X \subseteq V \setminus \{v\}\}$ – i.e., $\lambda_{(G,w)}(u, v)$ is the value of a minimum $\{u, v\}$-cut in the hypergraph. If all hyperedge weights are unit, then we drop w from the subscript and simply use d_G and λ_G.

Hypergraph Splitting-off. We now introduce our definition of splitting-off in hypergraphs. To compare and contrast our definition of splitting-off for hypergraphs with the classical definition of splitting-off for graphs, we include both our definition and the classical definition and distinguish them by identifying them as h-splitting-off and g-splitting-off. In the definitions below, we encourage the reader to consider the input hypergraph (G, w) to be a graph while considering g-splitting-off terminology and to be a hypergraph while considering h-splitting-off terminology. We encourage the reader to also assume unit weights during first read. See Figure 1 for an example.

Definition 1.1. Let $(G = (V, E), w : E \to \mathbb{Z}_+)$ be a hypergraph and $s \in V$.

1. In merge almost-disjoint hyperedges, we pick a pair of hyperedges $e, f \in \delta_G(s)$ such that $e \cap f = \{s\}$, pick a positive integer $\alpha \in \mathbb{Z}_+$ such that $\alpha \leq \min\{w(e), w(f)\}$, reduce the weights of hyperedges e and f by α, and increase the weight of a hyperedge g by α. Here,
   (a) if we choose $g := e \cup f$, then the associated operation will be called as h-merge almost-disjoint hyperedges operation.
   (b) if we choose $g := (e \cup f) \setminus \{s\}$, then the associated operation will be called as g-merge almost-disjoint hyperedges operation.

   In the above, if $\alpha = w(e)$ (resp. if $\alpha = w(f)$), then we discard the hyperedge e (resp. hyperedge f) from the hypergraph obtained after the operation; if the hyperedge $g \notin E$, then we introduce g as a new hyperedge with weight $w(g) := 0$ before performing the weight increase on the hyperedge g.
2. In trim hyperedge operation, we pick a hyperedge $e \in \delta_G(s)$, pick a positive integer $\alpha \in \mathbb{Z}_+$, reduce the weight of the hyperedge e and increase the weight of the hyperedge $g := e \setminus \{s\}$. Here,
   (a) if we choose $\alpha \leq w(e)$, reduce the weight of the hyperedge e by α, and increase the weight of the hyperedge g by α, then the associated operation will be called as h-trim operation (if $\alpha = w(e)$, then we discard e from the hypergraph obtained after the operation; if $g \notin E$, then we add g as a new hyperedge with weight $w(g) := 0$ before performing the weight increase on the hyperedge g).
   (b) if we choose $\alpha \leq w(e)/2$, reduce the weight of the hyperedge e by 2α, and increase the weight of the hyperedge g by 2α, then the associated operation will be called as g-trim operation (if $\alpha = w(e)/2$, then we discard e from the hypergraph obtained after the operation; if $g \notin E$, then we add g as a new hyperedge with weight $w(g) := 0$ before performing the weight increase on the hyperedge g).
3. We say that a hypergraph $(H = (V, E_H), w_H : E_H \to \mathbb{Z}_+)$ is obtained by applying a
   (a) h-splitting-off operation at s from (G, w) if (H, w_H) is obtained from (G, w) by either the h-merge almost-disjoint hyperedges operation or the h-trim hyperedge operation.
   (b) g-splitting-off operation at s from (G, w) if (H, w_H) is obtained from (G, w) by either the g-merge almost-disjoint hyperedges operation or the g-trim hyperedge operation.

Certain remarks regarding the definitions are in order. Firstly, the trim operation is valuable and unique to hypergraphs. It has been used in the hypergraph literature to obtain small-sized certificates for hypergraph connectivity [20] and for certain notions of directed hypergraph connectivity [27]. We note that the trim operation has limited value in graphs—trimming an edge leads to a singleton edge and singleton edges contribute only to the degree but not to the cut value of any set. Secondly, all operations mentioned above are degree preserving for vertices $u \in V \setminus \{s\}$: both h-trim and g-trim operations preserve degrees by definition; both h-merge and g-merge almost-disjoint hyperedges operations preserve degrees due to the almost-disjoint property of the chosen hyperedges. Thirdly, all operations mentioned above do not increase the cut values of subsets $X \subseteq V \setminus \{s\}$. Thus, the relevant goal with these operations is ensuring that the cut values do not decrease too much—i.e., preserving global/local connectivity. We will be interested in repeated application of h-splitting-off operations at a vertex from a given hypergraph to isolate that vertex while preserving global/local connectivity. We define these formally next.

Definition 1.2. Let $(G = (V, E), w : E \to \mathbb{Z}_+)$ be a hypergraph and $s \in V$.

1. We say that a hypergraph $(G^* = (V, E^*), w^* : E^* \to \mathbb{Z}_+)$ is a
   (a) complete h-splitting-off at s from (G, w) if $d_{(G^*,w^*)}(s) = 0$ and (G^*, w^*) is obtained from (G, w) by repeatedly applying h-splitting-off operations at s from the current hypergraph.
   (b) complete g-splitting-off at s from (G, w) if $d_{(G^*,w^*)}(s) = 0$ and (G^*, w^*) is obtained from (G, w) by repeatedly applying g-splitting-off operations at s from the current hypergraph.
2. Let (G^*, w^*) be a complete h-splitting-off/g-splitting-off at s from (G, w). We say that (G^*, w^*)
   (a) preserves local connectivity if $\lambda_{(G^*,w^*)}(u, v) = \lambda_{(G,w)}(u, v)$ for every distinct $u, v \in V \setminus \{s\}$ and
   (b) preserves global connectivity if $\min\{\lambda_{(G^*,w^*)}(u, v) : u, v \in V \setminus \{s\}, u \neq v\} = \min\{\lambda_{(G,w)}(u, v) : u, v \in V \setminus \{s\}, u \neq v\}$.

Our first contribution in this work is the definition of h-splitting-off operations at a vertex from a hypergraph. To the best of our knowledge, this definition has not appeared in the literature before. A notion of hypergraph splitting-off motivated by hypergraph connectivity augmentation applications has been studied in the literature before [4, 8, 22]. These works have explored local connectivity preserving complete g-splitting-off at a vertex s from a hypergraph under the assumption that all hyperedges incident to the vertex s are edges (i.e., have size at most 2). In contrast, our focus is on local connectivity preserving complete h-splitting-off at a vertex s from a hypergraph without any assumption on the size of the hyperedges incident to the vertex s (i.e., the vertex s could have arbitrary-sized hyperedges incident to it). We refer the reader to Figure 1 for an example of complete h-splitting-off at a vertex from a hypergraph.

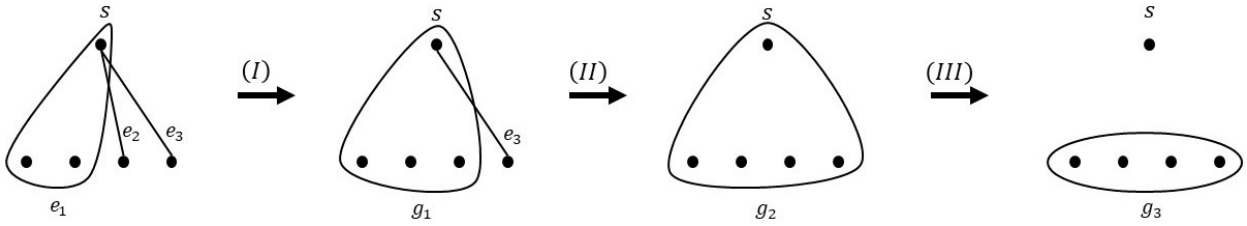


Figure 1: Example of (local connectivity preserving) complete h-splitting-off at a vertex s from a hypergraph. Consider the leftmost hypergraph where all hyperedge weights are one and the vertex s is as labeled. Operations (I) and (II) correspond to h-merge almost-disjoint hyperedges operations and Operation (III) corresponds to an h-trim hyperedge operation.

We will primarily be concerned with complete h-splitting-off at a vertex from a hypergraph and complete g-splitting-off at a vertex from a graph. Complete g-splitting-off at a vertex from a graph is equivalent to the classical and well-studied notion of complete splitting-off at a vertex from a graph (for the definition of the classical notion in graphs, see [27, 51]). We cast the results of Lovász [47, 49] and Mader [50] in the framework of our definitions now. Let (G, w) be a graph and let s be a vertex in G. Lovász [47, 49] showed that if $d_{(G,w)}(\{s\})$ is even and $\min\{\lambda_{(G,w)}(u, v) : u, v \in V \setminus \{s\}\} \geq K$ for some $K \geq 2$, then there exists a global connectivity preserving complete g-splitting-off at the vertex s from (G, w). Mader [50] showed that if $d_{(G,w)}(\{s\})$ is even, there is no cut-edge[1] incident to s, and (G, w) is connected, then there exists a local connectivity preserving complete g-splitting-off at the vertex s from (G, w).

We compare and contrast complete h-splitting-off at a vertex from a hypergraph and complete g-splitting-off at a vertex from a graph. Complete h-splitting-off at a vertex s from a hypergraph enables the vertex s to exit the hypergraph by informing its incident hyperedges about how to merge and trim themselves in order to preserve degrees. In this sense, the definition of complete h-splitting-off at a vertex from a hypergraph serves the same role as complete g-splitting-off at a vertex from a graph. On the other hand, there are

[1]Equivalently, for every edge $e \in \delta_G(s)$ with $w(e) = 1$, the removal of that edge does not disconnect the graph.

important differences between the two notions. Firstly, complete h-splitting-off at a vertex from a graph may not necessarily be a graph (owing to the creation of hyperedges of size at least 3) while it is an easy exercise to show that complete g-splitting-off at a vertex from a graph will necessarily be a graph. Secondly, local/global connectivity preserving complete g-splitting-off at a vertex from a graph may not exist—see Figure 2.

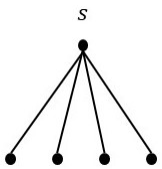


Figure 2: An example showing that global connectivity preserving complete g-splitting-off at a vertex from a graph may not exist. All edge weights are one and the vertex s is as labeled.

As our second main contribution, we show that local connectivity preserving complete h-splitting-off at a vertex from a hypergraph always exists and can be computed in strongly polynomial time (the rightmost hypergraph in Figure 1 is a local connectivity preserving complete h-splitting-off at the vertex s from the hypergraph in Figure 2).

Theorem 1.1. Given a hypergraph $(G = (V, E), w_G : E \to \mathbb{Z}_+)$ and a vertex $s \in V$, there exists a strongly polynomial-time algorithm to find a local connectivity preserving complete h-splitting-off at s from (G, w_G).

A distinction between Theorem 1.1 and the graph splitting-off results of Lovász and Mader is that Theorem 1.1 shows the existence of a local connectivity preserving complete h-splitting-off at a vertex from a hypergraph without any assumptions on the hypergraph whereas Lovász's and Mader's results hold only under certain technical assumptions on the graph. In several applications of their results, additional arguments are needed to address cases where those technical assumptions do not hold. For this reason, we believe that Theorem 1.1 could be useful in simplifying the arguments involved in some of the applications of Lovász's and Mader's graph splitting-off results (e.g., we will later see that the edge version of Menger's theorem in undirected graphs follows in a straightforward fashion from Theorem 1.1).

A crude run-time of our algorithm that proves Theorem 1.1 is $O(|V|^6(|V| + |E|)^3)$. We understand that this run-time is impractical for applications. Nevertheless, we mention it here explicitly for the sake of completeness and as a potential starting point for future work: it would be interesting to design a near-linear time algorithm to find a local connectivity preserving complete h-splitting-off at a vertex from a weighted hypergraph.

Remark 1.1. We note that existence of a local/global connectivity preserving complete h-splitting-off at a vertex from a hypergraph does not necessarily imply a polynomial-time algorithm to find it. This is because, a local/global connectivity preserving complete h-splitting-off at a vertex from a hypergraph (G, w_G) could contain exponential number of hyperedges although G contains only polynomial number of hyperedges. We give an example to illustrate this issue. Consider the graph $(G = (V, E), w_G)$ where G is the star graph on $n+1$ vertices with s being the center of the star and all edge weights are $2^{n-1} - 1$. Consider the hypergraph $(H = (V, E_H), w_H)$ such that $E_H := \{e : e \subseteq V \setminus \{s\} \text{ and } |e| \geq 2\}$ with all hyperedge weights being one. The hypergraph (H, w_H) is a local connectivity preserving complete h-splitting-off at s from (G, w_G), but (H, w_H) has exponential number of hyperedges although (G, w_G) has only n edges. In order to design a polynomial-time algorithm to find a local connectivity preserving complete h-splitting-off at a vertex from a hypergraph, a necessary step is to show the existence of a local connectivity preserving complete h-splitting-off at a vertex from a hypergraph that contains only polynomially many additional hyperedges. For the star graph (G, w_G) with edge weights $2^{n-1} - 1$ mentioned above, the hypergraph $(H' = (V, E_{H'}), w_{H'})$ containing only one hyperedge, namely $E_{H'} := \{V \setminus \{s\}\}$ with the weight of that hyperedge being $2^{n-1} - 1$ is also a local connectivity preserving complete h-splitting-off at s from (G, w_G). One of the features of our algorithmic

proof of Theorem 1.1 is the existence of a local connectivity preserving complete h-splitting-off at a vertex from a hypergraph that contains only polynomially many additional hyperedges.

As our third main contribution, we present two applications of Theorem 1.1.

Application 1: Constructive characterization of k-hyperedge-connected hypergraphs. For the purposes of this application, graphs and hypergraphs will refer to multi-graphs and multi-hypergraphs, respectively. Let k be a positive integer. A graph $G = (V, E)$ is k-edge-connected if $d_G(X) \geq k$ for every non-empty proper subset $X \subsetneq V$. Constructive characterization of k-edge-connected graphs is a central problem in graph theory. It is well-known that a graph is 1-edge-connected if and only if it admits a spanning tree. Robbins' [58] showed that a graph is 2-edge-connected if and only if it admits an ear decomposition (see [27] for definition of ear decomposition). Generalizing Robbins' result, Lovász [47,49] gave a constructive characterization of k-edge-connected graphs for even k using his result on global connectivity preserving complete g-splitting-off at a vertex from a graph. Mader [50] gave a constructive characterization of k-edge-connected graphs for odd k using his result on local connectivity preserving complete g-splitting-off at a vertex from a graph. Motivated by these results, we present a constructive characterization of k-hyperedge-connected hypergraphs using our splitting-off result in Theorem 1.1. A hypergraph $G = (V, E)$ is defined to be k-hyperedge-connected if $d_G(X) \geq k$ for every non-empty proper subset $X \subsetneq V$.

Both Lovász's and Mader's constructive characterizations of k-edge-connected graphs are based on a pinching operation in graphs. Our constructive characterization of k-hyperedge-connected hypergraphs is also based on a pinching operation, but our pinching operation is defined for hypergraphs. We define this operation now (see Figure 3 for an example).

Definition 1.3. Let $G = (V, E)$ be a hypergraph and $p, k \in \mathbb{Z}_+$ be positive integers such that $p \leq k$. In (k, p)-pinching hyperedges of G, we obtain a new hypergraph by performing the following sequence of operations:

1. pick p distinct hyperedges $e_1, \ldots, e_p \in E$,
2. pick p positive integers $t_1, \ldots, t_p \in \mathbb{Z}_+$ such that $\sum_{i=1}^{p} t_i = k$,
3. for each $i \in [p]$, choose a partition of the hyperedge e_i into t_i non-empty parts $e_i = \uplus_{j \in [t_i]} f_i^j$,
4. remove the hyperedges $e_1, \ldots, e_p$ from the hypergraph G,
5. add a new vertex s and hyperedges $\{f_i^j \cup \{s\} : j \in [t_i], i \in [p]\}$ to the hypergraph G.

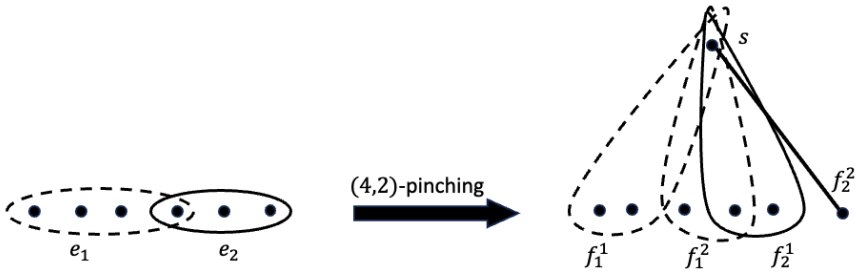


Figure 3: An example of a $(4, 2)$-pinching operation. Here, $t_1 = t_2 = 2$.

With this definition of pinching, we show the following constructive characterization of k-hyperedge-connected hypergraphs.

Theorem 1.2. Let $k \in \mathbb{Z}_+$ be a positive integer. A hypergraph $G = (V, E)$ is k-hyperedge-connected if and only if G can be obtained by starting from the single vertex hypergraph with no hyperedges and repeatedly applying one of the following two operations:

1. add a new hyperedge over a subset of vertices of the existing hypergraph, and

2. (k,p)-pinching hyperedges of the existing hypergraph for some positive integer $p \leq k$.

Our proof of Theorem 1.2 is constructive: i.e., given a k-hyperedge-connected hypergraph G, our proof gives a polynomial-time algorithm to construct a sequence of hypergraphs $G_0, G_1, G_2, \ldots, G_t$, where G_0 is the single vertex hypergraph with no hyperedges, $G_t = G$ and for each $i \in [t]$, the hypergraph G_i is obtained from G_{i-1} by either adding a new hyperedge over a subset of vertices in G_{i-1} or by (k,p)-pinching hyperedges in G_{i-1} for some positive integer $p \leq k$.

Remark 1.2. Robbins' constructive characterization of 2-edge-connected graphs and Lovász's constructive characterization of $2k$-edge-connected graphs find applications in graph orientation problems—e.g., Robbins' result leads to an algorithm to find a strongly connected orientation of 2-edge-connected graphs and Lovász's result leads to an algorithm to find a strongly k-arc-connected orientation of $2k$-edge-connected graphs. In fact, the latter leads to an alternative proof of Nash-Williams' weak orientation theorem [54]. Along the same vein, we hope that our above characterization of k-edge-connected hypergraphs might find applications in hypergraph orientation problems.

Application 2.1: Steiner Rooted k-arc-connected Orientation of Graphs. Orienting a graph to achieve high connectivity is a fundamental area in graph theory, combinatorial optimization, and algorithms. Let $G = (V, E)$ be an undirected graph. An orientation $\overrightarrow{G}$ of G is a directed graph obtained by assigning a direction to each edge of G. Let $G = (V, E)$ be an undirected graph, $T \subseteq V$ be a set of terminals, $r \in T$ be a root vertex, and k be a positive integer. An orientation $\overrightarrow{G}$ of G is defined to be Steiner rooted k-arc-connected if there exist k arc-disjoint paths in $\overrightarrow{G}$ from t to r for every terminal $t \in T \setminus \{r\}$. In Max Steiner Rooted-Connected Orientation problem, the goal is to find the maximum integer k and an orientation $\overrightarrow{G}$ of G such that $\overrightarrow{G}$ is Steiner rooted k-arc-connected. Max Steiner Rooted-Connected Orientation generalizes two classic problems in graph theory: The case of $|T| = 2$ is the max edge-disjoint $\{r, v\}$-paths problem and is solved via Menger's theorem. The case of $T = V$ is the max edge-disjoint spanning trees problem and is solved via Tutte and Nash-Williams' theorem [55, 61]. We mention that both these problems are also generalized by the Steiner Tree Packing problem and the associated Kriesell's conjecture [35, 42, 43], but we will not focus on that generalization.

Király and Lau [40] introduced the Max Steiner Rooted-Connected Orientation, showed that it is NP-hard, and gave a 2-approximation via an approximate min-max relation. We state their approximate min-max relation now. An undirected graph G is Steiner k-edge-connected if $\lambda_G(u, v) \geq k$ for every pair of distinct terminals $u, v \in T$. It is clear that if the graph G has a Steiner rooted k-arc-connected orientation, then G should be Steiner k-edge-connected. However, the converse is not necessarily true. Király and Lau observed that if the graph is Steiner $2k$-edge-connected, then it has a Steiner rooted k-arc-connected orientation.

Theorem 1.3 (Király and Lau [40]). Let $G = (V, E)$ be an undirected graph, $T \subseteq V$ be a subset of terminals, $r \in T$ be the root vertex, and k be a positive integer. If G is Steiner $2k$-edge-connected, then it has a Steiner rooted k-arc-connected orientation.

Király and Lau observed that Theorem 1.3 follows immediately from Nash-Williams' strong orientation theorem. Nash-Williams' strong orientation theorem [54] states that every undirected graph $G = (V, E)$ admits an orientation $\overrightarrow{G}$ such that $\lambda_{\overrightarrow{G}}(u, v) \geq \lfloor \lambda_G(u, v)/2 \rfloor$ for every distinct $u, v \in V$, where $\lambda_{\overrightarrow{G}}(u, v)$ is the maximum number of arc-disjoint directed paths from u to v in $\overrightarrow{G}$. In this work, we give an alternative proof of Theorem 1.3 that does not rely on Nash-Williams' strong orientation theorem. Instead, we use Theorem 1.1. Our proof strategy is unique since it proves an orientation result for graphs using tools developed for hypergraphs.

Remark 1.3. Nash-Williams' proof of the strong orientation theorem [54] is a sophisticated inductive argument. Giving a simple and more insightful proof of the strong orientation theorem has been a central topic

of interest in graph theory and combinatorial optimization (see [25]). Mader [50] gave a different proof of the strong orientation theorem using his local connectivity preserving splitting-off theorem, but his proof also involved sophisticated technical arguments. Frank [25] condensed the ideas of both Nash-Williams and Mader to present a proof of the strong orientation theorem using Mader's local connectivity preserving splitting-off, but it is still technically complicated. The technical complication in using Mader's local connectivity preserving splitting-off result arises from the assumptions that need to be satisfied by the vertex to be split-off. In contrast, our splitting-off result for hypergraphs (namely, Theorem 1.1) does not need any assumptions on the vertex to be split-off. In light of these considerations, our proof of Theorem 1.3 using Theorem 1.1 provides hope that Theorem 1.1 (or the ideas therein) could be used to give a conceptually simpler proof of Nash-Williams' strong orientation theorem.

Application 2.2: Steiner Rooted k-hyperarc-connected Orientation of Hypergraphs. Orienting hypergraphs is also a fundamental area in graph theory and combinatorial optimization (see Frank's book [27]) with far reaching implications. For example, Woodall's conjecture can be reformulated as a hypergraph orientation problem [1]; moreover, hypergraph orientation results have recently been used in coding theory [2]. Király and Lau [40] showed that the approximate min-max relation in Theorem 1.3 also holds for hypergraphs. To state their result, we need some terminology in hypergraph orientations.

Let $G = (V, E)$ be a hypergraph. An orientation $\overrightarrow{G} = (V, E, \texttt{head} : E \to V)$ of G is a directed hypergraph obtained by assigning a unique head vertex $\texttt{head}(e) \in e$ for each $e \in E$. A pair $(e, \texttt{head}(e))$ is denoted as a hyperarc with the head of the hyperarc being $\texttt{head}(e)$ and the tails of the hyperarc being $e \setminus \texttt{head}(e)$. Let $G = (V, E)$ be a hypergraph, $T \subseteq V$ be a set of terminals, $r \in T$ be a root vertex, and k be a positive integer. An orientation $\overrightarrow{G}$ of G is defined to be Steiner rooted k-hyperarc-connected if there exist k hyperarc-disjoint paths in $\overrightarrow{G}$ from t to r for every terminal $t \in T \setminus \{r\}$. Here, a path from t to r in a directed hypergraph is an alternating sequence of distinct vertices and hyperarcs $t = v_1, a_1, v_2, a_2, ..., a_{\ell-1}, v_\ell = r$ such that v_i is a tail of a_i and v_{i+1} is the head of a_i for every $i \in [\ell - 1]$. We say that a hypergraph G is Steiner k-hyperedge-connected if $\lambda_G(u, v) \geq k$ for every pair of distinct terminals $u, v \in T$. It is clear that if the hypergraph G has a Steiner rooted k-hyperarc-connected orientation, then G should be Steiner k-hyperedge-connected. However, the converse is not necessarily true. Király and Lau [40] showed that if the hypergraph is Steiner $2k$-hyperedge-connected, then it has a Steiner rooted k-hyperarc-connected orientation.

Theorem 1.4 (Király and Lau [40]). Let $G = (V, E)$ be a hypergraph, $T \subseteq V$ be a subset of terminals, $r \in T$ be the root vertex, and k be a positive integer. If G is Steiner $2k$-hyperedge-connected, then it has a Steiner rooted k-hyperarc-connected orientation.

Király and Lau's proof of Theorem 1.4 was based on careful uncrossing and contractions. In this work, we give an alternative proof of Theorem 1.4 using Theorem 1.1. Our proof of Theorem 1.4 reveals the source of the 2-factor gap in the approximate min-max relation of Király and Lau for Max Steiner Rooted-Connected Orientation Problem: it arises from the 2-factor gap between connectivity and weak-partition-connectivity of hypergraphs (see Definition 5.1 for the definition of weak-partition-connectivity and Lemma 5.2).

Remark 1.4. Our proof technique for Theorems 1.3 and 1.4 using Theorem 1.1—i.e., via the local-connectivity preserving splitting-off operation in hypergraphs—also leads to an alternate proof of Menger's theorem in undirected graphs and hypergraphs (edge-disjoint version). For details, we refer the reader to Section 5 where we discuss a proof of Menger's theorem using Theorem 1.1 as a warm-up towards a proof of Theorems 1.3 and 1.4.

Both Theorems 1.3 and 1.4 can be extended to weighted graphs/hypergraphs (by considering parallel copies of edges/hyperedges). The weighted version of Theorems 1.3 and 1.4 can also be shown to admit strongly polynomial-time algorithms using our proof strategy as well as the proof strategy of Király and Lau. We avoid stating the weighted versions in the interests of brevity.

1.1 Proof Technique for Theorem 1.2

We outline the proof technique for Theorem 1.2. The reverse direction follows by observing that if a hypergraph is k-hyperedge-connected, then both operations in the statement of the theorem preserve k-hyperedge-connectivity. We sketch a proof of the forward direction. The proof is by induction on the number of hyperedges plus the number of vertices. First, suppose that there exists a hyperedge $e \in E$ such that $G - e$ is still k-hyperedge-connected. We note that deleting the hyperedge e is the inverse of operation (1). Consequently, the proof follows by deleting the hyperedge e, using the induction hypothesis on the resulting hypergraph H, and then noting that the hypergraph G is obtained from H by operation (1). Next, suppose that there does not exist a hyperedge $e \in E$ such that $G - e$ is k-hyperedge-connected. We call such a hypergraph to be minimally k-hyperedge-connected. In Lemma 4.1, we show that a minimally k-hyperedge-connected hypergraph contains a vertex u with degree exactly k. By Theorem 1.1, there exists a global-connectivity preserving complete h-splitting-off at the vertex u from the hypergraph G. Let H be a global-connectivity preserving complete h-splitting-off at the vertex u from the hypergraph G. We note that complete h-splitting-off at u followed by deletion of the vertex u is the inverse of operation (2) at u. Consequently, the proof follows by using the induction hypothesis on the hypergraph $H - u$ and then noting that the hypergraph G is obtained from $H - u$ by operation (2).

1.2 Proof Technique for Theorems 1.3 and 1.4

We outline the proof technique for Theorem 1.4 and will remark after the proof about how it also implies a proof for Theorem 1.3. Our proof of Theorem 1.4 will be in three steps. Let us denote the set of non-terminals as Steiner vertices. Our first step is to obtain a hypergraph $H = (T, E_H)$ by applying our local connectivity preserving complete h-splitting-off at each Steiner vertex of G (sequentially, in arbitrary order of the Steiner vertices) and deleting the isolated vertices. We note that deleting the isolated vertices ensures that the vertex set of H is the set of terminals T. Moreover, our local connectivity preserving complete h-splitting-off ensures that the hypergraph H is $2k$-hyperedge-connected since the hypergraph G is Steiner $2k$-hyperedge-connected. Our second step is to show that this hypergraph $H = (T, E_H)$ admits a rooted k-hyperarc-connected orientation. A known characterization for the existence of a rooted k-hyperarc-connected orientation of a hypergraph is that the hypergraph is k-weak-partition-connected (see Definition 5.1 for the definition of weak-partition-connectivity and Theorem 5.1 for the characterization). We mention that the notion of weak-partition-connectivity in hypergraphs has been used recently in the context of coding theory [2,33]. In order to use the characterization for the existence of a rooted k-hyperarc-connected orientation of a hypergraph, we relate the connectivity of a hypergraph to its weak-partition-connectivity and conclude that if H is $2k$-hyperedge-connected, then it is k-weak-partition-connected (see Lemma 5.2). Consequently, the hypergraph H admits a rooted k-hyperarc-connected orientation. We note that such an orientation of H is equivalent to a Steiner rooted k-hyperarc-connected orientation of H since the vertex set of H is the set T of terminals. Our third step is to use this Steiner rooted k-hyperarc-connected orientation of H to obtain a Steiner rooted k-hyperarc-connected orientation of the hypergraph G: we will see that there is a natural way to extend the orientation of hyperedges while reversing the h-splitting-off operations to preserve Steiner rooted k-hyperarc-connected property (see Lemma 5.1). This would complete the proof of Theorem 1.4.

We note that if the hypergraph G is a graph, then the same proof above obtains the required graph orientation, thus proving Theorem 1.3. In particular, to prove Theorem 1.3, we start from a graph $G = (V, E)$ that is Steiner $2k$-edge-connected, but our local connectivity preserving complete h-splitting-off operations at Steiner vertices results in a hypergraph $H = (T, E_H)$ that is $2k$-hyperedge-connected; by Lemma 5.2, the hypergraph H is k-weak-partition-connected; now Theorem 5.1 gives a rooted k-hyperarc-connected orientation of the resulting hypergraph. Such an orientation is extended to a Steiner rooted k-hyperarc-connected orientation of the graph $G = (V, E)$ using Lemma 5.1. Essentially, the proof starts from the given graph, obtains a related hypergraph, orients that hypergraph, and extends that orientation of the hypergraph back into a desired orientation of the given graph.

Our proof technique for Theorems 1.3 and 1.4 also leads to an alternate proof of Menger's theorem in

undirected graphs and hypergraphs (edge-disjoint version)—see Section 5.

1.3 Proof Technique for Theorem 1.1

We prove a more general statement that implies Theorem 1.1. We begin with the definitions needed for the more general statement.

Definition 1.4. Let V be a finite set, $p : 2^V \to \mathbb{Z}$ be a set function, and $(H = (V, E), w : E \to \mathbb{Z}_+)$ be a hypergraph.

1. The set function p
   (a) is symmetric if $p(X) = p(V - X)$ for every $X \subseteq V$, and
   (b) is skew-supermodular if for every $X, Y \subseteq V$, at least one of the following inequalities hold:
      i. $p(X) + p(Y) \leq p(X \cap Y) + p(X \cup Y)$.
      ii. $p(X) + p(Y) \leq p(X - Y) + p(Y - X)$.
2. The coverage function $b_{(H,w)} : 2^V \to \mathbb{Z}_{\geq 0}$ is defined by $b_{(H,w)}(X) := \sum_{e \in B_H(X)} w(e)$ for every $X \subseteq V$, where $B_H(X) := \{e \in E : e \cap X \neq \emptyset\}$ for every $X \subseteq V$.
3. The hypergraph (H, w) weakly covers the function p if $b_{(H,w)}(X) \geq p(X)$ for every $X \subseteq V$.
4. The hypergraph (H, w) strongly covers the function p if $d_{(H,w)}(X) \geq p(X)$ for every $X \subseteq V$.

If a hypergraph (H, w) strongly covers a function $p : 2^V \to \mathbb{Z}$, then it also weakly covers the function p. However, the converse is false – i.e., a weak cover is not necessarily a strong cover[2]. Bernáth and Király [7] showed that a weak cover of a symmetric skew-supermodular function can be converted to a strong cover of the same function by repeated merging of disjoint hyperedges. We recall their definition of the merging operation, discuss their result, and its significance now.

Definition 1.5. Let $(H = (V, E), w : E \to \mathbb{Z}_+)$ be a hypergraph. We use H_w to denote the unweighted multi-hypergraph over vertex set V containing $w(e)$ copies of every hyperedge $e \in E$. By merging two disjoint hyperedges of H_w, we refer to the operation of replacing them by their union in H_w. We will say that a hypergraph $(G = (V, E_G), c : E_G \to \mathbb{Z}_+)$ is obtained from (H, w) by merging hyperedges if the multi-hypergraph G_c is obtained from the multi-hypergraph H_w by repeatedly merging two disjoint hyperedges in the current hypergraph.

Bernáth and Király showed the following result:

Theorem 1.5 (Bernáth and Király [7]). Let $(H = (V, E), w : E \to \mathbb{Z}_+)$ be a hypergraph and $p : 2^V \to \mathbb{Z}$ be a symmetric skew-supermodular function such that $b_{(H,w)}(X) \geq p(X)$ for every $X \subseteq V$. Then, there exists a hypergraph $\left(H^* = (V, E^*), w^* : E^* \to \mathbb{Z}_+\right)$ such that

(1) $d_{(H^*,w^*)}(X) \geq p(X)$ for every $X \subseteq V$ and

(2) the hypergraph (H^*, w^*) is obtained by merging hyperedges of the hypergraph (H, w).

We observe that Theorem 1.5 can be used to prove the existential version of Theorem 1.1: namely, for every hypergraph $(G = (V, E), w_G : E \to \mathbb{Z}_+)$ and a vertex $s \in V$, there exists a local connectivity preserving complete h-splitting-off at s from (G, w_G). This can be shown by setting up the hypergraph (H, w) and the function p suitably based on (G, w_G) and using Theorem 1.5 (see the first two paragraphs of the proof of Theorem 3.1 in Section 3). We emphasize that this conclusion regarding hypergraph splitting-off from Bernáth and Király's result was not known before in the literature and is one of our contributions.

[2]For example, consider the function $p : 2^V \to \mathbb{Z}$ defined by $p(X) := 1$ for every non-empty proper subset $X \subsetneq V$ and $p(\emptyset) := p(V) := 0$, and the hypergraph $(H = (V, E := \{\{u\} : u \in V\}), w : E \to \{1\})$.

Remark 1.5. We were also able to prove the existential version of Theorem 1.1 using element-connectivity preserving reduction operations [6] (see [18] for the definition of element-connectivity and the notion of element-connectivity preserving reduction operations) – we omit the details of this alternate proof in the interests of brevity. The alternate proof does not seem to be helpful for the purposes of a strongly polynomial time algorithm. In fact, it remains open to design a strongly polynomial-time algorithm to perform complete element-connectivity preserving reduction operations in the weighted setting [18].

We recall that existence of a local connectivity preserving complete h-splitting-off at a vertex from a hypergraph does not immediately imply a polynomial-time algorithm—see the example in Remark 1.1. However, the above-mentioned proof of existence of a local-connectivity preserving complete splitting-off at an arbitrary vertex from a hypergraph (i.e., existential version of Theorem 1.1) via Theorem 1.5 suggests a natural approach towards designing a strongly polynomial time algorithm to find a local-connectivity preserving complete splitting-off at a given vertex from a given hypergraph: it suffices to prove a constructive version of Theorem 1.5 via a strongly polynomial-time algorithm. Towards this end, the example in Remark 1.1 suggests a necessary structural step towards a strongly polynomial-time algorithmic version of Theorem 1.5: we need to show Theorem 1.5 with the extra conclusion that the number of additional hyperedges in H^* is polynomial in the number of hyperedges and vertices in H.

Bernáth and Király proved Theorem 1.5 in the context of a reduction between certain hypergraph connectivity augmentation problems. For that reduction, the existential version of Theorem 1.5 is sufficient. However, for the purposes of our application to hypergraph splitting-off, we need an algorithmic version of Theorem 1.5. Bernáth and Király's proof of Theorem 1.5 is in fact algorithmic, but the run-time of the associated algorithm is not necessarily polynomial. Their proof implies that the number of additional hyperedges in the hypergraph (H^*, c^*) returned by their algorithm is at most $\sum_{e\in E} w(e)$ (i.e., $|E^*| - |E| \leq \sum_{e\in E} w(e)$) and the run-time of the algorithm is $O(\sum_{e\in E}(|e| + w(e))$. In particular, their run-time is polynomial only if the input weights are given in unary. Moreover, the exponential-sized hypergraph given in the example in Remark 1.1 could indeed arise as a consequence of their algorithm.

We address both the structural and the algorithmic issues mentioned above by proving a stronger algorithmic version of Theorem 1.5. In order to phrase an algorithmic version of Theorem 1.5, we need suitable access to the function p. Bernáth and Király [7] suggested access to a certain function maximization oracle associated with the function p that we describe below.

Definition 1.6. Let $p : 2^V \to \mathbb{Z}$ be a set function. p-`max-sc-Oracle` $\left((G_0, c_0), S_0, T_0\right)$ takes as input a hypergraph $(G_0 = (V, E_0), c_0 : E_0 \to \mathbb{Z}_+)$ and disjoint sets $S_0, T_0 \subseteq V$, and returns a tuple $(Z, p(Z))$, where Z is an optimum solution to the following problem:

$$\max \left\{ p(Z) - d_{(G_0,c_0)}(Z) : S_0 \subseteq Z \subseteq V - T_0 \right\}. \qquad (p\text{-}\texttt{max-sc-Oracle})$$

For the purposes of our application (namely local connectivity preserving complete h-splitting-off at a vertex from a hypergraph), the above-mentioned function maximization oracle can be implemented to run in strongly polynomial time (see Lemma 3.1). Using the above mentioned oracle, we prove the following algorithmic version of Theorem 1.5.

Theorem 1.6. Let $(H = (V, E), w : E \to \mathbb{Z}_+)$ be a hypergraph and $p : 2^V \to \mathbb{Z}$ be a symmetric skew-supermodular function such that $b_{(H,w)}(X) \geq p(X)$ for every $X \subseteq V$. Then, there exists a hypergraph $\left(H^* = (V, E^*), w^* : E^* \to \mathbb{Z}_+\right)$ such that

(1) $d_{(H^*,w^*)}(X) \geq p(X)$ for every $X \subseteq V$,

(2) the hypergraph (H^*, w^*) is obtained by merging hyperedges of the hypergraph (H, w), and

(3) $|E^*| - |E| = O(|V|)$.

Furthermore, given a hypergraph $(H = (V, E), w : E \to \mathbb{Z}_+)$ and access to p-`max-sc-Oracle` of a symmetric skew-supermodular function $p : 2^V \to \mathbb{Z}$ where $b_{(H,w)}(X) \geq p(X)$ for every $X \subseteq V$, there exists an algorithm

that runs in $O(|V|^3(|V|+|E|)^2)$ time using $O(|V|^3(|V|+|E|))$ queries to p-`max-sc-Oracle` and returns a hypergraph $\left(H^* = (V, E^*), w^* : E^* \to \mathbb{Z}_+\right)$ satisfying the above three properties. The run-time includes the time to construct the hypergraphs used as input to the queries to p-`max-sc-Oracle`. Moreover, for each query to p-`max-sc-Oracle`, the hypergraph (G_0, c_0) used as input to the query has $O(|V|)$ vertices and $O(|V|+|E|)$ hyperedges.

Theorem 1.6 is a strengthening of Theorem 1.5 in two ways. Firstly, our theorem shows the existence of a hypergraph that not only satisfies properties (1) and (2), but also satisfies property (3) – i.e., the number of additional hyperedges in the returned hypergraph is linear in the size of the vertex set. Secondly, our Theorem 1.6 shows the existence of a strongly polynomial-time algorithm that returns a hypergraph satisfying the three properties. Our main contribution is modifying Bernáth and Király's algorithm and analyzing the modified algorithm to bound the number of additional hyperedges and the run-time. We mention that property (3) cannot be tightened to guarantee that $|E^* - E| = O(|V|)$ – we were able to construct an example where $|E^* - E| = \Omega(|V|^2)$ (see Section A).

Theorem 1.6 immediately leads to a proof of Theorem 1.1 (see Theorem 3.1 and its proof in Section 3). Instead of using Theorem 1.6 as a black-box, if we delve into the proof of it in the context of the proof of Theorem 1.1, we obtain the following theorem:

Theorem 1.7. Let $(G = (V, E), w : E \to \mathbb{Z}_+)$ be a hypergraph and $s \in V$. Then, there exists a hypergraph $(G' = (V, E'), w' : E' \to \mathbb{Z}_+)$ obtained by applying a h-splitting-off operation at s from (G, w) such that $\lambda_{(G',w')}(u, v) = \lambda_{(G,w)}(u, v)$ for every distinct $u, v \in V \setminus \{s\}$.

We omit the proof of Theorem 1.7 in the interests of brevity. Theorem 1.7 closely resembles the existential edge splitting-off results of Lovász [47, 49] and Mader [50] for graphs. Lovász's and Mader's existential edge splitting-off results for graphs are important since they have been used to simplify the proofs of fundamental results in graph theory—e.g., Nash-Williams' Strong Orientation Theorem. On the other hand, Theorem 1.7 does not immediately imply a strongly polynomial-time algorithm for finding a local connectivity preserving complete h-splitting off at a vertex from a given weighted hypergraph. So, Theorem 1.1 may be useful in algorithmic contexts while Theorem 1.7 may be useful in graph-theoretical contexts.

1.4 Proof Technique for Theorem 1.6

In this section, we describe our proof technique for the existential result in Theorem 1.6. The algorithmic results in that theorem follow from the existential result using standard algorithmic tools for submodular functions, so we focus only on outlining a proof of the existential result. Let $(H = (V, E), w : E \to \mathbb{Z}_+)$ be a hypergraph and $p : 2^V \to \mathbb{Z}_+$ be a symmetric skew-supermodular function such that (H, w) weakly covers the function p. Our goal is to show that there exists a hypergraph $(H^* = (V, E^*), w^* : E \to \mathbb{Z}_+)$ such that

(1) (H^*, w^*) strongly covers the function p,

(2) (H^*, w^*) is obtained by merging hyperedges of the hypergraph (H, w), and

(3) $|E^*| - |E| = O(|V|)$.

Preliminaries. We define a set $X \subseteq V$ to be (p, H, w)-tight if $b_{(H,w)}(X) = p(X)$. For a function p and hypergraph (H, w), let $T_{p,H,w}$ denote the family of (p, H, w)-tight sets and let $\mathcal{T}_{p,H,w}$ be the family of inclusionwise maximal sets in $T_{p,H,w}$. We will need the following two operations:

(i) For hyperedges $e, f \in E$ and a positive integer $\alpha \leq \min\{w(e), w(f)\}$, the operation Merge $((H, w), e, f, \alpha))$ returns the hypergraph obtained from (H, w) by decreasing the weight of hyperedges e and f by α and increasing the weight of the hyperedge $e \cup f$ by α. All hyperedges with zero weight are discarded.

(ii) For a hyperedge $e \in E$ and a positive integer $\alpha \leq w(e)$, the operation Reduce $((H, w), e, \alpha)$ returns the hypergraph obtained by decreasing the weight of the hyperedge e by α. All hyperedges with zero weight are discarded.

Algorithm of [7]. Our proof of the existential result builds on the techniques of Bernáth and Király [7] who proved the existence of a hypergraph $(H^* = (V, E^*), w^* : E^* \to \mathbb{Z}_+)$ satisfying properties (1) and (2), so we briefly recall their techniques. We present the algorithmic version of their proof since it will be useful for our purposes.

The proof in [7] is inductive, and consequently, the algorithm implicit in their proof is recursive. The algorithm takes as input a hypergraph $((H = (V, E), w : E \to \mathbb{Z}_+)$ and a symmetric skew-supermodular function $p : 2^V \to \mathbb{Z}$ such that the hypergraph (H, w) weakly covers the function p. If $w(E) = 0$, then the algorithm is in its base case and returns the empty hypergraph. Otherwise, $w(E) > 0$; the algorithm chooses an arbitrary hyperedge $e \in E$ and defines hypergraphs (H_0, w_0) and (H', w') and the function p' by considering two cases. First, suppose that the hyperedge e is not contained in any set of the family $\mathcal{T}_{p,H,w}$. In this case, the algorithm defines (H_0, w_0) to be the hypergraph on vertex set V consisting of a single hyperedge e with $w_0(e) = 1$, constructs the hypergraph $(H', w') := \text{Reduce}((H, w), e, 1)$, and defines the function $p' := p - d_{(H_0,w_0)}$. Second, suppose that the hyperedge e is contained in some set $X \in \mathcal{T}_{p,H,w}$. It can be shown that there exists a hyperedge $f \in E$ such that $f \subseteq V - X$. In this case, the algorithm defines (H_0, w_0) to be the empty hypergraph on vertex set V, constructs the hypergraph $(H', w') := \text{Merge}((H, w), e, f, 1)$, and defines the function $p' := p$. In both cases, the algorithm recurses on the inputs (H', w') and p' to obtain a hypergraph (H_0^*, w_0^*) and returns the hypergraph $(H^*, w^*) = (H_0^* + H_0, w_0^* + w_0)$. Here, the hyperedges of H^* are the union of the hyperedges of H_0^* and H_0 with the weight $w^*(e)$ of a hyperedge e being the sum of the weights $w_0^*(e) + w_0(e)$ if the hyperedge e is present in both H_0^* and H_0, being $w_0^*(e)$ if the hyperedge e is present only in H_0^*, and being $w_0(e)$ if the hyperedge e is present only in H_0.

We note that $w'(E') = w(E) - 1$. Furthermore, it can be shown that the function p' is symmetric skew-supermodular and the hypergraph (H', w') weakly covers the function p'. Consequently, by induction on $w(E)$, the algorithm can be shown to terminate in $w(E)$ recursive calls and returns a hypergraph satisfying properties (1) and (2). Moreover, the number of additional hyperedges in the returned hypergraph is at most the number of recursive calls where the Merge operation is performed, which is also at most $w(E)$. Thus, in order to reduce the number of additional hyperedges and to design a strongly polynomial-time algorithm, our goal is to reduce the recursion depth of the algorithm. We emphasize that the recursion depth of Bernáth and Király's algorithm could indeed be exponential (the exponential sized example mentioned in Remark 1.1 could arise in the execution of their algorithm), so we need to necessarily modify their algorithm.

Preprocessing for Additional Structure. Similar to Bernáth and Király's algorithm, our algorithm also takes as input a hypergraph $(H = (V, E), w : E \to \mathbb{Z}_+)$ and a symmetric skew-supermodular function $p : 2^V \to \mathbb{Z}$ such that the hypergraph (H, w) weakly covers the function p. However, unlike Bernáth and Király's algorithm, our algorithm performs a preprocessing step so that the inputs (H, w) and p satisfy the following two additional conditions:

(a) every hyperedge $e \in E$ is contained in some set of $\mathcal{T}_{p,H,w}$ and

(b) the degree of every vertex in (H, w) is non-zero.

As a consequence of these additional conditions, the family $\mathcal{T}_{p,H,w}$ will be a disjoint family, a property that we leverage heavily throughout our analysis. Furthermore, we modify Bernáth and Király's algorithm to ensure that these conditions hold during every recursive call.

Our Algorithm. We now describe our modification of the above-mentioned Bernáth and Király's algorithm to reduce the recursion depth. Our algorithm is also recursive and its base case is the same as that of Bernáth and Király's algorithm (i.e., $w(E) = 0$). During recursive cases (i.e., if $w(E) > 0$), instead of performing one of the two (i.e., Merge or Reduce) operations, our algorithm performs both operations in a sequential fashion. In particular, we find a pair of disjoint hyperedges $e, f \in E$ contained in distinct sets of $\mathcal{T}_{p,H,w}$ (such a pair exists by condition (a) and the arguments of Bernáth and Király mentioned above). Next, instead of performing one Merge operation (as was done by Bernáth and Király's algorithm), we perform as many Merge operations as possible using the hyperedges e and f. Formally, let

$$\alpha^M := \max\left\{\alpha \in \mathbb{Z}_+ : \text{hypergraph returned by Merge}((H, w), e, f, \alpha) \text{ weakly covers } p\right\}.$$

We let $(H^M, w^M) := \text{Merge}((H, w), e, f, \alpha^M)$ and $p^M := p$. Next, instead of recursing on $((H^M, w^M), p^M)$ (as was done by Bernáth and Király's algorithm), we perform as many Reduce operations as possible on the newly created hyperedge $e \cup f$. Formally, let

$$\alpha^R := \max\left\{\alpha \in \mathbb{Z}_{\geq 0} : \text{hypergraph returned by Reduce}((H^M, w^M), e \cup f, \alpha) \text{ weakly covers } (p^M - d_{(H_0^\alpha, w_0^\alpha)})\right\},$$

where (H_0^α, w_0^α) denotes the hypergraph on vertex set V consisting of a single hyperedge $e \cup f$ with $w_0^\alpha(e \cup f) = \alpha$. We construct the hypergraph $(H_0, w_0) := (H_0^{\alpha^R}, w_0^{\alpha^R})$, the hypergraph $(H^R, w^R) := \text{Reduce}((H^M, w^M), e \cup f, \alpha^R)$ and define the function $p^R := p^M - d_{(H_0^{\alpha^R}, w_0^{\alpha^R})}$. This immediate reduce step ensures that the hypergraph (H^R, w^R) and the function p^R satisfy condition (a) – i.e., every hyperedge in (H^R, w^R) is contained in some set of $\mathcal{T}_{p^R, H^R, w^R}$. Finally, we compute sets $\mathcal{Z} := \{u \in V : b_{(H^R, w^R)}(u) = 0\}$ and $V' := V - \mathcal{Z}$, hypergraph $(H' := (V', E' := E^R), w' := w^R)$, and define the function $p' : 2^{V'} \to \mathbb{Z}$ by $p'(X) := \max\{p(X \cup R) : R \subseteq \mathcal{Z}\}$ for every $X \subseteq V'$ – this final step can be viewed as a clean up step since it gets rid of vertices that are not incident to any hyperedges (and revises the function p appropriately). This clean up step ensures that the hypergraph (H', w') satisfies condition (b) – i.e., the degree of every vertex in (H', w') is non-zero. It can be shown that the function p' is symmetric skew-supermodular and the hypergraph (H', w') weakly covers the function p'. Furthermore, the function p' and hypergraph (H', w') satisfy conditions (a) and (b). We recursively call the algorithm on input $((H', w'), p')$ to obtain a hypergraph (H_0^*, w_0^*). We obtain the hypergraph (G, c) from (H_0^*, w_0^*) by adding the vertices $\mathcal{Z}$ and return the hypergraph $(G + H_0, c + w_0)$. By induction on the total weight of hyperedges in the input hypergraph, it can be shown that our algorithm returns a hypergraph satisfying properties (1) and (2) and also terminates within a finite number of recursive calls.

Recursion Depth and Potential Functions. We now sketch our proof to show that the recursion depth of our algorithm is $|E| + O(|V|)$. We note that this also bounds the number of additional hyperedges in the hypergraph returned by the algorithm, and consequently this hypergraph also satisfies property (3). Let ℓ be the number of recursive calls made by the algorithm on the input instance $((H, w), p)$. We partition the set $[\ell]$ of recursive calls into two parts: let $P_1 \subseteq [\ell]$ be the set of recursive calls during which the merged hyperedge $e \cup f$ survives in the hypergraph (H', w') that is input to the subsequent recursive call and $P_2 \subseteq [\ell]$ be the recursive calls during which the merged hyperedge $e \cup f$ does not survive in the hypergraph (H', w') that is input to the subsequent recursive call. We note that $[\ell] = P_1 \uplus P_2$. We bound $|P_1|$ and $|P_2|$ separately using certain carefully designed potential functions.

First, we show that $|P_1| = O(|V|)$ as follows: for $i \in [\ell]$, consider the maximal tight set family $\mathcal{T}_i = \mathcal{T}_{p_i, H_i, w_i}$ where $((H_i, w_i), p_i)$ is the input to the ith recursive call. Also, let $\mathcal{T}_{\leq 1} := \mathcal{T}_1$ and $\mathcal{T}_{\leq i} := \mathcal{T}_i \cup \{X \cap V_i : X \in \mathcal{T}_{\leq i-1}\}$ for integers i where $2 \leq i \leq \ell$ and V_i is the ground set of the input to the ith recursive call. Thus, $\mathcal{T}_{\leq i}$ is the projection of all the maximal tight sets encountered in the first i recursive calls of the algorithm onto the ground set of the inputs at the ith recursive call. We show that $\mathcal{T}_{\leq i}$ is laminar for every $i \in [\ell]$ (Lemma 6.11). However, $|\mathcal{T}_{\leq i}|$ is not necessarily non-decreasing with i since projection of a set family to a subset could result in the loss of sets from the family. Consequently, $|\mathcal{T}_{\leq i}|$ is not suitable as a potential function to measure progress. Instead, we use the potential function $\phi(i) := |\mathcal{T}_{\leq i}| + 3|\mathcal{Z}_{\leq i-1}|$, where $\mathcal{Z}_{\leq i}$ is the union of the sets $\mathcal{Z}$ computed up to the ith recursive call. We show that $\phi(i)$ is non-decreasing and strictly increases if $i \in P_1$ (Claim 6.4 in Lemma 6.12). Consequently, $|P_1| = O(|V|)$.

Secondly, we bound $|P_2|$ as follows. We use a lookahead-potential function: let $\Phi_1(i)$ be the number of recursive calls between i and ℓ during which the merged hyperedge $e \cup f$ survives in the hypergraph (H', w') that is input to the subsequent recursive call and let $\Phi(i) := |E_i| + \Phi_1(i)$, where E_i is the set of hyperedges in the hypergraph (H_i, w_i) input to the ith recursive call. We show that $\Phi(i)$ is non-increasing and strictly decreases if $i \in P_2$ (Claim 6.5 in Lemma 6.12). Hence, $|P_2| \leq \Phi(1) - \Phi(\ell) \leq \Phi(1) \leq |E_1| + |P_1| = |E| + O(|V|)$, where the last equality is because of the bound on $|P_1|$ from the previous paragraph.

Remark 1.6. Our key technical contributions are twofold. Our first key technical contribution is identifying conditions (a) and (b) under which $\mathcal{T}_{p,H,w}$ becomes a disjoint family. We ensure that conditions (a) and

(b) hold in every recursive call by performing immediate reduction and clean-up steps in the algorithm. Our second key technical contribution is identifying appropriate potential functions to measure progress of the algorithm. The disjointness of $\mathcal{T}_{p,H,w}$ was crucial for identifying the laminar structure of the family of projected maximal tight sets across recursive calls, which was subsequently helpful in bounding the number of recursive calls corresponding to P_1. Moreover, we bound the number of recursive calls corresponding to P_2 using a lookahead-potential function that relates $|P_2|$ to $|P_1|$. As discussed above, the additive $|E|$ in the recursion depth comes from the bound on $|P_2|$.

2 Preliminaries

We denote the sets of integers, non-negative integers, and positive integers by $\mathbb{Z}$, $\mathbb{Z}_{\geq 0}$, and $\mathbb{Z}_+$ respectively. For a positive integer n, we use $[n]$ to denote the set $\{1, 2, \ldots, n\}$. Let V be a finite set. We use 2^V to denote all subsets of V. Let $X, Y \subseteq V$. We will denote $X \setminus Y$ by $X - Y$ and $X \cup Y$ by $X + Y$. If Y consists of a single element y, then $X - \{y\}$ and $X + \{y\}$ are abbreviated as $X - y$ and $X + y$, respectively. Let $\mathcal{F} \subseteq 2^V$ be a family of subsets of V. The family $\mathcal{F}$ is a disjoint family if $X \cap Y = \emptyset$ for every $X, Y \in \mathcal{F}$, a chain family if either $X \subseteq Y$ or $Y \subseteq X$ for every $X, Y \in \mathcal{F}$, and a laminar family if either $X \cap Y = \emptyset$ or $X \subseteq Y$ or $Y \subseteq X$ for every $X, Y \in \mathcal{F}$. We recall that the size of a laminar family $\mathcal{F} \subseteq 2^V$ is at most $2|V|$. For a subset $U \subseteq V$, we define the projection of $\mathcal{F}$ to U as $\mathcal{F}|_U := \{X \cap U : X \in \mathcal{F}\} - \{\emptyset\}$ (we will use the convention that $\mathcal{F}|_U$ is a set and not a multi-set). The next lemma shows that the projection of a laminar family is also laminar and the size of the projection is comparable to that of the original family. We give a proof in Section B.

Lemma 2.1. Let $\mathcal{L} \subseteq 2^V$ be a laminar family and $\mathcal{Z} \subseteq V$ be a subset of elements. Let $\mathcal{L}' := \mathcal{L}|_{V-\mathcal{Z}}$. Then, the family $\mathcal{L}'$ is a laminar family and $|\mathcal{L}| \leq |\mathcal{L}'| + 3|\mathcal{Z}|$.

Let $p : 2^V \to \mathbb{R}$ be a set function over a finite ground set V. The function p is monotone if $p(X) \leq p(Y)$ for every $X \subseteq Y \subseteq V$. If $p(X) + p(Y) \leq p(X \cap Y) + p(X \cup Y)$ for every $X, Y \subseteq V$, then the function p is said to be supermodular. If $-p$ is supermodular, then the function p is said to be submodular. We recall that for a hypergraph $(H = (V, E), w : E \to \mathbb{Z}_+)$, the cut function $d_{(H,w)} : 2^V \to \mathbb{Z}_{\geq 0}$ is symmetric submodular and the coverage function $b_{(H,w)} : 2^V \to \mathbb{Z}_{\geq 0}$ is monotone submodular [27]. For a subset $Z \subseteq V$, the contracted function $p/_Z : 2^{V-Z} \to \mathbb{Z}$ is defined as $p/_Z(X) := \max\{p(X \cup R) : R \subseteq Z\}$. We note that if p is a skew-supermodular function and $Z \subseteq V$, then $p/_Z$ is skew-supermodular. Moreover, if p is a symmetric function and $Z \subseteq V$, then $p/_Z$ is symmetric.

We recall that in our algorithmic problems, we have access to the input set function p via p-`max-sc-Oracle`. We describe an additional oracle that will simplify our proofs. We will show that the oracle defined below can be implemented in strongly polynomial time using p-`max-sc-Oracle`.

Definition 2.1 (p-maximization oracle). Let $p : 2^V \to \mathbb{Z}$ be a set function. p-`max-wc-Oracle` $((G_0, c_0), S_0, T_0)$ takes as input a hypergraph $(G_0 = (V, E_0), c_0 : E_0 \to \mathbb{Z}_+)$ and disjoint sets $S_0, T_0 \subseteq V$, and returns a tuple $(Z, p(Z))$, where Z is an optimum solution to the following problem:

$$\max \left\{ p(Z) - b_{(G_0,c_0)}(Z) : S_0 \subseteq Z \subseteq V - T_0 \right\}. \qquad (p\text{-}\texttt{max-wc-Oracle})$$

The evaluation oracle for a function p can be implemented using one query to p-`max-wc-Oracle` where the input hypergraph (G_0, c_0) is the empty hypergraph and $S_0 := Z$, $T_0 := V - Z$. The next lemma shows that p-`max-wc-Oracle` can be implemented using at most $|V| + 1$ queries to p-`max-sc-Oracle` where the hypergraphs used as input to p-`max-sc-Oracle` have size of the order of the size of the hypergraph input to p-`max-wc-Oracle`. We give a proof in Section C.1.

Lemma 2.2. Let $p : 2^V \to \mathbb{Z}$ be a set function, $(G = (V, E), w : E \to \mathbb{Z}_+)$ be a hypergraph, and $S, T \subseteq V$ be disjoint sets. Then, p-`max-wc-Oracle`$((G, w), S, T)$ can be implemented to run in $O(|V|(|V|+|E|))$ time using at most $|V| + 1$ queries to p-`max-sc-Oracle`. The run-time includes the time to construct the hypergraphs used as input to the queries to p-`max-sc-Oracle`. Moreover, each query to p-`max-sc-Oracle` is on an input hypergraph (G_0, c_0) that has at most $|V|$ vertices and $|E|$ hyperedges.

For two hypergraphs $(G = (V, E_G), c_G : E_G \to \mathbb{Z}_+)$ and $(H = (V, E_H), w_H : E_H \to \mathbb{Z}_+)$ on the same vertex set V, we recall that the hypergraph $(G+H = (V, E_{G+H}), c_G + w_H)$ is the hypergraph with vertex set V and hyperedge set $E_{G+H} \coloneqq E_G \cup E_H$ with the weight of every hyperedge $e \in E_G \cap E_H$ being $c_G(e)+w_H(e)$, the weight of every hyperedge $e \in E_G \setminus E_H$ being $c_G(e)$, and the weight of every hyperedge $e \in E_H \setminus E_G$ being $w_H(e)$.

3 Local Connectivity Preserving Complete h-Splitting-Off

We prove Theorem 1.1 using Theorem 1.6 in this section. We need the following lemma showing that a certain set function $p : 2^V \to \mathbb{Z}$ is symmetric skew-supermodular and admits an efficient algorithm to implement p-`max-sc-Oracle`. This lemma has been observed in the literature before. We refer the reader to Section C.2 for its proof.

Lemma 3.1. Let $(G = (V, E), c : E \to \mathbb{Z}_+)$ be a hypergraph. Let $r : \binom{V}{2} \to \mathbb{Z}_{\geq 0}$ be a function on pairs of elements of V and $R, p_{(G,c,r)} : 2^V \to \mathbb{Z}$ be functions defined by $R(X) \coloneqq \max\{r(\{u, v\}) : u \in X, v \in V \setminus X\}$ for every $X \subseteq V$, $R(\emptyset) \coloneqq 0$, $R(V) \coloneqq 0$, and $p_{(G,c,r)}(X) \coloneqq R(X) - d_{(G,c)}(X)$ for every $X \subseteq V$.

1. The function $p_{(G,c,r)}$ is symmetric skew-supermodular.
2. Given hypergraph (G, c), function r, hypergraph $(G_0 = (V, E_0), c_0 : E_0 \to \mathbb{Z}_+)$, and disjoint sets $S_0, T_0 \subseteq V$, the oracle $p_{(G,c,r)}$-`max-sc-Oracle` $((G_0, c_0), S_0, T_0)$ can be computed in $O(|V|^3(|V| + |E_0| + |E|)(|E_0| + |E|))$ time.

We now prove the following stronger form of Theorem 1.1.

Theorem 3.1. Let $(G = (V, E), w_G : E \to \mathbb{Z}_+)$ be a hypergraph and $s \in V$. Then, there exists a hypergraph $(G^* = (V, E^*), c^*)$ such that

1. (G^*, c^*) is a local connectivity preserving complete h-splitting-off at s from (G, w_G) and
2. $|E^*| - |E| = O(|V|)$.

Moreover, given the hypergraph (G, w_G), a hypergraph (G^*, c^*) satisfying the above two properties can be obtained in $O(|V|^6(|V| + |E|)^3)$ time.

Proof. We begin by proving the existence of a local connectivity preserving complete h-splitting-off at s from $(G = (V, E), w_G : E \to \mathbb{Z}_+)$ such that the number of additional hyperedges is $O(|V|)$. We will use Theorem 1.6 with an appropriate setting of the symmetric skew-supermodular function p and the hypergraph (H, w). Let $V' \coloneqq V \setminus \{s\}$. Let the hypergraph $(H \coloneqq (V', E_H), w_H : E_H \to \mathbb{Z}_+)$ be defined by $E_H \coloneqq \{e - \{s\} : e \in \delta_G(s)\}$ with $w_H(e - \{s\}) \coloneqq w_G(e)$ for every $e \in \delta_G(s)$. In order to define the function $p : 2^{V'} \to \mathbb{Z}$, we consider another hypergraph $(J \coloneqq (V', E_J), w_J : E_J \to \mathbb{Z}_+)$ defined by $E_J \coloneqq E \setminus B_G(s)$ with $w_J(e) \coloneqq w_G(e)$ for every $e \in E \setminus B_G(s)$. We consider the function $R : 2^{V'} \to \mathbb{Z}_{\geq 0}$ defined by $R(X) \coloneqq \max\{\lambda_{(G,w_G)}(u, v) : u \in X, v \in V' \setminus X\}$ for every non-empty proper subset $X \subsetneq V'$, $R(\emptyset) \coloneqq 0$, and $R(V') \coloneqq 0$. Now, consider the function $p : 2^{V'} \to \mathbb{Z}$ defined by $p(X) \coloneqq R(X) - d_{(J,w_J)}(X)$ for every $X \subseteq V'$. The function p is symmetric skew-supermodular by Lemma 3.1. We also note that $b_{(H,w_H)}(X) \geq p(X)$ for every $X \subseteq V'$: for an arbitrary $X \subseteq V'$, we have that $b_{(H,w_H)}(X) + d_{(J,w_J)}(X) = d_{(G,w_G)}(X) \geq R(X)$ where the last inequality is by definition of the function R and hence, $b_{(H,w_H)}(X) \geq R(X) - d_{(J,w_J)}(X) = p(X)$. Hence, (H, w_H) weakly covers the symmetric skew-supermodular function p. Applying Theorem 1.6 to the hypergraph (H, w_H) and the function p, we obtain that there exists a hypergraph $(H^* = (V', E_{H^*}), w_{H^*} : E_{H^*} \to \mathbb{Z}_+)$ satisfying the following three properties: (i) $d_{(H^*,w_{H^*})} \geq p(X)$ for every $X \subseteq V'$, (ii) the hypergraph (H^*, w_{H^*}) is obtained by merging hyperedges of the hypergraph (H, w_H), and (iii) $|E_{H^*}| - |E_H| = O(|V|)$. Consider the hypergraph $(G^* = (V, E^*), c^*)$ obtained from the hypergraph $(J + H^*, w_J + w_{H^*})$ by adding the vertex s.

We have that

$$\begin{aligned}
|E^*| &= |E_J \cup E_{H^*}| && \text{(by definition of } E^*) \\
&\leq |E_J| + |E_{H^*}| \\
&= |E_J| + |E_H| + O(|V|) && \text{(by property (iii))} \\
&= |E_J \cup E_H| + O(|V|) && \text{(by definitions of } E_J \text{ and } E_H) \\
&= |E| + O(|V|), && \text{(by definitions of } E_J \text{ and } E_H)
\end{aligned}$$

and hence, $|E^*| - |E| = O(|V|)$.

We now show that (G^*, c^*) is a complete h-splitting-off at s from (G, w_G) and (G^*, c^*) preserves local connectivity. We note that property (ii) implies that (G^*, c^*) is a complete h-splitting-off at s from (G, c): this is because, hyperedges of $E_{H^*} \setminus E_H$ that are obtained by merging hyperedges of E_H are equivalent to being obtained by a sequence of h-merge almost-disjoint hyperedges operation starting from the hypergraph (G, w_G) and a final h-trim operation; moreover, hyperedges of $E_{H^*} \cap E_H$ are equivalent to being obtained by the h-trim operation from the hypergraph (G, w_G); finally, we have that $d_{(G^*,c^*)}(s) = 0$ by definition of (G^*, c^*). Next, we note that property (i) implies that (G^*, c^*) preserves local connectivity: consider an arbitrary pair of distinct vertices $u, v \in V'$. We recall that h-merge almost-disjoint hyperedges as well as h-trim operations do not increase the cut value of any subset $X \subseteq V'$ and hence, $\lambda_{(G^*,c^*)}(u, v) \leq d_{(G^*,c^*)}(X) \leq d_{(G,w_G)}(X)$ for every $X \subseteq V'$ such that $u \in X, v \in V' \setminus X$ and in particular, $\lambda_{(G^*,c^*)}(u, v) \leq \lambda_{(G,w_G)}(u, v)$. Hence, it suffices to show that $d_{(G^*,c^*)}(X) \geq \lambda_{(G,w_G)}(u, v)$ for every $X \subseteq V'$ such that $u \in X, v \in V' \setminus X$. Let $X \subseteq V'$ such that $u \in X, v \in V' \setminus X$. We have that

$$\begin{aligned}
d_{(G^*,c^*)}(X) &= d_{(J,w_J)}(X) + d_{(H^*,w_{H^*})}(X) && \text{(by definition of } (G^*, c^*)) \\
&\geq d_{(J,w_J)}(X) + p(X) && \text{(by property (i) of } (H^*, w_{H^*})) \\
&= d_{(J,w_J)}(X) + R(X) - d_{(J,w_J)}(X) && \text{(by definition of the function } p) \\
&= R(X) \\
&\geq \lambda_{(G,w_G)}(u, v). && \text{(by definition of the function } R)
\end{aligned}$$

This completes the proof that (G^*, c^*) is a complete splitting off at s from (G, w_G) and preserves local connectivity.

We now analyze the run-time of the algorithm to obtain such a hypergraph (G^*, c^*). Given $(G = (V, E), w_G : E \to \mathbb{Z}_+)$ as input, we can construct (H, w_H) and (J, w_G) as above in time $O(|V| + |E|)$. We note that both (H, w_H) and (J, w_G) have vertex set $V - \{s\}$ and at most $|E|$ hyperedges. For the function p defined as above, we consider the time to implement p-`max-sc-Oracle`: by Lemma 3.1, for a given hypergraph $(G_0 = (V - \{s\}, E_0), c_0 : E_0 \to \mathbb{Z}_+)$, sets $S_0, T_0 \subseteq V$, and a vector $y_0 \in \mathbb{R}^V$, the oracle p-`max-sc-Oracle` $((G_0, c_0), S_0, T_0, y_0)$ can be implemented to run in $O(|V|^3(|V|+|E_0|+|E|)(|E_0|+|E|))$ time. By the algorithmic conclusion of Theorem 1.6, it follows that the hypergraph (H^*, w_{H^*}) mentioned in the previous paragraph can be obtained in $O(|V|^6(|V| + |E|)^3)$ time. Finally, we can construct the hypergraph (G^*, c^*) using the hypergraph (H^*, w_{H^*}) and the hypergraph (J, w_J) in $O(|V| + |E_{H^*}| + |E|) = O(|V| + |E|)$ time. Thus, the overall run-time is $O(|V|^6(|V| + |E|)^3)$. □

4 Constructive Characterization of k-Hyperedge-Connected Hypergraphs

In this section, we prove Theorem 1.2 using Theorem 1.1. Throughout this section, we use the term hypergraph to refer to a multi-hypergraph. We define a hypergraph $H = (V, E)$ to be minimally k-hyperedge-connected if $H - e$ is not k-hyperedge-connected for every $e \in E$. We need the following observation about minimally k-hyperedge-connected hypergraphs.

Lemma 4.1. Let $k \in \mathbb{Z}_+$ be a positive integer and $G = (V, E)$ be a minimally k-hyperedge-connected hypergraph with $|V| \geq 2$. Then, there exists a vertex $u \in V$ such that $d_G(u) = k$.

Proof. Since G is minimally k-hyperedge-connected, there exists a set $X \subseteq V$ such that $d_G(X) = k$. Let $X \subseteq V$ be an inclusion-wise minimal set such that $d_G(X) = k$. We note that if $|X| = 1$, then the claim holds. Suppose that $|X| \geq 2$.

We first show that there exists a hyperedge $e \in E$ contained in the set X. By way of contradiction, suppose this is false. Let $u \in X$ be an arbitrary vertex. We note $X - \{u\} \neq \emptyset$ since $|X| \geq 2$. Furthermore, every hyperedge incident to u is not contained in the set X, i.e. $\delta_G(u) \subseteq \delta_G(X)$. Thus, we have that $k \leq d_G(u) \leq d_G(X) = k$, where the first inequality is because G is k-hyperedge-connected. Thus, all inequalities are equations, and we have that $d_G(u) = k$, contradicting minimality of the set X.

Let $e \in E$ be a hyperedge contained in the set X. Next, we show that there exists a set $Y \subseteq V$ such that $d_G(Y) = k$ and $e \in \delta_G(Y)$. By way of contradiction, suppose this is false. Then, for every $Y \subseteq V$ such that $e \in \delta_G(Y)$, we have that $d_G(Y) > k$. Thus, the hypergraph $G - e$ is k-hyperedge-connected, contradicting the minimally k-hyperedge-connected property of G.

We now complete the proof of the lemma. By minimality of the set X, we have that $Y - X \neq \emptyset$ and $(V - Y) - X \neq \emptyset$. Furthermore, since $e \subseteq X$ but $e \in \delta_G(Y)$, we have that $X - Y \neq \emptyset$ and $X \cap Y \neq \emptyset$. Thus, all four sets $Y - X, X - Y, X \cap Y, V - (X \cup Y)$ are non-empty. Hence, we have the following: $2k = d_G(X) + d_G(Y) \geq d_G(X \cap Y) + d_G(X \cup Y) \geq 2k$, where the first inequality is by submodularity of the cut function and the second inequality is because G is k-hyperedge-connected. Thus, all inequalities are equations and we have that $d_G(X \cap Y) = k$, contradicting minimality of the set X. □

We now restate and prove Theorem 1.2.

Theorem 1.2. Let $k \in \mathbb{Z}_+$ be a positive integer. A hypergraph $G = (V, E)$ is k-hyperedge-connected if and only if G can be obtained by starting from the single vertex hypergraph with no hyperedges and repeatedly applying one of the following two operations:

1. add a new hyperedge over a subset of vertices of the existing hypergraph, and
2. (k, p)-pinching hyperedges of the existing hypergraph for some positive integer $p \leq k$.

Proof. If H is a k-hyperedge-connected hypergraph, then performing both operations mentioned in the theorem gives a k-hyperedge-connected hypergraph. Consequently, the reverse direction follows by induction on $|V| + |E|$. We prove the forward direction. By way of contradiction, suppose the forward direction of the theorem is false. Let $G = (V, E_G)$ be a counter-example that minimizes $|V| + |E_G|$, i.e. G is k-hyperedge-connected but cannot be obtained by applying operations (1) and (2). We note that G is minimally k-hyperedge-connected since otherwise, there exists a hyperedge $e \in E_G$ such that $G - e$ is k-hyperedge-connected, and consequently, the hypergraph $G - e$ is also a counter-example to the forward direction of the theorem contradicting minimality of the counter-example G. Thus, by Lemma 4.1, there exists a vertex $u \in V$ such that $d_G(u) = k$. Let $H = (V - u, E_H)$ be the hypergraph obtained as a local connectivity preserving complete h-splitting-off at the vertex u from G as guaranteed to exist by Theorem 1.1. Then, the hypergraph H is k-hyperedge-connected. Moreover, $|V - u| + |E_H| < |V| + |E_G|$ because complete h-splitting-off at a vertex from a multi-hypergraph does not increase the total number of hyperedges. By minimality of the counter-example G, the hypergraph H can be obtained by starting from the single vertex hypergraph with no hyperedges and repeatedly applying operations (1) and (2). We note that G can be obtained from H by applying operation (2). This is because H was obtained by complete h-splitting-off at vertex u from G where $d_G(u) = k$ and operation (2) is the inverse of a complete h-splitting-off operation at a vertex with degree exactly k. Thus, G is not a counter-example to the forward direction of the theorem, a contradiction. □

We remark that our proof of Theorem 1.2 is constructive since it is equivalent to a proof by induction on $|V| + |E|$. Using the polynomial-time algorithm in Theorem 1.1, we get the following conclusion: given a k-hyperedge-connected hypergraph H, our proof gives a polynomial-time algorithm to construct a sequence of hypergraphs $H_0, H_1, H_2, \ldots, H_t$, where H_0 is the single vertex hypergraph with no hyperedges, $H_t = H$ and for each $i \in [t]$, the hypergraph H_i is obtained from H_{i-1} by either adding a new hyperedge over a subset of vertices in H_{i-1} or by (k, p)-pinching hyperedges in H_{i-1} for some positive integer $p \leq k$.

5 Steiner Rooted Connected Orientation of Graphs and Hypergraphs

In this section, we use Theorem 1.1 to prove Theorems 1.3 and 1.4. We recall the Max Steiner Rooted-Connected Orientation problem: the input here is a hypergraph $G = (V, E)$, a subset $T \subseteq V$ of terminals, and a root vertex $r \in T$. The goal is to find a maximum integer k and an orientation $\overrightarrow{G}$ of G such that $\overrightarrow{G}$ is Steiner rooted k-hyperarc-connected. Although Max Steiner Rooted-Connected Orientation is NP-hard, two extreme cases of the problem are polynomial-time solvable: the number of terminals $|T| = 2$ and the number of terminals $|T| = |V|$. We discuss these polynomial-time solvable cases as warm-ups towards Theorems 1.3 and 1.4. The ideas underlying these cases will be useful in proving Theorems 1.3 and 1.4. We recall that non-terminals are denoted as Steiner vertices.

The case of $|T| = 2$. Max Steiner Rooted-Connected Orientation problem where we have exactly 2 terminals is polynomial-time solvable owing to Menger's theorem in hypergraphs. A path between u and v in a hypergraph is an alternative sequence of distinct vertices and hyperedges $v_1 = u, e_1, v_2, e_2, ..., e_{\ell-1}, v_\ell = v$ such that $v_i, v_{i+1} \in e_i$ for every $i \in [\ell - 1]$. We recall Menger's theorem: for a hypergraph $G = (V, E)$ and a pair of distinct vertices $r, v \in V$, there exist k hyperedge-disjoint paths between r and v in G if and only if $\lambda_G(r, v) \geq k$. We present a proof of this fundamental graph-theoretical result using Theorem 1.1. Our proof also holds for undirected graphs. Parts of our proof will be useful in the proof of Theorems 1.3 and 1.4.

Let $G = (V, E)$ be a hypergraph, $r, v \in V$ be distinct vertices, and k be a positive integer. It is easy to see that if there exist k hyperedge-disjoint paths between r and v in G, then $\lambda_G(r, v) \geq k$. We prove the converse. Suppose that $\lambda_G(r, v) \geq k$. We would like to show that there exist k hyperedge-disjoint paths between r and v in G. Consider performing local connectivity preserving complete h-splitting-off operations at Steiner vertices in the current hypergraph and then deleting them. This will result in a 2-vertex hypergraph $H = (\{r, v\}, E_H)$ such that $\lambda_H(r, v) \geq k$. Since H contains only 2 vertices, it immediately follows that all hyperedges of H are in fact edges, and they can be oriented to obtain k hyperarc-disjoint paths from v to r. Let $\overrightarrow{H}$ be such an orientation. We now extend this orientation $\overrightarrow{H}$ to an orientation $\overrightarrow{G}$ of the original hypergraph G that still contains k hyperarc-disjoint paths from v to r. We use Lemma 5.1 below for this extension. Lemma 5.1 below proves a more general statement for arbitrary number of terminals and for a single h-splitting-off operation. We recall that a complete h-splitting-off at a vertex from a hypergraph is a sequence of h-splitting-off operations. Lemma 5.1 can be applied inductively starting from $\overrightarrow{H}$ to arrive at an orientation $\overrightarrow{G}$ of the hypergraph G such that there exist k hyperarc-disjoint paths from v to r in $\overrightarrow{G}$. Such an orientation immediately gives the k hyperedge-disjoint paths between r and v in G. This concludes our proof of Menger's theorem in hypergraphs and graphs (undirected).

Lemma 5.1. Let $G = (V, E_G)$ be a hypergraph, $T \subseteq V$ be a set of terminals, $r \in T$ be a specified root vertex, and $s \in V - T$. Let $H = (V, E_H)$ be a hypergraph obtained by applying a h-splitting-off operation at the vertex s from G. Suppose there exists an orientation $\overrightarrow{H}$ of H such that $\overrightarrow{H}$ is Steiner rooted k-hyperarc-connected. Then, there exists an orientation $\overrightarrow{G}$ of G such that $\overrightarrow{G}$ is Steiner rooted k-hyperarc-connected.

Proof. Let $\overrightarrow{H} = (V, E_H, \mathtt{head}_H : E_H \to V)$ be an orientation of H that is Steiner rooted k-hyperarc-connected. We will use the orientation $\overrightarrow{H}$ to define an orientation $\overrightarrow{G} = (V, E_G, \mathtt{head}_G : E_G \to V)$ for the hypergraph G. We recall that H is obtained by applying a h-splitting-off operation at s from G. We have two cases:

1. Suppose that H is obtained from G by a h-merge almost disjoint hyperedges operation. Let $e, f \in \delta_G(s)$ be the hyperedges with $e \cap f = \{s\}$ that are h-merged during this operation. For notational convenience, we denote $g := e \cup f$ as the hyperedge formed after the h-merge operation. Then, we define the orientation $\mathtt{head}_G : E_G \to V$ as follows: for each $e' \in E_G$,

$$\mathtt{head}_G(e') := \begin{cases} s & \text{if } e' \in \{e, f\} \text{ and } \mathtt{head}_H(g) \notin e' \\ \mathtt{head}_H(g) & \text{if } e' \in \{e, f\} \text{ and } \mathtt{head}_H(g) \in e' \\ \mathtt{head}_H(e') & \text{otherwise (i.e., } e' \in E_G - \{e, f\}). \end{cases}$$

2. Suppose that H is obtained from G by a h-trim hyperedge operation. Let $e \in \delta_G(s)$ be the hyperedge that is h-trimmed during this operation. For notational convenience, we denote $g := e - \{s\}$ as the hyperedge formed after the h-trim operation. We define an orientation $\texttt{head}_G : E_G \to V$ as follows: for each $e' \in E_G$,

$$\texttt{head}_G(e') := \begin{cases} \texttt{head}_H(g) & \text{if } e' = e \\ \texttt{head}_H(e') & \text{otherwise (i.e., } e' \in E_G - \{e\}). \end{cases}$$

We now show that the orientation $\overrightarrow{G} = (V, E_G, \texttt{head}_G)$ is Steiner rooted k-hyperarc-connected. We will prove this only for the first case above, i.e. H is obtained from G by a h-merge almost disjoint hyperedges operation, and remark that the proof for the second case is along similar lines. Let $v \in T - r$ be a terminal. Since $\overrightarrow{H}$ is a Steiner rooted k-hyperarc-connected orientation of H, there exist k hyperarc-disjoint paths $\mathcal{P} := \{P_1, \ldots, P_k\}$ from v to r in $\overrightarrow{H}$. We will use the set of paths $\mathcal{P}$ to show that there are also k hyperarc-disjoint paths from v to r in $\overrightarrow{G}$. We recall that $e, f \in \delta_G(s)$ are the hyperedges that are h-merged and $g = e \cup f$ is the hyperedge formed during the splitting-off almost disjoint hyperedges operation. First, suppose that the hyperarc $(g, \texttt{head}_H(g))$ is not present in any of the paths of $\mathcal{P}$. Then, $\mathcal{P}$ is a set of k hyperarc-disjoint paths from v to r in $\overrightarrow{G}$ and the claim holds. Next, suppose that the hyperarc $(g, \texttt{head}_H(g))$ is present in the path $P_j \in \mathcal{P}$ for some $j \in [k]$. We note that since the paths in $\mathcal{P}$ are hyperarc-disjoint, P_j is the unique path containing the hyperarc $(g, \texttt{head}_H(g))$ and the paths in $\mathcal{P} - \{P_j\}$ are hyperarc-disjoint in $\overrightarrow{G}$. We now define a new path $\tilde{P}_j$ from v to r in $\overrightarrow{G}$ that is hyperarc-disjoint from the paths in $\mathcal{P} - \{P_j\}$. Let $\ell \in \mathbb{Z}_+$ denote the length of the path P_j and let the vertices and hyperarcs along the path P_j be

$$P_j = \left(v, (e_0, u_1), u_1, (e_1, u_2), u_2, \ldots, u_{i-1}, (e_{i-1}, u_i), u_i, (g, u_{i+1}), u_{i+1}, (e_{i+1}, u_{i+2}), u_{i+1}, \ldots, u_\ell, (e_\ell, r), r\right).$$

If the vertices $u_i, u_{i+1} \in e$, then we define the path $\tilde{P}_j$ by replacing the hyperarc $(g, \texttt{head}_H(g)) = (g, u_{i+1})$ with the hyperarc $(e, \texttt{head}_G(e)) = (e, u_{i+1})$ in P_j (the case where $u_i, u_{i+1} \in f$ can be shown similarly). Suppose that $u_i \in e - f$ and $u_{i+1} \in f - e$: then, we define the path $\tilde{P}_j$ by replacing the hyperarc $(g, \texttt{head}_H(g)) = (g, u_{i+1})$ with the sub-path $(e, s), s, (f, u_{i+1})$ in P_j; we note that $\texttt{head}_G(e) = s$, s is a tail of f, and $\texttt{head}_G(f) = u_{i+1}$ (the case where $u_i \in f - e$ and $u_{i+1} \in e - f$ can be shown similarly). In both cases, the path $\tilde{P}_j$ is a v to r path in the directed hypergraph $\overrightarrow{H}$. Moreover, the k paths in $\mathcal{P} - \{P_j\} + \{\tilde{P}_j\}$ are hyperarc-disjoint. Consequently, $\overrightarrow{G}$ is a Steiner rooted k-hyperarc-connected orientation of G. □

The case of $|T| = |V|$. Max Steiner Rooted-Connected Orientation problem where all vertices are terminals is polynomial-time solvable owing to a characterization for the existence of rooted k-hyperarc-connected orientation via weak-partition-connectivity. We define weak-partition-connectivity now.

Definition 5.1. Let $k \in \mathbb{Z}_{\geq 0}$ be a non-negative integer. A hypergraph $H = (V, E)$ is k-weakly-partition-connected if for every partition $\mathcal{P}$ of V, we have that

$$\sum_{e \in E} \left(\mathcal{P}(e) - 1\right) \geq k(|\mathcal{P}| - 1),$$

where for each $e \in E$, $\mathcal{P}(e)$ denotes the number of parts of $\mathcal{P}$ that have non-empty intersection with the hyperedge e.

For a hypergraph $H = (V, E)$ and a root vertex $r \in V$, we recall that an orientation $\overrightarrow{H} = (V, E, \texttt{head} : E \to V)$ of H is rooted k-hyperarc-connected if for each $u \in V$, there exist k hyperarc-disjoint paths from the vertex u to the root vertex r. The next theorem is a consequence of Theorem 9.4.13 in [27] and shows that weak-partition-connectivity characterizes the existence of rooted hyperarc-connected orientations.

Theorem 5.1 (Theorem 9.4.13 in [27]). Let $H = (V, E)$ be an undirected hypergraph and $r \in V$ be a root vertex. There exists a rooted k-hyperarc-connected orientation of H if and only if H is k-weakly-partition-connected.

Theorem 5.1 leads to a polynomial-time algorithm to solve Max Steiner Rooted-Connected Orientation problem where all vertices are terminals.

Proof of Theorems 1.4 and 1.3. We will now use Lemma 5.1 and Theorems 5.1 and 1.1 to prove Theorems 1.3 and 1.4. We will prove Theorem 1.4 and remark after the proof about how our proof also implies Theorem 1.3. To prove Theorem 1.4, we relate connectivity and weak-partition-connectivity. We recall that a hypergraph $H = (V, E)$ is defined to be k-hyperedge-connected if $d_H(X) \geq k$ for every non-empty proper subset $X \subseteq V$. It is easy to see that if a hypergraph is k-weakly-partition-connected, then it is also k-hyperedge-connected, but the converse is not necessarily true (counterexamples exist even for graphs). We show an approximate converse: if a hypergraph is $2k$-hyperedge-connected, then it is k-weak-partition-connected.

Lemma 5.2. Let $H = (V, E)$ be an undirected hypergraph that is $2k$-hyperedge-connected for some non-negative integer k. Then, H is k-weak-partition-connected.

Proof. Let $\mathcal{P} = (P_1, P_2, \ldots, P_t)$ be a partition of V such that $t > 1$. Let $H' = (V', E')$ be the hypergraph obtained after contracting the parts of $\mathcal{P}$ and discarding the singleton hyperedges. We note that $|V'| = t$ and $|e| \geq 2$ for each $e \in E'$. The following claim gives a useful intermediate step.

Claim 5.1. $\frac{\sum_{e \in E'} |e|}{t} \leq \frac{2 \sum_{e \in E'} (|e|-1)}{t-1}$.

Proof. First, we observe that $2t|E'| \leq \sum_{e \in E'} |e|$ because $|e| \geq 2$ for each $e \in E'$. Adding $(t-1) \sum_{e \in E'} |e| - 2t|E'|$ to the LHS of the previous equation, and $(2t-1) \sum_{e \in E'} |e| - 2t|E'|$ to the RHS of the previous equation, we obtain the following inequality: $(t-1) \sum_{e \in E'} |e| \leq 2t \sum_{e \in E'} |e| - 2t|E'|$. This inequality implies the claim since $t > 1$. □

With the above claim, we have the following:

$$2k \leq \min_{i \in [t]} \{d_H(P_i)\} = \min_{u \in V'} \{d_{H'}(u)\} \leq \frac{\sum_{u \in V'} d_{H'}(u)}{t} = \frac{\sum_{e \in E'} |e|}{t} \leq \frac{2 \sum_{e \in E'} (|e| - 1)}{t - 1} = \frac{2 \sum_{e \in E'} (\mathcal{P}(e) - 1)}{|P| - 1}.$$

Here, the first inequality is because H is $2k$-hyperedge-connected and the third inequality is by Claim 5.1. Thus, H is k-weak-partition-connected. □

We now restate and prove Theorem 1.4.

Theorem 1.4 (Király and Lau [40]). Let $G = (V, E)$ be a hypergraph, $T \subseteq V$ be a subset of terminals, $r \in T$ be the root vertex, and k be a positive integer. If G is Steiner $2k$-hyperedge-connected, then it has a Steiner rooted k-hyperarc-connected orientation.

Proof. Let $u_1, \ldots, u_p$ denote an arbitrary ordering of the Steiner vertices. By repeated application of Theorem 1.1, there exists a hypergraph $H = (V, E_H)$ that can be obtained by complete h-splitting-off vertices $u_1, \ldots, u_p$ in order while preserving local connectivity. Let $H' = (T, E_H)$ be the hypergraph obtained from H by deleting the isolated vertices. We note that H is Steiner $2k$-hyperedge-connected, and consequently, the hypergraph $H' = (T, E_H)$ obtained from H by deleting the isolated Steiner vertices is $2k$-hyperedge-connected. By Lemma 5.2, the hypergraph H' is k-weak-partition-connected. By Theorem 5.1, there exists a rooted k-hyperarc-connected orientation $\overrightarrow{H'} = (T, E_H, \texttt{head}_{H'} : E_H \to T)$ of the hypergraph H'. Consequently, the orientation $\overrightarrow{H} = (V, E_H, \texttt{head}_H : E_H \to V)$, where $\texttt{head}_H(e) := \texttt{head}_{H'}(e)$ for each $e \in E_H$, is a Steiner rooted k-hyperarc-connected orientation of H. By Lemma 5.1 and induction on the number of h-splitting-off operations needed to obtain H from G, we have that G has a Steiner rooted k-hyperarc-connected orientation. □

Remark 5.1. Certain remarks regarding our proof of Theorem 1.4 are in order.

1. Our proof of Theorem 1.4 is constructive and leads to a polynomial-time algorithm to find a Steiner rooted k-hyperarc-connected orientation of a $2k$-hyperedge-connected-hypergraph.

2. Our proof of Theorem 1.4 also implies a proof of Theorem 1.3: to prove Theorem 1.3, we start from a graph $G = (V, E)$, but our local connectivity preserving complete h-splitting-off operations at Steiner vertices results in a hypergraph $H = (T, E_H)$ that is $2k$-hyperedge-connected; by Lemma 5.2, the hypergraph H is k-weak-partition-connected; now Theorem 5.1 gives a rooted k-hyperarc-connected orientation of the resulting hypergraph. Such an orientation is extended to a Steiner rooted k-arc-connected orientation of the graph $G = (V, E)$ using Lemma 5.1.

3. Our proof of Theorem 1.4 reveals the source of the 2-factor gap in the approximate min-max relation of Király and Lau for Max Steiner Rooted-Connected Orientation Problem: it arises from the 2-factor gap between connectivity and weak-partition-connectivity of hypergraphs. This insight immediately leads to a minimal example that exhibits tightness of the 2-factor in Theorem 1.3: consider the cycle graph on 3 vertices with all vertices being terminals and an arbitrary vertex chosen as the root. This graph is 2-edge-connected, 1-weak-partition-connected but not 2-weak-partition-connected, and admits a Steiner rooted 2-arc-connected orientation but does not admit a Steiner rooted 2-arc-connected orientation.

6 Weak to Strong Cover in Strongly Polynomial Time

In this section, we prove Theorem 1.6. We restate it below.

Theorem 1.6. Let $(H = (V, E), w : E \to \mathbb{Z}_+)$ be a hypergraph and $p : 2^V \to \mathbb{Z}$ be a symmetric skew-supermodular function such that $b_{(H,w)}(X) \geq p(X)$ for every $X \subseteq V$. Then, there exists a hypergraph $\left(H^* = (V, E^*), w^* : E^* \to \mathbb{Z}_+\right)$ such that

(1) $d_{(H^*,w^*)}(X) \geq p(X)$ for every $X \subseteq V$,

(2) the hypergraph (H^*, w^*) is obtained by merging hyperedges of the hypergraph (H, w), and

(3) $|E^*| - |E| = O(|V|)$.

Furthermore, given a hypergraph $(H = (V, E), w : E \to \mathbb{Z}_+)$ and access to p-`max-sc-Oracle` of a symmetric skew-supermodular function $p : 2^V \to \mathbb{Z}$ where $b_{(H,w)}(X) \geq p(X)$ for every $X \subseteq V$, there exists an algorithm that runs in $O(|V|^3(|V| + |E|)^2)$ time using $O(|V|^3(|V| + |E|))$ queries to p-`max-sc-Oracle` and returns a hypergraph $\left(H^* = (V, E^*), w^* : E^* \to \mathbb{Z}_+\right)$ satisfying the above three properties. The run-time includes the time to construct the hypergraphs used as input to the queries to p-`max-sc-Oracle`. Moreover, for each query to p-`max-sc-Oracle`, the hypergraph (G_0, c_0) used as input to the query has $O(|V|)$ vertices and $O(|V|+|E|)$ hyperedges.

Our proof of Theorem 1.6 will require the following definitions.

Definition 6.1. Let $p : 2^V \to \mathbb{Z}$ be a set function and $(H = (V, E), w)$ be a hypergraph. We will say that the hypergraph H is non-empty if $E \neq \emptyset$. A set $X \subseteq V$ is said to be (p, H, w)-tight if $b_{(H,w)}(X) = p(X)$. We let $T_{p,H,w}$ denote the family of (p, H, w)-tight sets and $\mathcal{T}_{p,H,w}$ denote the family of inclusion-wise maximal sets of $T_{p,H,w}$. For a hyperedge $e \in E$, we let $\tau_{p,H,w}(e)$ denote an arbitrary set in $T_{p,H,w}$ that contains e; if there is no such set, then we will use the convention that $\tau_{p,H,w}(e)$ is undefined. Additionally, we define two operations as follows:

1. For hyperedges $e, f \in E$ and an integer $\alpha \leq \min\{w(e), w(f)\}$, the operation $\text{Merge}((H, w), e, f, \alpha)$ returns the hypergraph obtained from (H, w) by decreasing the weight of hyperedges e and f by α and increasing the weight of the hyperedge $e \cup f$ by α (if $\alpha = w(e)$, then we discard e; if $\alpha = w(f)$, then we discard f; if $e \cup f \notin E$, then we add the hyperedge $e \cup f$ and set its weight to be α).

2. For a hyperedge $e \in E$ and an integer $\alpha \leq w(e)$, the operation $\text{Reduce}((H, w), e, \alpha)$ returns the hypergraph obtained by decreasing the weight of the hyperedge e by α (if $\alpha = w(e)$, then we discard e).

We first describe how to prove Theorem 1.6 under the assumptions that the input function p and hypergraph $\big(H = (V, E), w\big)$ satisfy the following two technical conditions:

(a) every hyperedge $e \in E$ is contained in some (p, H, w)-tight set, and

(b) $b_{(H,w)}(u) \neq 0$ for every $u \in V$, i.e. every vertex is contained in some hyperedge.

Under these assumptions, we will prove Theorem 1.6 using the algorithm that we describe in Section 6.1 (see Algorithm 1). In Section 6.2, we show certain properties of set families that will be used in the description and analysis of our algorithm. In Section 6.3, we show that our algorithm terminates, and the hypergraph returned by the algorithm satisfies properties (1) and (2) (see Lemma 6.8 and Lemma 6.10). In Section 6.4, we show that the hypergraph returned by the algorithm additionally satisfies property (3) (see Lemma 6.12). In Section 6.5, we show that our algorithm can be implemented to run in the claimed strongly polynomial time bound given oracle access to p-`max-sc-Oracle` (see Lemma 6.14). Algorithm 1, Lemma 6.8, Lemma 6.10, Lemma 6.12, and Lemma 6.14 together complete the proof of Theorem 1.6 under the assumptions that conditions (a) and (b) hold.

We now describe how to circumvent assumptions (a) and (b) on the input function p and hypergraph (H, w). Let $e \in E$ be a hyperedge not contained in any (p, H, w)-tight set. Let $\alpha_e^R := \min\{w(e), \beta^R(e)\}$, where

$$\beta^R(e) := \min\{b_{(H,w)}(X) - p(X) : e \subseteq X \subseteq V\}.$$

We construct the hypergraph $(H_0^e := (V, E_0^e := \{e\}), w_0^e : E_0 \to \{\alpha_e^R\})$. Now, consider the function $p' := p - d_{(H_0^e, w_0^e)}$ and the hypergraph $(H', w') := \text{Reduce}((H, w), e, \alpha_e^R)$; we term this operation as the reduce operation with respect to the hyperedge e. Iteratively performing this reduce operation with respect to every hyperedge $e \in E$ results in a function p_1 and a hypergraph $(H_1 = (V, E_1), w_1)$ such that $b_{(H_1,w_1)}(X) \geq p_1(X)$ for every $X \subseteq V$ and also satisfies condition (a). Furthermore, suppose that $\mathcal{Z} := \{u \in V : b_{H_1,w_1}(u) = 0\}$. Then, the function $p_2 := p_1/_{\mathcal{Z}}$ and hypergraph $(H_2 = (V - \mathcal{Z}, E_1), w_1)$ is an instance with $b_{(H_2,w_1)}(X) \geq p_2(X)$ for every $X \subseteq V$ and also satisfies conditions (a) and (b). Finally, we observe that if (H^*, w^*) is a hypergraph that strongly covers the function p_2, then the hypergraph $(H^* + \sum_{e \in E} H_0^e, w^* + w_0^e)$ strongly covers the original function p. Moreover, if (H^*, w^*) is obtained by merging hyperedges of (H_2, w_1), then the hypergraph $(H^* + \sum_{e \in E} H_0^e, w^* + w_0^e)$ is obtained by merging hyperedges of (H, w). Thus, our final algorithm is to first obtain the instance (H_2, w_1) and an oracle for p_2-`max-sc-Oracle` using p-`max-sc-Oracle` such that the instance (p_2, H_2, w_1) is such that (H_2, w_1) weakly covers p_2 and satisfies the technical conditions (a) and (b), obtain a hypergraph (H^*, w^*) by the procedure described in the previous paragraph, and finally return $(H^* + \sum_{e \in E} H_0^e, w^* + w_0^e)$.

6.1 Weak to Strong Cover Algorithm

Our algorithm is recursive and takes two inputs: (1) a hypergraph $(H = (V, E), w)$, and (2) a symmetric skew-supermodular function $p : 2^V \to \mathbb{Z}$. We note that in contrast to the statement of Theorem 1.6, the inputs (H, w) and p to the algorithm are required to satisfy the two technical conditions (a) and (b) specified in the previous section. The algorithm returns a hypergraph $\big(H^* = (V, E^*), w^*\big)$.

We now give an informal description of the algorithm (refer to Algorithm 1 for a formal description). If $E = \emptyset$, then the algorithm returns the empty hypergraph. Otherwise, the algorithm chooses two hyperedges $e, f \in E$ such that $\tau_{p,H,w}(e) \neq \tau_{p,H,w}(f)$ ($\tau_{p,H,w}(e)$ is well-defined for every hyperedge $e \in E$ owing to condition (a); in the next section, we show that $\tau_{p,H,w}(e)$ is unique for every hyperedge $e \in E$ under condition (a) and also that if (H, w) is not a strong cover of p, then there exists a pair of hyperedges $e, f \in E$ such that $\tau_{p,H,w}(e) \neq \tau_{p,H,w}(f)$). Next, the algorithm computes the value

$$\beta^M := \min\Big\{b_{(H,w)}(X) - p(X) : X \subseteq V \text{ and } e \cap X, f \cap X \neq \emptyset\Big\},$$

sets $\alpha^M := \min\{\beta^M, w(e), w(f)\}$, and defines the hypergraph $(H^M, w^M) := \text{Merge}((H, w), e, f, \alpha^M)$. Next, the algorithm computes the value

$$\beta^R := \min\Big\{b_{(H^M,w^M)}(X) - p(X) : e \cup f \subseteq X \subseteq V\Big\},$$

sets $\alpha^R \coloneqq \min\{\beta^R, w^M(e \cup f)\}$, and defines the hypergraph $(H^R, w^R) \coloneqq \text{Reduce}((H^M, w^M), e \cup f, \alpha^R)$. It computes hypergraph $(H_0 = (V, E_0 \coloneqq \{e \cup f\}), w_0 : E_0 \to \{\alpha^R\})$ and defines $p^R \coloneqq p - d_{(H_0, w_0)}$. Next, the algorithm computes the set $\mathcal{Z} \coloneqq \{u \in V : b_{(H^R, w^R)}(u) = 0\}$, and recursively calls itself with the inputs $(H' = (V' \coloneqq V - \mathcal{Z}, E' \coloneqq E^R), w' \coloneqq w^R)$ and $p' \coloneqq p^R/_{\mathcal{Z}}$ to obtain a hypergraph (H_0^*, w_0^*). Finally, the algorithm obtains the hypergraph (G, c) from (H_0^*, w_0^*) by adding the set $\mathcal{Z}$ of vertices and returns the hypergraph $(G + H_0, c + w_0)$.

Algorithm 1 Weak to Strong Cover

Algorithm $\left(H = (V, E), w, p\right)$:

if $E = \emptyset$ then Base Case:
    return (H, w).
else:
    Pick a pair of hyperedges $e, f \in E$ s.t. $\tau_{p,H,w}(e) \neq \tau_{p,H,w}(f)$
    $\alpha^M \coloneqq \min \begin{cases} \beta^M \coloneqq \min\left\{b_{(H,w)}(X) - p(X) : X \subseteq V \text{ and } e \cap X, f \cap X \neq \emptyset\right\} \\ \min\left\{w(e), w(f)\right\} \end{cases}$
    $(H^M, w^M) \coloneqq \text{Merge}\left((H, w), e, f, \alpha^M\right)$
    $\alpha^R \coloneqq \min \begin{cases} \beta^R \coloneqq \min\left\{b_{(H,w^M)}(X) - p(X) : e \cup f \subseteq X \subseteq V\right\} \\ w^M(e \cup f) \end{cases}$
    $(H^R, w^R) \coloneqq \text{Reduce}\left(\left(H^M, w^M\right), e \cup f, \alpha^R\right)$
    Construct $\left(H_0 \coloneqq \left(V, E_0 \coloneqq \{e \cup f\}\right), w_0 : E_0 \to \left\{\alpha^R\right\}\right)$
    $p^R \coloneqq p - d_{(H_0, w_0)}$
    $\mathcal{Z} \coloneqq \left\{v \in V : b_{(H^R, w^R)}(v) = 0\right\}$
    $V' \coloneqq V - \mathcal{Z}$
    $p' \coloneqq p^R/_{\mathcal{Z}}$
    $(H_0^*, w_0^*) \coloneqq \text{Algorithm}\left(H' = \left(V', E' \coloneqq E^R\right), w' \coloneqq w^R, p'\right)$
    Obtain (G, c) from (H_0^*, w_0^*) by adding vertices $\mathcal{Z}$
    return $(G + H_0, c + w_0)$

6.2 Properties of Tight Sets

In this section, we show properties of the set families $T_{p,H,w}$ and $\mathcal{T}_{p,H,w}$ which will be useful to show correctness and convergence of Algorithm 1 in subsequent sections. In particular, Lemma 6.1 below shows an important property of the family of tight sets and Corollary 6.1 shows that for every hyperedge $e \in E$, there exists a unique maximal (p, H, w)-tight set containing e under condition (a). Furthermore, Lemma 6.2 shows that the family $\mathcal{T}_{p,H,w}$ is disjoint under condition (b), and consequently, Corollary 6.2 implies that Algorithm 1 Algorithm 1 is well-defined.

Lemma 6.1. Let $p : 2^V \to \mathbb{Z}$ be a skew-supermodular function and let $\left(H = (V, E), w : E \to \mathbb{Z}_+\right)$ be a hypergraph such that $b_{(H,w)}(Z) \geq p(Z)$ for every $Z \subseteq V$. Let $X, Y \in T_{p,H,w}$ be sets such that $X \cap Y \neq \emptyset$. If there exists a hyperedge $e \in E$ such that (1) e intersects the set $X \cap Y$ and (2) e intersects at most one of the sets $X - Y$ and $Y - X$, then $X \cup Y, X \cap Y \in T_{p,H,w}$.

Proof. We note that the claim holds if $X \subseteq Y$ or $Y \subseteq X$. Suppose that $X - Y, Y - X \neq \emptyset$. First, suppose

that the function p satisfies $p(X)+p(Y) \leq p(X-Y)+p(Y-X)$. Then, the following gives us a contradiction:

$$\begin{aligned} b_{(H,w)}(X) + b_{(H,w)}(Y) &= p(X) + p(Y) \\ &\leq p(X-Y) + p(Y-X) \\ &\leq b_{(H,w)}(X-Y) + b_{(H,w)}(Y-X) \\ &< b_{(H,w)}(X) + b_{(H,w)}(Y). \end{aligned}$$

Here, the first equality is because $X, Y \in T_{p,H,w}$. The second inequality is because $b_{(H,w)}(Z) \geq p(Z)$ for every $Z \subseteq V$. The final inequality is because the hyperedge e intersects at most one of the sets $X-Y$ and $Y-X$. Thus, the function p satisfies $p(X)+p(Y) \leq p(X \cap Y) + p(X \cup Y)$. Consequently, we have that

$$\begin{aligned} b_{(H,w)}(X) + b_{(H,w)}(Y) &= p(X) + p(Y) \\ &\leq p(X \cup Y) + p(X \cap Y) \\ &\leq b_{(H,w)}(X \cup Y) + b_{(H,w)}(X \cap Y) \\ &\leq b_{(H,w)}(X) + b_{(H,w)}(Y). \end{aligned}$$

Here, the first equality is because $X, Y \in T_{p,H,w}$. The second inequality is because $b_{(H,w)}(Z) \geq p(Z)$ for every $Z \subseteq V$. The final inequality is because the coverage function $b_{(H,w)}$ is submodular. Thus, all inequalities in the above sequence are in fact equalities, and consequently we have that $X \cup Y, X \cap Y \in T_{p,H,w}$. □

Corollary 6.1. Let $p : 2^V \to \mathbb{Z}$ be a skew-supermodular function and let $\big(H = (V, E), w : E \to \mathbb{Z}_+\big)$ be a hypergraph such that every hyperedge $e \in E$ is contained in some (p, H, w)-tight set and $b_{(H,w)}(Z) \geq p(Z)$ for every $Z \subseteq V$. Then, for every hyperedge $e \in E$, there is a unique set from the family $\mathcal{T}_{p,H,w}$ containing the hyperedge e.

Proof. Let $e \in E$ be a hyperedge. We note that there exists a (p, H, w)-tight set containing the hyperedge e. Consequently, there exists a set in the family $\mathcal{T}_{p,H,w}$ containing the hyperedge e. By way of contradiction, suppose that $X, Y \in \mathcal{T}_{p,H,w}$ are distinct sets containing the hyperedge e. We note that $X-Y, Y-X \neq \emptyset$ by the maximality of the sets $X, Y \in \mathcal{T}_{p,H,w}$. Also, $X \cap Y \neq \emptyset$ since e is contained in $X \cap Y$. Moreover, e is disjoint from both $X-Y$ and $Y-X$. Hence, by Lemma 6.1, we have that $X \cup Y \in T_{p,H,w}$, contradicting maximality of the sets $X, Y \in \mathcal{T}_{p,H,w}$. □

The next lemma shows that for a skew-supermodular function p and hypergraph (H, w) satisfying conditions (a) and (b), the family $\mathcal{T}_{p,H,w}$ is disjoint. The corollary following the lemma says that if the function p is also symmetric and the hypergraph H is non-empty, then there exist two hyperedges that are each contained in distinct maximal (p, H, w)-tight sets w.r.t. p and (H, w). We note that this makes Algorithm 1 Algorithm 1 well-defined.

Lemma 6.2. Let $p : 2^V \to \mathbb{Z}$ be a skew-supermodular function and let $\big(H = (V, E), w : E \to \mathbb{Z}_+\big)$ be a hypergraph such that every hyperedge $e \in E$ is contained in some (p, H, w)-tight set, $b_{(H,w)}(u) \neq 0$ for every $u \in V$, and $b_{(H,w)}(X) \geq p(X)$ for every $X \subseteq V$. Then, the family $\mathcal{T}_{p,H,w}$ is a disjoint family.

Proof. By way of contradiction, let $X, Y \in \mathcal{T}_{p,H,w}$ be distinct sets such that $X \cap Y \neq \emptyset$. By maximality of the sets X, Y, we also have that $X-Y, Y-X \neq \emptyset$. Let $u \in X \cap Y$. Since $b_{(H,w)}(u) > 0$, there exists a hyperedge $e \in E$ such that $u \in e$. We note that if either $e \cap (X-Y) = \emptyset$ or $e \cap (Y-X) = \emptyset$, then the set $X \cup Y \in T_{p,H,w}$ by Lemma 6.1, contradicting maximality of $X, Y \in \mathcal{T}_{p,H,w}$. Thus, we have that $e \cap (X-Y), e \cap (Y-X) \neq \emptyset$. Furthermore, since the hyperedge e is contained in some (p, H, w)-tight set, we have that $\tau_{p,H,w}(e) \neq \emptyset$. Consequently, we have that $\tau_{p,H,w}(e) \cap (X-Y), \tau_{p,H,w}(e) \cap (Y-X) \neq \emptyset$. However, $e \subseteq \tau_{p,H,w}(e)$ and thus, $e \cap (X - \tau_{p,H,w}(e)) = \emptyset$. By Lemma 6.1, we have that the set $X \cup \tau_{p,H,w}(e) \in T_{p,H,w}$, contradicting maximality of the set $X \in \mathcal{T}_{p,H,w}$. □

We recall that a hypergraph is non-empty if it contains at least one hyperedge.

Corollary 6.2 ([7]). Let $p : 2^V \to \mathbb{Z}$ be a symmetric skew-supermodular function and $\big(H = (V, E), w : E \to \mathbb{Z}_+\big)$ be a non-empty hypergraph such that every hyperedge $e \in E$ is contained in some (p, H, w)-tight set, $b_{(H,w)}(u) \neq 0$ for every $u \in V$, and $b_{(H,w)}(X) \geq p(X)$ for every $X \subseteq V$. Then, there exist distinct hyperedges $e, f \in E$ such that $\tau_{p,H,w}(e) \cap \tau_{p,H,w}(f) = \emptyset$.

Proof. Let $e \in E$ be a hyperedge. We will show that that there exists a hyperedge $f \in E$ such that $f \subseteq V - \tau_{p,H,w}(e)$. The claim will then follow by Lemma 6.2 since $\tau_{p,H,w}(f) \neq \emptyset$ (because the hyperedge f is contained in some (p, H, w)-tight set) and $\tau_{p,H,w}(e), \tau_{p,H,w}(f) \in \mathcal{T}_{p,H,w}$. By way of contradiction, suppose that there is no hyperedge contained in the set $V - \tau_{p,H,w}(e)$. Then, we have that $p(V - \tau_{p,H,w}(e)) = p(\tau_{p,H,w}(e)) = b_{(H,w)}(\tau_{p,H,w}(e)) > b_{(H,w)}(V - \tau_{p,H,w}(e))$, a contradiction. Here, the first equality is because the function p is symmetric and the final inequality is because every hyperedge that intersects the set $V - \tau_{p,H,w}(e)$ also intersects the set $\tau_{p,H,w}(e)$ by our contradiction assumption. □

6.3 Termination and Partial Correctness of the Algorithm

In this section, we show that every recursive call of Algorithm 1 is well-defined. Furthermore, we show that Algorithm 1 terminates and that the hypergraph returned by the algorithm satisfies properties (1) and (2) of Theorem 1.6. The main results of this section are Lemma 6.8 and Lemma 6.10. We prove certain additional lemmas in this section which will be useful in subsequent sections.

Suppose that the input to Algorithm 1 is a non-empty hypergraph $(H = (V, E), w)$ and a symmetric skew-supermodular function $p : 2^V \to \mathbb{Z}$ such that every hyperedge $e \in E$ is contained in some (p, H, w)-tight set, $b_{(H,w)}(u) \neq 0$ for every $u \in V$, and $b_{(H,w)}(X) \geq p(X)$ for every $X \subseteq V$. Then, by Corollary 6.2, there exist hyperedges $e, f \in E$ such that $\tau_{p,H,w}(e) \neq \tau_{p,H,w}(f)$. In particular, Algorithm 1 Algorithm 1 is well-defined. The next two lemmas establish several useful properties of the intermediate quantities used in the algorithm.

Lemma 6.3. Suppose that the input to Algorithm 1 is a non-empty hypergraph $(H = (V, E), w)$ and a symmetric skew-supermodular function $p : 2^V \to \mathbb{Z}$ such that every hyperedge $\tilde{e} \in E$ is contained in some (p, H, w)-tight set, $b_{(H,w)}(u) \neq 0$ for every $u \in V$, and $b_{(H,w)}(X) \geq p(X)$ for every $X \subseteq V$. Let $e, f, \alpha^M, (H^M, w^M), \alpha^R, (H^R, w^R), (H_0, w_0), p^R$ and $(H' = (V', E'), w')$ be as defined by Algorithm 1. Then, we have the following:

(1) $\alpha^M \in \mathbb{Z}_+$,

(2) $E^M = E - \{e\} \cdot \mathbb{1}_{\alpha^M = w(e)} - \{f\} \cdot \mathbb{1}_{\alpha^M = w(f)} + \{e \cup f\}$ with the weights of the hyperedges in E^M being

$$w^M(\tilde{e}) = \begin{cases} w(\tilde{e}) & \text{if } \tilde{e} \in E - \{e, f, e \cup f\}, \\ w(e \cup f) + \alpha^M & \text{if } \tilde{e} = e \cup f \text{ and } e \cup f \in E, \\ \alpha^M & \text{if } \tilde{e} = e \cup f \text{ and } e \cup f \notin E, \\ w(e) - \alpha^M & \text{if } \tilde{e} = e \text{ and } \alpha^M < w(e), \\ w(f) - \alpha^M & \text{if } \tilde{e} = f \text{ and } \alpha^M < w(f). \end{cases}$$

(3) $b_{(H^M, w^M)}(X) \geq p(X)$ for every $X \subseteq V$,

(4) $\alpha^R \in \mathbb{Z}_{\geq 0}$,

(5) $E' = E^R = E^M - \{e \cup f\} \cdot \mathbb{1}_{\alpha^R = w^M(e \cup f)}$ with the weights of the hyperedges in $E' = E^R$ being

$$w'(\tilde{e}) = w^R(\tilde{e}) = \begin{cases} w^M(\tilde{e}) & \text{if } \tilde{e} \in E^M - \{e \cup f\}, \\ w^M(e \cup f) - \alpha^R & \text{if } \tilde{e} = e \cup f \text{ and } \alpha^R < w^M(e \cup f), \text{ and} \end{cases}$$

(6) $b_{(H^R, w^R)}(X) \geq p^R(X)$ for every $X \subseteq V$.

Proof. We prove each claim separately below.

1. We recall that the weight function $w : E \to \mathbb{Z}_+$ is a positive integer-valued function, and the function p is an integer-valued function. Thus, it suffices to show that $\beta^M > 0$. We note that $\beta^M \geq 0$ since $b_{(H,w)}(X) \geq p(X)$ for every $X \subseteq V$. By way of contradiction, suppose that $\beta^M = 0$. Let $X \subseteq V$ witness β^M, i.e. $e \cap X, f \cap X \neq \emptyset$ and $b_{(H,w)}(X) - p(X) = 0$. The latter property implies that $X \in T_{p,H,w}$ and thus, there exists $Y \in \mathcal{T}_{p,H,w}$ such that $X \subseteq Y$. We recall that by the algorithm's choice of hyperedges $e, f \in E$, we have that $\tau_{p,H,w}(e) \neq \tau_{p,H,w}(f)$. Since, $\tau_{p,H,w}(e), \tau_{p,H,w}(f) \in \mathcal{T}_{p,H,w}$, we have that $\tau_{p,H,w}(e) \cap \tau_{p,H,w}(f) = \emptyset$ by Lemma 6.2. In particular, we have that $\tau_{p,H,w}(e) \neq Y$ since $f \subseteq V - \tau_{p,H,w}(e)$ but $f \cap Y \neq \emptyset$. Thus, $Y \cap \tau_{p,H,w}(e) = \emptyset$ by Lemma 6.2, contradicting $\emptyset \neq e \cap X \subseteq e \cap Y \subseteq \tau_{p,H,w}(e) \cap Y$.

2. Follows from the definition of Merge operation (see Definition 6.1).

3. Let $X \subseteq V$. If at most one of the sets $X \cap e$ or $X \cap f$ is non-empty, then by part (2) of the current lemma (shown above) we have that $b_{(H^M,w^M)}(X) = b_{(H,w)}(X) \geq p(X)$. Suppose that $e \cap X, f \cap X \neq \emptyset$. Then, we have that $\alpha^M \leq \beta^M \leq b_{(H,w)}(X) - p(X)$. Thus, $b_{(H^M,w^M)}(X) = b_{(H,w)}(X) - \alpha^M \geq b_{(H,w)}(X) - \beta^M \geq p(X)$, where the equality is by part (2) of the current lemma (shown above).

4. We note that $w^M(e \cup f) = \alpha^M \in \mathbb{Z}_+$ by parts (1) and (2) of the current lemma (shown above). Furthermore, $b_{H^M,w^M}(X) \geq p(X)$ for every $X \subseteq V$ by part (3) of the current lemma (shown above). Thus, $\beta^R \in \mathbb{Z}_{\geq 0}$ since p is an integer-valued function.

5. Follows from the definition of Reduce operation (see Definition 6.1).

6. Let $X \subseteq V$. First, suppose that the hyperedge $e \cup f$ is not contained in the set X. Then, we have that $b_{(H^R,w^R)}(X) = b_{(H^M,w^M)}(X) - d_{(H_0,w_0)}(X) \geq p(X) - d_{(H_0,w_0)}(X) = p^R(X)$. Here, the first equality is by part (5) of the current lemma and the inequality is by part (3) of the current lemma (proofs of both parts shown above). Next, suppose that $e \cup f \subseteq X$. Then, we have that $b_{(H^R,w^R)}(X) = b_{(H^M,w^M)}(X) - \alpha^R \geq b_{H^M,w^M}(X) - \beta^R \geq b_{(H^M,w^M)}(X) - (b_{(H^M,w^M)}(X) - p(X)) = p(X) = p^R(X)$, where the first equality is by part (5) of the current lemma (shown above) and because $e \cup f \subseteq X$, and the final equality is because $e \cup f \subseteq X$.

□

With the previous properties established, we now show that the function p' and hypergraph (H', w') constructed by Algorithm 1 as input to the subsequent recursive call of the algorithm satisfy the two hypothesis conditions of Theorem 1.6 – the function p' is symmetric skew-supermodular, and the hypergraph (H', w') weakly covers the function p'. This is a necessary step in order to give an inductive proof of Theorem 1.6.

Lemma 6.4. Suppose that the input to Algorithm 1 is a non-empty hypergraph $(H = (V, E), w)$ and a symmetric skew-supermodular function $p : 2^V \to \mathbb{Z}$ such that every hyperedge $\tilde{e} \in E$ is contained in some (p, H, w)-tight set, $b_{(H,w)}(u) \neq 0$ for every $u \in V$, and $b_{(H,w)}(X) \geq p(X)$ for every $X \subseteq V$. Let p' and $(H' = (V', E'), w')$ be as defined by Algorithm 1. Then, we have the following:

1. the function p' is symmetric skew-supermodular, and

2. $b_{(H',w')}(X) \geq p'(X)$ for every $X \subseteq V'$.

Proof. We prove each claim separately below. Let $(H_0, w_0), p^R, \mathcal{Z}$, and (H^R, w^R) be as defined by Algorithm 1.

1. We recall that $p^R = p - d_{(H_0,w_0)}$. We first observe that the function p^R is symmetric skew-supermodular because the function p is symmetric skew-supermodular and the cut function $d_{(H_0,w_0)}$ is symmetric submodular. Next, we observe that the function $p' = p^R/_{\mathcal{Z}}$. The claim then follows because the contraction operation preserves skew-supermodularity and symmetry.

2. Let $X \subseteq V'$. Let $R \subseteq \mathcal{Z}$ be such that $p'(X) = p^R/_{\mathcal{Z}}(X) = p^R(X \cup R)$. Then, we have the following: $b_{(H',w')}(X) = b_{(H^R,w^R)}(X \cup R) \geq p^R(X \cup R) = p'(X)$. Here, the first equality is because $w' = w^R$ and $b_{(H^R,w^R)}(u) = 0$ for every $u \in R$ by the definition of $\mathcal{Z}$. The inequality is by Lemma 6.3(6).

□

In order to show that the function p' and (H', w') constructed by Algorithm 1 satisfy conditions (a) and (b), we examine how the families of (p, H, w)-tight sets change during a recursive call of the algorithm. The next lemma shows that (p, H, w)-tight sets persist through the merge and reduce steps of a recursive call. Furthermore, the lemma also shows that the projection of a (p, H, w)-tight set onto the ground set of the subsequent recursive call is (p', H', w')-tight.

Lemma 6.5. Suppose that the input to Algorithm 1 is a non-empty hypergraph $(H = (V, E), w)$ and a symmetric skew-supermodular function $p : 2^V \to \mathbb{Z}$ such that every hyperedge $\tilde{e} \in E$ is contained in some (p, H, w)-tight set, $b_{(H,w)}(u) \neq 0$ for every $u \in V$, and $b_{(H,w)}(X) \geq p(X)$ for every $X \subseteq V$. Let $(H^M, w^M), (H^R, w^R), p'$ and $(H' = (V', E'), w')$ be as defined by Algorithm 1. Then, we have the following:

1. $T_{p,H,w} \subseteq T_{p,H^M,w^M} \subseteq T_{p^R,H^R,w^R}$, and
2. $T_{p^R,H^R,w^R}|_{V'} \subseteq T_{p',H',w'}$.

Proof. We prove the claims separately below. Let $e, f, \alpha^M, \mathcal{Z}$ be as defined by Algorithm 1.

1. We will first show that $T_{p,H,w} \subseteq T_{p,H^M,w^M}$. Let $X \in T_{p,H,w}$. Suppose $e \cap X \neq \emptyset$ and $f \cap X \neq \emptyset$, then $b_{(H^M,w^M)}(X) = b_{(H,w)}(X) - \alpha^M = p(X) - \alpha^M < p(X)$, which is a contradiction. Here, the first equality is because $e \cap X \neq \emptyset$ and $f \cap X \neq \emptyset$ and the final inequality is by Lemma 6.3(1). So, at most one of $e \cap X$ and $f \cap X$ is non-empty and consequently, $b_{(H,w)}(X) = b_{(H^M,w^M)}(X)$. Thus, we have that $p(X) = b_{(H,w)}(X) = b_{(H^M,w^M)}(X)$, i.e. $X \in T_{p,H^M,w^M}$.

   We will now show the second inclusion in the claim, i.e. $T_{p,H^M,w^M} \subseteq T_{p^R,H^R,w^R}$. Let $X \in T_{p,H^M,w^M}$. First, suppose that $\alpha^R = 0$. Then, we have that $b_{(H^R,w^R)}(X) = b_{(H^M,w^M)}(X) = p(X) = p^R(X)$, where the first and final equalities are because $\alpha^R = 0$. Thus, $X \in T_{p^R,H^R,w^R}$. Next, suppose that $\alpha^R > 0$. We note that if $(e \cup f) \cap X = \emptyset$, then $b_{(H^R,w^R)}(X) = b_{(H^M,w^M)}(X) = p(X) = p^R(X)$ and so $X \in T_{p^R,H^R,w^R}$. Consider the case where $(e \cup f) \cap X \neq \emptyset$. Here, we may assume that $e \cup f \in \delta(X)$ since otherwise, we obtain the following contradiction $b_{(H^R,w^R)}(X) = b_{(H^M,w^M)}(X) - \alpha^R = p(X) - \alpha^R < p(X) = p^R(X)$, where the final equality is because $d_{(H_0,w_0)}(X) = 0$ since $e \cup f \notin \delta_H(X)$. Consequently, we have that $b_{(H^R,w^R)}(X) = b_{(H^M,w^M)}(X) - \alpha^R = p(X) - \alpha^R = p^R(X)$, and the claim holds.

2. Let $X \in T_{p^R,H^R,w^R}$. Let $\mathcal{Z}$ be as defined by Algorithm 1. Then, we have the following:

$$p'(X - \mathcal{Z}) = p^R/_{\mathcal{Z}}(X - \mathcal{Z}) \geq p^R(X) = b_{(H^R,w^R)}(X) = b_{(H^R,w^R)}(X - \mathcal{Z}) = b_{(H',w')}(X - \mathcal{Z}),$$

   where the penultimate equality is because $\mathcal{Z} = \{u \in V : b_{(H^R,w^R)}(u) = 0\}$. However, by Lemma 6.4, we have that $b_{H',w'}(X - \mathcal{Z}) \geq p'(X - \mathcal{Z})$. Thus, the inequality in the above sequence is an equation and we have that $X - \mathcal{Z} \in T_{p',H',w'}$.

□

We now show another useful property of Algorithm 1 which we will need while reasoning about the convergence of Algorithm 1 in the next section.

Lemma 6.6. Suppose that the input to Algorithm 1 is a non-empty hypergraph $(H = (V, E), w)$ and a symmetric skew-supermodular function $p : 2^V \to \mathbb{Z}$ such that every hyperedge $\tilde{e} \in E$ is contained in some (p, H, w)-tight set, $b_{(H,w)}(u) \neq 0$ for every $u \in V$, and $b_{(H,w)}(X) \geq p(X)$ for every $X \subseteq V$. Let α^M, α^R and β^M be as defined by Algorithm 1. Then, $\alpha^R = 0$ if and only if $\alpha^M = \beta^M$.

Proof. We prove the claim by showing the implication in both directions. Let $e, f, (H^M, w^M), \beta^R$ be as defined by Algorithm 1. First, suppose that $\alpha^M = \beta^M$. Let $X \subseteq V$ be such that $\beta^M = b_{(H,w)}(X) - p(X)$ and $e \cap X, f \cap X \neq \emptyset$ (such a set exists by definition of β^M). Then, we have that $b_{(H^M,w^M)}(X) = b_{(H,w)}(X) - \alpha^M = p(X)$, where the first equality is by Lemma 6.3(2). Thus, $X \in T_{p,H^M,w^M}$. Let $Y \in \mathcal{T}_{p,H^M,w^M}$ such that $X \subseteq Y$. Claim 6.1 below says that the set Y contains the hyperedge $e \cup f$. With this claim, we complete the proof of the forward direction as follows: By Claim 6.1, we have that $\alpha^R \leq \beta^R \leq b_{H^M,w^M}(Y) - p(Y) = 0$, where the final equality is because $Y \in T_{p,H^M,w^M}$. Consequently, $\alpha^R = 0$ because $\alpha^R \geq 0$ by Lemma 6.3(4).

Claim 6.1. $e \cup f \subseteq Y$.

Proof. By way of contradiction, say $(e \cup f) - Y \neq \emptyset$. Thus, $e - Y \neq \emptyset$ or $f - Y \neq \emptyset$ (or both). We consider the case where $e - Y \neq \emptyset$ and remark that the proof of the case where $f - Y \neq \emptyset$ is along the same lines. We recall that the hyperedge e is contained in some (p, H, w)-tight set, and thus the set $\tau_{p,H,w}(e)$ is well-defined and non-empty. Furthermore, by Lemma 6.5(1), we have that the set $\tau_{p,H,w}(e) \in T_{p,H^M,w^M}$. Consequently, there exists a set $Z \in \mathcal{T}_{p,H^M,w^M}$ such that $e \subseteq \tau_{p,H,w}(e) \subseteq Z$. Since $e - Y \neq \emptyset$ but $e \subseteq Z$, we have that $Y \neq Z$. We recall that $e \cap X \neq \emptyset$ by our choice of the set X, and hence, $e \cap Y \neq \emptyset$. Furthermore, by Lemma 6.3(3), we have that $b_{(H^M,w^M)}(A) \geq p(A)$ for every $A \subseteq V$. Thus, by Lemma 6.1 applied to the function p, hypergraph (H^M, w^M), sets Y and Z, and the hyperedge e, we have that the set $Y \cup Z \in \mathcal{T}_{p,H^M,w^M}$, a contradiction to the maximality of the sets $Y, Z \in \mathcal{T}_{p,H^M,w^M}$. □

Next, we prove the reverse direction of the claim. Suppose that $\alpha^R = 0$. We note that $w^M(e \cup f) \geq \alpha^M > 0$ by Lemma 6.3(2) and 6.3(1). Thus, we have that $\alpha^R < w^M(e \cup f)$ and so $\alpha^R = \beta^R$. It follows that $\beta^R = 0$. Consequently, there exists a set $X \subseteq V$ such that $e \cup f \subseteq X$ and $b_{(H^M,w^M)}(X) = p(X)$ (by definition of β^R). Then, we have the following:

$$p(X) = b_{(H^M,w^M)}(X) = b_{(H,w)}(X) - \alpha^M \geq b_{(H,w)}(X) - \beta^M \geq b_{(H,w)}(X) - (b_{(H,w)}(X) - p(X)) = p(X).$$

Thus, all inequalities are equations, and we have that $\alpha^M = \beta^M$. Here, the second equality is by Lemma 6.3(2) and $e \cup f \subseteq X$. The second inequality is also because $e \cup f \subseteq X$. □

We now show that if the inputs to Algorithm 1 satisfy conditions (a) and (b), then the inputs to every subsequent recursive call of Algorithm 1 also satisfy the two conditions.

Lemma 6.7. Suppose that the input to Algorithm 1 is a non-empty hypergraph $(H = (V, E), w)$ and a symmetric skew-supermodular function $p : 2^V \to \mathbb{Z}$ such that every hyperedge $\tilde{e} \in E$ is contained in some (p, H, w)-tight set, $b_{(H,w)}(u) \neq 0$ for every $u \in V$, and $b_{(H,w)}(X) \geq p(X)$ for every $X \subseteq V$. Let $(H' = (V', E'), w')$ and p' be as defined by Algorithm 1. Then, we have the following:

1. every hyperedge $\tilde{e} \in E'$ is contained in some (p', H', w')-tight set, and
2. $b_{H',w'}(u) > 0$ for every vertex $u \in V'$.

Proof. We prove the claims separately below. Let $e, f, \mathcal{Z}, \alpha^R, \beta^R, p', (H^M, w^M), (H^R, w^R)$ be as defined by Algorithm 1.

1. We consider two cases. First, suppose that $\tilde{e} \in E \cap E'$. Since the hyperedge $\tilde{e}$ is contained in some (p, H, w)-tight set, we have that $\tau_{p,H,w}(\tilde{e})$ is well-defined and non-empty. Furthermore, by Lemma 6.5(2), we have that $\tau_{p,H,w}(\tilde{e})|_{\mathcal{Z}} \in T_{p',H',w'}$. Consequently, $\tilde{e} \subseteq \tau_{p,H,w}(\tilde{e})|_{\mathcal{Z}} \in T_{p',H',w'}$. Next, suppose that $\tilde{e} \in E' - E$. In particular, this implies that $E' - E \neq \emptyset$. We recall that $e, f \in E$ are the hyperedges chosen by Algorithm 1 Algorithm 1. By Lemma 6.3(5) and 6.3(2), we have that $\tilde{e} = e \cup f$ and $\alpha^R = \beta^R < w^M(e \cup f)$. Consequently, $\mathcal{Z} = \emptyset$ and $V' = V$. Let $Z \subseteq V$ be a set such that $\alpha^R = b_{(H^M,w^M)}(Z) - p(Z)$ and $e \cup f \subseteq Z$ (such a set exists by definition of β^R). Then, we have the following:
$$b_{H',w'}(Z) = b_{(H^R,w^R)}(Z) = b_{(H^M,w^M)}(Z) - \alpha^R = p(Z) = p^R(Z) = p'(Z),$$

and consequently the set $Z \in T_{p',H',w'}$. Here, the first equality is by definitions of the weight function w^R and the set $\mathcal{Z}$. The second and the fourth equalities are because $e \cup f \subseteq Z$. The third equality is because the set Z is a witness set for $\alpha^R = \beta^R$. The fifth equality is because $\mathcal{Z} = \emptyset$.

2. We recall that $V' = V - \mathcal{Z} = V - \{u \in V : b_{(H^R,w^R)}(u) = 0\}$. The claim then follows because $w' = w^R$.

□

We now show the first main result of the section which says that Algorithm 1 terminates, and the hypergraph returned by the algorithm indeed strongly covers the input symmetric skew-supermodular function, i.e. satisfies property (1) of Theorem 1.6.

Lemma 6.8. Suppose that the input to Algorithm 1 is a hypergraph $(H = (V, E), w)$ and a symmetric skew-supermodular function $p : 2^V \to \mathbb{Z}$ such that every hyperedge $\tilde{e} \in E$ is contained in some (p, H, w)-tight set, $b_{(H,w)}(u) \neq 0$ for every $u \in V$, and $b_{(H,w)}(X) \geq p(X)$ for every $X \subseteq V$. Then, Algorithm 1 terminates in finite number of recursive calls. Furthermore, let (H^*, w^*) be the hypergraph returned by the algorithm. Then, $d_{(H^*,w^*)}(X) \geq p(X)$ for every $X \subseteq V$.

Proof. By Lemmas 6.4 and 6.7, the input to every recursive call (except the last one) of the execution satisfies the hypothesis of Corollary 6.2, and consequently, during every recursive call of the execution (except the last one), there exist two hyperedges such that Algorithm 1 Algorithm 1 is well-defined. We first show that algorithm terminates in a finite number of recursive calls. Consider the potential function $\phi(H = (V, E), w) \coloneqq \sum_{e \in E} w(e)$. We note that $\phi(H, w) \in \mathbb{Z}_{\geq 0}$ since the weight function $w : E \to \mathbb{Z}_+$ is a positive integral function. Furthermore, if $\phi(H, w) = 0$, then we have that $E = \emptyset$ and the algorithm terminates. We recall that p' and (H', w') denote the inputs to the subsequent recursive call of Algorithm 1. By Lemmas 6.4 and 6.7, we have that p' is a symmetric skew-supermodular function and $(H' = (V', E'), w')$ is a non-empty hypergraph such that every hyperedge $\tilde{e} \in E'$ is contained in some (p', H', w')-tight set, $b_{(H',w')}(u) \neq 0$ for every $u \in V'$, and $b_{(H',w')}(X) \geq p'(X)$ for every $X \subseteq V'$. Then, by Lemma 6.3(1), 6.3(2) and 6.3(5), we have that $\phi(H, w) > \phi(H', w')$, i.e. the value of the potential function strictly decreases between subsequent recursive calls. Consequently, the algorithm terminates in a finite number of recursive calls.

We now show that the hypergraph (H^*, w^*) returned by Algorithm 1 strongly covers the function p, i.e. $d_{(H^*,w^*)}(X) \geq p(X)$ for every $X \subseteq V$. We show this by induction on $w(E)$. For the base case, consider $w(E) = 0$. Since the weight function w is positive, we have that $E = \emptyset$. Let $X \subseteq V$. We have that $d_{(H,w)}(X) = 0 = b_{(H,w)}(X) \geq p(X)$ and so the claim holds. For the inductive case, consider $w(E) > 0$. Let $X \subseteq V$ be an arbitrary subset. Let $(H_0^* = (V', E_0^*), w_0^*)$ be the hypergraph returned by the subsequent recursive call on input hypergraph $(H' = (V', E'), w')$ and function p'. We recall that $w'(E') < w(E)$ and so by the inductive hypothesis, we have that $d_{(H_0^*,w_0^*)}(X) \geq p'(X)$ for every $X \subseteq V'$. We also recall that the hypergraph $(H^*, w^*) = (G + H_0, c + w_0)$, where the hypergraph (G, c) is obtained from (H_0^*, w_0^*) by adding the vertices $\mathcal{Z}$. Then, we have the following:

$$\begin{aligned}
d_{(H^*,w^*)}(X) &= d_{G,c}(X) + d_{(H_0,w_0)}(X) \\
&= d_{H_0^*,w_0^*}(X) + \alpha^R \mathbb{1}_{e \cup f \in \delta(X)} \\
&\geq p'(X) + \alpha^R \mathbb{1}_{e \cup f \in \delta(X)} && \text{(by inductive hypothesis)} \\
&= \left(p - \alpha^R \cdot \mathbb{1}_{e \cup f \in \delta(X)}\right)/_{\mathcal{Z}}(X) + \alpha^R \mathbb{1}_{e \cup f \in \delta(X)} \\
&\geq \left(p - \alpha^R \cdot \mathbb{1}_{e \cup f \in \delta(X)}\right)(X) + \alpha^R \mathbb{1}_{e \cup f \in \delta(X)} \\
&= p(X).
\end{aligned}$$

□

We now show that the hypergraph returned by Algorithm 1 is obtained by merging hyperedges of the input hypergraph (see Definition 1.5 for the definition of merging hyperedges). This will imply that the

hypergraph returned by the algorithm satisfies property (2) of Theorem 1.6. To show this property, we will need the following lemma about the choice of hyperedges e and f being merged by the algorithm.

Lemma 6.9. Suppose that the input Algorithm 1 call is a non-empty hypergraph $(H = (V, E), w)$ and a symmetric skew-supermodular function $p : 2^V \to \mathbb{Z}$ such that every hyperedge $\tilde{e} \in E$ is contained in some (p, H, w)-tight set, $b_{(H,w)}(u) \neq 0$ for every $u \in V$, and $b_{(H,w)}(X) \geq p(X)$ for every $X \subseteq V$. Let e, f be as defined by Algorithm 1. Then,

1. $e \cap f = \emptyset$, and
2. $e \cup f \notin E$.

Proof. We prove the claims separately below.

1. By the choice of hyperedges e and f, we have that $\tau_{p,H,w}(e) \neq \tau_{p,H,w}(f)$. We observe that by maximality of the sets $\tau_{p,H,w}(e)$ and $\tau_{p,H,w}(f)$, we have that $\tau_{p,H,w}(e), \tau_{p,H,w}(f) \in \mathcal{T}_{p,H,w}$. By Lemma 6.2, we have that $\tau_{p,H,w}(e) \cap \tau_{p,H,w}(f) = \emptyset$. Consequently, $e \cap f = \emptyset$.
2. By way of contradiction, suppose that $e \cup f \in E$. Thus, $\tau_{p,H,w}(e \cup f) \neq \emptyset$. Since $e, f \subseteq \tau_{p,H,w}(e \cup f)$, we have that $\tau_{p,H,w}(e) = \tau_{p,H,w}(f) = \tau_{p,H,w}(e \cup f)$ by the maximality of the three sets, contradicting the choice of hyperedges e, f made by the algorithm.

□

We now conclude the section by showing that the hypergraph returned by Algorithm 1 indeed satisfies property (2) of Theorem 1.6.

Lemma 6.10. Suppose that the input to Algorithm 1 is a hypergraph $(H = (V, E), w)$ and a symmetric skew-supermodular function $p : 2^V \to \mathbb{Z}$ such that every hyperedge $\tilde{e} \in E$ is contained in some (p, H, w)-tight set, $b_{(H,w)}(u) \neq 0$ for every $u \in V$, and $b_{(H,w)}(X) \geq p(X)$ for every $X \subseteq V$. Then, the hypergraph $(H^* = (V, E^*), w^*)$ returned by the algorithm is obtained by merging hyperedges of the hypergraph (H, w).

Proof. We show the claim by induction on $w(E)$. For the base case, consider $w(E) = 0$. Since the weight function w is positive, we have that $E = \emptyset$ and hence (H, w) is the empty hypergraph. In this case, the hypergraph (H^*, w^*) returned by the algorithm is also the empty hypergraph and so the claim holds.

For the inductive case, suppose that $w(E) > 0$. Let $(H^M, w^M), \alpha^M$, $(H' = (V', E'), w'), p', (H^R, w^R)$, $(H_0 = (V, E_0), w_0), \mathcal{Z}, (H_0^*, w_0^*), (G, c)$ be as defined by Algorithm 1. By Lemmas 6.4 and 6.7, the inputs (H', w') and p' satisfy the hypothesis of Lemma 6.8. Thus, the execution of the algorithm from this recursive call terminates in a finite number of recursive calls and returns the hypergraph $(H_0^* = (V', E_0^*), w_0^*)$. Then, by Claims 6.2 and 6.3 below, we have that the hypergraph (H^*, w^*) is obtained by merging hyperedges of (H, w). □

Claim 6.2. (H^*, w^*) is obtained by merging hyperedges of (H^M, w^M).

Proof. We first show that the hypergraph (G, c) is obtained by merging hyperedges of (H^R, w^R). By Lemma 6.3(1), (2), and (5), we have that $w'(E') < w(E)$. Thus, by the inductive hypothesis, we have that the hypergraph (H_0^*, w_0^*) is obtained by merging hyperedges of (H', w'). Consequently, (G, c) is formed by merging hyperedges of (H^R, w^R) because of the following two observations: (H', w') is obtained from (H^R, w^R) by removing the vertices $\mathcal{Z}$ and (G, c) is obtained by adding vertices $\mathcal{Z}$ to (H_0^*, w_0^*).

Next, we recall that $(H^*, w^*) = (G + H_0, c + w_0)$. Thus, by the conclusion of the previous paragraph, we have that (H^*, w^*) is obtained by merging hyperedges of $(H^R + H_0, w^R + w_0)$. The claim then follows because $(H^R + H_0, w^R + w_0) = (H^M, w^M)$ by Lemma 6.3(5). □

Claim 6.3. (H^M, w^M) is obtained by merging hyperedges of (H, w).

Proof. We recall that $(H^M, w^M) = \text{Merge}((H, w), e, f, \alpha^M)$ (see Algorithm 1 Algorithm 1). Thus, by definition of the Merge operation, the hypergraph (H^M, w^M) is obtained by decreasing the weight of hyperedges e and f by α^M and increasing the weight of the hyperedge $e \cup f$ by α^M. Consequently, the multi-hypergraph $H^M_{w^M}$ can be obtained from the multi-hypergraph H_w by repeatedly merging hyperedges e and f exactly α^M times. We note that the hyperedges e and f are disjoint by Lemma 6.9(1). Thus, the hypergraph (H^M, w^M) is obtained by merging hyperedges of (H, w) by definition. □

6.4 Number of Recursive Calls and Hypergraph Support Size

We first set up some notation that will be used in the remainder of this section. By Lemma 6.8, the number of recursive calls made by Algorithm 1 is finite. We will use ℓ to denote the depth of recursion of the algorithm. We will refer to the recursive call at depth i as recursive call i or the ith recursive call. Let $i \in [\ell]$ be a recursive call. We let the hypergraph $(H_i = (V_i, E_i), w_i : E_i \to \mathbb{Z}_+)$ and the function $p_i : 2^{V_i} \to \mathbb{Z}$ denote the inputs to the ith recursive call. Furthermore, we let $e_i, f_i, \beta_i^M, \alpha_i^M, (H_i^M, w_i^M), \beta_i^R, \alpha_i^R, (H_i^R, w_i^R), p_i^R, (H_0^i, w_0^i), E_0^i, \mathcal{Z}_i, V_i', p_i'$ denote the quantities $e, f, \beta^M, \alpha^M, (H^M, w^M), \beta^R, \alpha^R, (H^R, w^R), p^R, (H_0, w_0), E_0, \mathcal{Z}, V', p'$ as defined by Algorithm 1 during the ith recursive call.

We now define certain set families of that will be crucial in our analysis. We let T_i, T_i^M, T_i^R denote the families $T_{p_i,H_i,w_i}, T_{p_i,H_i^M,w_i^M}$ and $T_{p_i^R,H_i^R,w_i^R}$ respectively, and $\mathcal{T}_i, \mathcal{T}_i^M, \mathcal{T}_i^R$ denote the families $\mathcal{T}_{p_i,H_i,w_i}, \mathcal{T}_{p_i,H_i^M,w_i^M}$ and $\mathcal{T}_{p_i^R,H_i^R,w_i^R}$ respectively. We define the family of cumulative projected maximal (p_i, H_i, w_i)-tight sets as:

$$\mathcal{T}_{\leq i} := \begin{cases} \mathcal{T}_1 & \text{if } i = 1, \\ \mathcal{T}_{\leq i-1}|_{V_i} \cup \mathcal{T}_i & \text{if } i \geq 2. \end{cases}$$

The next lemma states two important properties of these set families.

Lemma 6.11. Suppose that the input to Algorithm 1 is a non-empty hypergraph $(H = (V, E), w)$ and a symmetric skew-supermodular function $p : 2^V \to \mathbb{Z}$ such that every hyperedge $\tilde{e} \in E$ is contained in some (p, H, w)-tight set, $b_{(H,w)}(u) \neq 0$ for every $u \in V$, and $b_{(H,w)}(X) \geq p(X)$ for every $X \subseteq V$. Furthermore, suppose that the execution of Algorithm 1 on the input instance terminates using $\ell \in \mathbb{Z}_+$ recursive calls. Then, for every $i \in [\ell - 1]$, we have the following:

1. $\mathcal{T}_{\leq i}|_{V_{i+1}} \subseteq T_{i+1}$, and
2. the family $\mathcal{T}_{\leq i}$ is laminar.

Proof. Let $i \in [\ell - 1]$ be a recursive call. We recall that $p_i : 2^{V_i} \to \mathbb{Z}$ and (H_i, w_i) are inputs to the ith recursive call, where $p_1 := p$ and $(H_1, w_1) := (H, w)$. By Lemmas 6.4 and 6.7 and induction on i, we have that p_i is a symmetric skew-supermodular function and $(H_i = (V_i, E_i), w_i)$ is a non-empty hypergraph such that every hyperedge $\tilde{e} \in E_i$ is contained in some (p_i, H_i, w_i)-tight set, $b_{(H_i,w_i)}(u) \neq 0$ for every $u \in V_i$, and $b_{(H_i,w_i)}(X) \geq p_i(X)$ for every $X \subseteq V_i$. We now prove both claims separately below.

1. By way of contradiction, let $i \in [\ell - 1]$ be the least index such that $\mathcal{T}_{\leq i}|_{V_{i+1}} - T_{i+1} \neq \emptyset$. We note that $i \geq 2$ since $\mathcal{T}_{\leq 1} = \mathcal{T}_1 \subseteq T_1$ and $T_1|_{V_2} \subseteq T_2$ by Lemma 6.5(1) and 6.5(2), and hence, $\mathcal{T}_{\leq 1}|_{V_2} \subseteq T_2$. Thus, by our choice of index i, we have that $\mathcal{T}_{\leq i-1}|_{V_i} \subseteq T_i$, and so it follows that $\mathcal{T}_{\leq i} = \mathcal{T}_{\leq i-1}|_{V_i} \cup \mathcal{T}_i \subseteq T_i$. Moreover, $T_i|_{V_{i+1}} \subseteq T_{i+1}$ by Lemma 6.5(1) and 6.5(2). Consequently we have that $\mathcal{T}_{\leq i}|_{V_{i+1}} \subseteq T_i|_{V_{i+1}} \subseteq T_{i+1}$, a contradiction to the choice of the index i.

2. By way of contradiction, let $i \in [\ell - 1]$ be the least index such that the family $\mathcal{T}_{\leq i}$ is not laminar. We note that $i \geq 2$ since the family $\mathcal{T}_{\leq 1} = \mathcal{T}_1$ is disjoint by Lemma 6.2. Thus, we have that $\mathcal{T}_{\leq i} = \mathcal{T}_{\leq i-1}|_{V_i} \cup \mathcal{T}_i$. We note that the family $\mathcal{T}_i$ is also disjoint by Lemma 6.2. Furthermore, the family $\mathcal{T}_{\leq i-1}$ (and consequently, the family $\mathcal{T}_{\leq i-1}|_{V_i}$) is laminar by our choice of $i \in [\ell - 1]$. Since the family $\mathcal{T}_{\leq i}$ is not laminar, there exist distinct sets $X \in \mathcal{T}_{\leq i-1}|_{V_i}$ and $Y \in \mathcal{T}_i$ such that $X - Y, Y - X, X \cap Y \neq \emptyset$. By part (1) of the current lemma (shown above), we have that $X \in T_i$. Consequently, there exists $Z \in \mathcal{T}_i$ such that $X \subseteq Z$. We note that $Y \neq Z$ because $X - Z = \emptyset$ and $X - Y \neq \emptyset$. Thus, by Lemma 6.2 we have that $Z \cap Y = \emptyset$. Therefore, $X \cap Y = \emptyset$, contradicting our choice of sets X and Y.

□

Lemma 6.12. Suppose that the input to Algorithm 1 is a non-empty hypergraph $(H = (V, E), w)$ and a symmetric skew-supermodular function $p : 2^V \to \mathbb{Z}$ such that every hyperedge $\tilde{e} \in E$ is contained in some (p, H, w)-tight set, $b_{(H,w)}(u) \neq 0$ for every $u \in V$, and $b_{(H,w)}(X) \geq p(X)$ for every $X \subseteq V$. Then, we have the following:

1. the recursion depth $\ell \in \mathbb{Z}_+$ of the algorithm is at most $|E| + 10|V| - 1$, and
2. $|E^*| \leq |E| + 10|V| - 2$, where $(H^* = (V, E^*), w^*)$ denotes the hypergraph returned by the algorithm.

Proof. Let $i \in [\ell - 1]$ be a recursive call. We recall that $(H_i = (V_i, E_i), w_i : E_i \to \mathbb{Z}_+)$ and $p_i : 2^{V_i} \to \mathbb{Z}$ denote the inputs to the ith recursive call, and $(H_i^* = (V, E_i^*), w_i^*)$ denotes the hypergraph returned by the ith recursive call. Furthermore, $e_i, f_i, \beta_i^M, \alpha_i^M, (H_i^M, w_i^M), \beta_i^R, \alpha_i^R, (H_i^R, w_i^R), p_i^R, (H_0^i, w_0^i), E_0^i, \mathcal{Z}_i$ denote the quantities as defined by Algorithm 1 during the ith recursive call. We prove the claims separately below.

1. We have the following:

$$\ell \leq \left|\left\{i \in [\ell - 1] : \alpha_i^R = \beta_i^R\right\}\right| + \left|\left\{i \in [\ell - 1] : \alpha_i^R = w_i^M(e_i \cup f_i)\right\}\right| + 1 \leq |E| + 10|V| - 1,$$

   where the final inequality is by Claim 6.4 and Claim 6.5 below.

2. We have the following:

$$|E_1^*| \leq |E_2^*| + |E_0^2| = |E_2^*| + \mathbb{1}_{\alpha_1^R > 0} = |E_\ell^*| + \sum_{i \in [\ell-1]} \mathbb{1}_{\alpha_i^R > 0} \leq \ell - 1 \leq |E_1| + 10|V_1| - 2.$$

   Here, the second inequality is because $E_\ell^* = \emptyset$ by definition of the Algorithm 1 and the final inequality is by part (1) of the claim (shown above).

□

Claim 6.4. $\left|\left\{i \in [\ell - 1] : \alpha_i^R = \beta_i^R\right\}\right| \leq 5|V| - 1.$

Proof. We define a potential function $\phi : [\ell - 1] \to \mathbb{Z}_{\geq 0}$ as follows: for every $i \in [\ell]$,

$$\phi(i) \coloneqq \left|\mathcal{T}_{\leq i}\right| + 3|\mathcal{Z}_{\leq i-1}|,$$

where we define the set $\mathcal{Z}_{\leq 0} \coloneqq \emptyset$. We first show that the function ϕ is monotone non-decreasing. Let $i \in [\ell-1]$ be a recursive call of Algorithm 1. If $\mathcal{Z}_i = \emptyset$, then $|\mathcal{T}_{\leq i}| \leq |\mathcal{T}_{\leq i+1}|$ since $\mathcal{T}_{\leq i+1} = \mathcal{T}_{\leq i}|_{V_{i+1}} \cup \mathcal{T}_{i+1} = \mathcal{T}_{\leq i} \cup \mathcal{T}_{i+1}$, where the second equality is because $V_{i+1} = V_i - \mathcal{Z}_i = V_i$. Then, we have that $\phi(i + 1) \geq \phi(i)$ since $\mathcal{Z}_{\leq i-1} \subseteq \mathcal{Z}_{\leq i}$ by definition. Thus, we suppose that $\mathcal{Z}_i \neq \emptyset$. We have the following:

$$\begin{aligned}
\phi(i+1) - \phi(i) &= |\mathcal{T}_{\leq i+1}| + 3|\mathcal{Z}_{\leq i}| - |\mathcal{T}_{\leq i}| - 3|\mathcal{Z}_{\leq i-1}| \\
&= \left|\mathcal{T}_{\leq i}|_{V_{i+1}} \cup \mathcal{T}_{i+1}\right| - |\mathcal{T}_{\leq i}| + 3|\mathcal{Z}_i| \\
&\geq \left|\mathcal{T}_{\leq i}|_{V_{i+1}}\right| - |\mathcal{T}_{\leq i}| + 3|\mathcal{Z}_i| \\
&\geq 0.
\end{aligned}$$

Here, the final inequality is because of the following: we recall that the family $\mathcal{T}_{\leq i}$ is laminar by Lemma 6.11. Furthermore, the family $\mathcal{T}_{\leq i}|_{V_{i+1}}$ is the projection of the laminar family $\mathcal{T}_{\leq i}$ to the ground set $V_{i+1} = V_i - \mathcal{Z}_i$. The inequality then follows by Lemma 2.1 – here, we observe that Lemma 2.1 says that this inequality is strict because $\mathcal{Z}_i \neq \emptyset$. We will refer back to this observation in the next part of the proof.

Consider a recursive call $i \in \left\{i \in [\ell - 1] : \alpha_i^R = \beta_i^R\right\}$. We now show that $\phi(i + 1) > \phi(i)$, i.e. the potential value strictly increases. We suppose that $\mathcal{Z}_i = \emptyset$ as otherwise $\phi(i + 1) > \phi(i)$ by the observation

in the final line of the previous paragraph. Thus, $p_{i+1} = p_i^R$ and $(H', w') = (H^R, w^R)$. We note that the hypergraph (H_i, w_i) is non-empty since $i < \ell$. We consider a set $X \subseteq V_i$ such that $e_i \cup f_i \subseteq X \subseteq V_i$ and $b_i^M(X) - p_i(X) = \beta_i^R$ (such a set exists by the definition of β_i^R). Then, we have the following: $b_{i+1}(X) = b_i^R(X) = b_i^M(X) - \alpha_i^R = p_i(X) = p_i^R(X) = p_{i+1}(X)$, where the first equality is because $(H', w') = (H^R, w^R)$, the second equality is because $\mathcal{Z}_i = \emptyset$, the third equality is because $\alpha_i = \beta_i$ by our choice of $i \in [\ell - 1]$, and the final equality is because $p_{i+1} = p_i^R$. Thus, $X \in T_{i+1}$. Consequently, there exists a set $Y \in \mathcal{T}_{i+1}$ such that $e_i \cup f_i \subseteq X \subseteq Y$. We note that $Y \notin T_i$ due to the following: suppose (for sake of contradiction) that $Y \in T_i$, then $b_i(Y) = p_i(Y)$ and $b_i^M(Y) = b_i(Y) - \alpha_i^M = p_i(Y) - \alpha_i^M < p_i(Y)$, a contradiction to Lemma 6.3(3) – the first equality is by definition of b_i^M and $e_i \cup f_i \subseteq Y$ and the inequality is because $\alpha_i^M \geq 1$ by Lemma 6.3(1). Thus, $Y \in \mathcal{T}_{i+1} - T_i \subseteq \mathcal{T}_{i+1} - \mathcal{T}_{\leq i}$ by Lemma 6.11(1). Consequently, we have that $\mathcal{T}_{\leq i} \subsetneq \mathcal{T}_{\leq i} \cup \mathcal{T}_{i+1} = \mathcal{T}_{\leq i+1}$.

We now show the claim. By the previous two properties of our potential function ϕ, we have the following:

$$\left|\left\{i \in [\ell - 1] : \alpha_i^R = \beta_i^R\right\}\right| \leq \phi(\ell - 1) - \phi(1) \leq |\mathcal{T}_{\leq \ell - 1}| + 3|\mathcal{Z}_{\leq \ell - 1}| \leq 5|V| - 1,$$

where the final inequality is because the family $\mathcal{T}_{\leq \ell - 1}$ is laminar by Lemma 6.11(2). □

Claim 6.5. $\left|\left\{i \in [\ell - 1] : \alpha_i^R = w_i^M(e_i \cup f_i)\right\}\right| \leq |E| + 5|V| - 1.$

Proof. We define a potential function $\phi : [\ell - 1] \to \mathbb{Z}_{\geq 0}$ as follows: for every $i \in [\ell]$,

$$\phi(i) \coloneqq -|E_i| - \left|\left\{j \in [i, \ell - 1] : \alpha_j^R < w_j^M(e_j \cup f_j)\right\}\right|.$$

First, consider a recursive call $i \in [\ell - 1]$ such that $\alpha_i^R = \beta_i^R < w_i^M(e_i \cup f_i)$. We will show that the potential function ϕ does not decrease. By Lemma 6.3(2) and 6.3(5), we have that

$$\begin{aligned} E_{i+1} &= E_i - \{e_i\} \cdot \mathbb{1}_{\alpha_i^M = w_i(e_i)} - \{f_i\} \cdot \mathbb{1}_{\alpha_i^M = w_i(f_i)} + \{e_i \cup f_i\} \cdot \mathbb{1}_{\alpha_i^R = \beta_i^R < w_i^M(e_i \cup f_i)} \\ &= E_i - \{e_i\} \cdot \mathbb{1}_{\alpha_i^M = w_i(e_i)} - \{f_i\} \cdot \mathbb{1}_{\alpha_i^M = w_i(f_i)} + \{e_i \cup f_i\}. \end{aligned}$$

Consequently, we have that $|E_{i+1}| \leq |E_i| + 1$. Furthermore, by the choice of recursive call i, we have that $\left|\{j \in [i + 1, \ell - 1] : \alpha_j^R < w_j^M(e_j \cup f_j)\}\right| = \left|\{j \in [i, \ell - 1] : \alpha_j^R < w_j^M(e_j \cup f_j)\}\right| - 1$. Thus, $\phi(i + 1) \geq \phi(i)$.

Next, consider a recursive call $i \in [\ell - 1]$ such that $\alpha_i^R = w_i^M(e_i \cup f_i)$. We will show that the potential function ϕ strictly increases. We note that the hypergraph (H_i, w_i) is non-empty since $i < \ell$. By Lemma 6.3(1), we have that $\alpha_i^R = w_i^M(e_i \cup f_i) = \alpha_i^M \geq 1$. Consequently, by Lemma 6.6, we have that $\alpha_i^M < \beta_i^M$ and hence, $\alpha_i^M \in \{w_i(e_i), w_i(f_i)\}$ by definition. Suppose that $\alpha_i^M = w_i(e_i)$. Then, we have that $w_i^R(e_i) = w_i^M(e_i) = w_i(e_i) - \alpha_i^M = 0$. Thus, we have that $e_i \notin E_{i+1}$. Then, by our choice of i and by Lemma 6.3(2) and 6.3(5), we have that $E_{i+1} \subsetneq E_i$. We note that this strict inclusion can also be obtained in the case where $\alpha_i^M = w_i(f_i)$ using the same argument. Furthermore, by the choice of recursive call i, we have that $\left|\{j \in [i + 1, \ell - 1] : \alpha_j^R < w_j^M(e_j \cup f_j)\}\right| = \left|\{j \in [i, \ell - 1] : \alpha_j^R < w_j^M(e_j \cup f_j)\}\right|$. Thus, $\phi(i + 1) > \phi(i)$.

The previous two properties of our potential function give us the following:

$$\left|\left\{i \in [\ell - 1] : \alpha_i^R = w_i^M(e_i \cup f_i)\right\}\right| \leq \phi(\ell - 1) - \phi(1) \leq |E_1| + \left|\left\{j \in [\ell - 1] : \alpha_j^R < w_j^M(e_j \cup f_j)\right\}\right| \leq |E| + 5|V| - 1,$$

where the final inequality is by Claim 6.4 – we note that if $\alpha_j^R < w_j^M(e_j \cup f_j)$, then $\alpha_j^R = \beta_j^R$ by definition. □

6.5 Run-time of the Algorithm

In this section, we show that Algorithm 1 can be implemented to run in strongly polynomial time. Lemma 6.14 is the main lemma of this section. In order to show that each recursive call of Algorithm 1 can be implemented in polynomial time, we will require the ability to compute the family $\mathcal{T}_{p,H,w}$. In the following lemma, we show that this family can indeed be computed in polynomial time. We defer the proof of the lemma to Section C.3.

Lemma 6.13. Let $p : 2^V \to \mathbb{Z}$ be a skew-supermodular function and let $\big(H = (V, E), w : E \to \mathbb{Z}_+\big)$ be a hypergraph such that every hyperedge $e \in E$ is contained in some (p, H, w)-tight set, $b_{(H,w)}(u) \neq 0$ for every $u \in V$, and $b_{(H,w)}(X) \geq p(X)$ for every $X \subseteq V$. Then, the family $\mathcal{T}_{p,H,w}$ can be computed in $O(|V|^2(|V| + |E|))$ time using $O(|V|^2)$ queries to p-`max-wc-Oracle`. The run-time includes the time to construct the hypergraphs used as input to the queries to p-`max-wc-Oracle`. Moreover, for each query to p-`max-wc-Oracle`, the hypergraph (G_0, c_0) used as input to the query has at most $|V|$ vertices and at most $|E| + 1$ hyperedges.

We now bound the run-time of Algorithm 1.

Lemma 6.14. Suppose that the input to Algorithm 1 is a non-empty hypergraph $(H = (V, E), w)$ and a symmetric skew-supermodular function $p : 2^V \to \mathbb{Z}$ such that every hyperedge $\tilde{e} \in E$ is contained in some (p, H, w)-tight set, $b_{(H,w)}(u) \neq 0$ for every $u \in V$, and $b_{(H,w)}(X) \geq p(X)$ for every $X \subseteq V$. Then, Algorithm 1 can be implemented to run in $O(|V|^3(|V| + |E|)^2)$ time using $O(|V|^3(|V| + |E|))$ queries to p-`max-sc-Oracle`. The run-time includes the time to construct the hypergraphs used as input to the queries to p-`max-sc-Oracle`. Moreover, for each query to p-`max-sc-Oracle`, the hypergraph (G_0, c_0) used as input to the query has $O(|V|)$ vertices and $O(|V| + |E|)$ hyperedges.

Proof. Let ℓ denote the recursion depth of Algorithm 1. By Lemma 6.12, we have that $\ell = O(|E| + |V|)$. Let $i \in [\ell]$ be a recursive call. We recall that $p_i : 2^{V_i} \to \mathbb{Z}$ and (H_i, w_i) are inputs to the ith recursive call, where $p_1 := p$ and $(H_1, w_1) := (H, w)$. By Lemmas 6.4 and 6.7 and induction on i, we have that p_i is a symmetric skew-supermodular function and $(H_i = (V_i, E_i), w_i)$ is a (non-empty if $i \leq \ell - 1$) hypergraph such that every hyperedge $\tilde{e} \in E_i$ is contained in some (p_i, H_i, w_i)-tight set, $b_{(H_i,w_i)}(u) \neq 0$ for every $u \in V_i$, and $b_{(H_i,w_i)}(X) \geq p_i(X)$ for every $X \subseteq V_i$. Let $e_i, f_i, \beta_i^M, \alpha_i^M, (H_i^M, w_i^M), \beta_i^R, \alpha_i^R, (H_i^R, w_i^R)$, p_i^R, (H_0^i, w_0^i), E_0^i, $\mathcal{Z}_i$ denote the quantities as defined by Algorithm 1 during the ith recursive call. We recall that $V_i \subseteq V$. By Lemma 6.3(2), (5) and induction on i, we have that $|E_i| \leq |E| + i \leq |E| + \ell = O(|E| + |V|)$, where the final equality is by Lemma 6.12(1). Furthermore, the hypergraph returned in Step 14 contains at most $|V|$ vertices and at most $|E_i| + O(|V_i|) = O(|V| + |E|)$ hyperedges by Lemma 6.12(2) and induction on i. The following claim shows that all steps except for Steps 10, 13, and 14 of Algorithm 1 in the ith recursive call can be implemented to run in $O(|V_i|^2(|V_i| + |E_i|))$ time using $O(|V_i|^2)$ queries p_i-`max-wc-Oracle`. This includes the run-time to construct the hypergraphs used in the queries to p_i-`max-wc-Oracle`. We will address the run-time for Steps 10, 13, and 14 later.

Claim 6.6. For every $i \in [\ell]$, all steps except for Steps 10, 13, and 14 of Algorithm 1 in the ith recursive call can be implemented to run in $O(|V_i|^2(|V_i| + |E_i|))$ time using $O(|V_i|^2)$ queries to p_i-`max-wc-Oracle`. The run-time includes the time to construct the hypergraphs used as input to the queries to p_i-`max-wc-Oracle`. Moreover, for each query to p_i-`max-wc-Oracle`, the hypergraph (G_0, c_0) used as input to the query has $|V_i|$ vertices and at most $|E_i| + 1$ hyperedges.

Proof. We first show how to compute hyperedges e_i and f_i (Step 4). We recall that by Corollary 6.2, there exist two hyperedges $e_i, f_i \in E_i$ such that $\tau_{p_i,H_i,w_i}(e_i) \neq \tau_{p_i,H_i,w_i}(f_i)$ and consequently Step 4 is well defined. By Lemma 6.2, the family $\mathcal{T}_{p_i,H_i,w_i}$ is a disjoint family. Furthermore, by Lemma 6.13, we can compute the family $\mathcal{T}_{p_i,H_i,w_i}$ in $O(|V_i|^2(|V_i| + |E_i|))$ time using $O(|V_i|^2)$ queries to p_i-`max-wc-Oracle`. Moreover, for each query to p_i-`max-wc-Oracle`, the hypergraph (G_0, c_0) used as input to the query has at most $|V_i|$ vertices and at most $|E_i| + 1$ hyperedges. We note that $\tau_{p_i,H_i,w_i}(\tilde{e}) \in \mathcal{T}_{p_i,H_i,w_i}$ for every $\tilde{e} \in E$. Consequently, we can find the required hyperedges for Step 4 by finding a pair of distinct sets $X, Y \in \mathcal{T}_{p_i,H_i,w_i}$ that each contains a hyperedge, and define these hyperedges to be e_i and f_i respectively. Thus, Step 4 can be implemented to run in $O(|V_i|^2(|V_i| + |E_i|))$ time using $O(|V_i|^2)$ queries to p_i-`max-wc-Oracle`. Moreover, for each query to p_i-`max-wc-Oracle`, the hypergraph (G_0, c_0) used as input to the query has at most $|V_i|$ vertices and at most $|E_i| + 1$ hyperedges.

Next, we show how to compute the values α_i^M (Step 5) and α_i^R (Step 7). We first show how to compute

the value β_i^M. We have the following:

$$\begin{aligned}\beta_i^M &= \min\left\{b_{(H_i,w_i)}(X) - p_i(X) : X \subseteq V_i \text{ and } e_i \cap X, f_i \cap X \neq \emptyset\right\} \\ &= \min_{u\in e_i, v\in f_i}\left\{\min\left\{b_{(H_i,w_i)}(X) - p_i(X) : \{u,v\} \subseteq X \subseteq V_i\right\}\right\}.\end{aligned}$$

For $u \in e_i$ and $v \in f_i$, let $\beta_i^{uv} := \min\{b_{(H_i,w_i)}(X) - p_i(X) : \{u,v\} \subseteq X \subseteq V_i\}$. Thus, $\beta_i^M = \min_{u\in e_i, v\in f_i} \beta_i^{uv}$. Then, β_i^M (and consequently the value α_i^M) can be computed in $O(|V_i|^2(|V_i|+|E_i|))$ time using $|V_i|^2$ queries to p_i-`max-wc-Oracle`, where each query is on a input hypergraph (G_0, c_0) that has at most $|V_i|$ vertices and at most $|E_i|+1$ hyperedges. This is because for every $u \in e_i$ and $v \in f_i$ (there are $O(|V_i|^2)$ choices for u and v), the value β_i^{uv} can be computed using a single oracle call to p_i-`max-wc-Oracle` on a hypergraph (G_0, c_0) that has at most $|V_i|$ vertices and at most $|E_i|+1$ hyperedges. Next, the value β_i^R, (and consequently the value α_i^R) can be computed in $O(|V_i|+|E_i|)$ time using a single query to p_i-`max-wc-Oracle` on the input hypergraph $(G_0 = H_i^M, c_0 = w_i^M)$ that has $|V_i|$ vertices and at most $|E_i|+1$ hyperedges. Here, the hypergraph (H_i^M, w_i^M) has $|V_i|$ vertices and $|E_i|+1$ hyperedges because of Lemma 6.3(2). Next, we show that the hypergraphs (H_i^M, w_i^M) (Step 6) and (H_i^R, w_i^R) (Step 8) can be computed in $O(|V_i|+|E_i|)$ time. We recall that the Merge operation decreases the weights of hyperedges e_i and f_i by α_i^M, increases the weight of the hyperedge $e_i \cup f_i$ by α_i^M and discards the resulting zero-weighted edges (if any). This can be implemented to run in $O(|E_i|)$ time. Furthermore, the Reduce operation decreases the weight of the hyperedge $e_i \cup f_i$ by α_i^R, and so can also be implemented to run in $O(|E_i^R|) = O(|E_i|)$ time by Lemma 6.3(2) and (5). Finally, the computations of the hypergraph (H_0^i, w_0^i) (Step 9), the set $\mathcal{Z}_i$ (Algorithm 1) and the set V_i' (Algorithm 1) can each be done in $O(|V_i|)$ time. Furthermore, the hypergraph returned in Step 14 contains at most $|V|$ vertices and at most $|E_i| + O(|V_i|)$ hyperedges. Consequently, Steps 15 and 16 can be implemented to run in $O(|V_i| + |E_i|)$ time. □

Claim 6.6 implies that each query to p_i-`max-wc-Oracle` takes as input a hypergraph with at most $|V_i|$ vertices and $O(|E_i|)$ hyperedges. Next, we focus on the time to implement a query to p_i-`max-wc-Oracle` on such hypergraphs.

Claim 6.7. For every $i \in [\ell]$ and for every given hypergraph $(G = (V_i, E_G), c : E_G \to \mathbb{Z}_+)$ such that $|E_G| = O(|E_i|)$ AND disjoint subsets $S, T \subseteq V_i$, p_i-`max-wc-Oracle`$((G, c), S, T, y)$ can be computed in $O(|V|(|V| + |E|))$ time using at most $|V_i|+1$ queries to p-`max-sc-Oracle`. The run-time includes the time to construct the hypergraphs used as input to the queries to p-`max-sc-Oracle`. Moreover, for each query to p-`max-sc-Oracle`, the hypergraph (G_0, c_0) used as input to the query has $O(|V|)$ vertices and $O(|V| + |E|)$ hyperedges.

Proof. Let the input to p_i-`max-wc-Oracle` be a hypergraph $(G = (V_i, E_G), c : E_G \to \mathbb{Z}_+)$ such that $|E_G| = O(|E_i|)$ and disjoint subsets $S, T \subseteq V_i$. By Lemma 2.2, we can implement p_i-`max-wc-Oracle`$((G, c), S, T, y_0)$ in $O(|V_i|(|V_i|+|E_i|))$ time using at most $|V_i|+1$ queries to p_i-`max-sc-Oracle`, where each query to p_i-`max-sc-Oracle` is on a input hypergraph with $|V_i| + 1$ vertices and $O(|E_i|)$ hyperedges. Next, we show that the query p_i-`max-sc-Oracle`$((G, c), S, T)$ where $(G = (V_i, E_G), c : E_G \to \mathbb{Z}_+)$ is a hypergraph with $|E_G| = O(|E_i|)$ and $S, T \subseteq V_i$ are disjoint subsets, can be implemented in $O(|V| + |E|)$ time using one query to p-`max-sc-Oracle` on an input hypergraph (G_0, c_0) that has $O(|V|)$ vertices and $O(|V| + |E|)$ hyperedges.

We first recall some notation and define some quantities for convenience. For $i \in [\ell]$, we recall that (H_0^i, w_0^i) is the hypergraph (H_0, w_0), p_i^R, p_i' are the functions p^R, p' respectively, and $\mathcal{Z}_i$ is the set $\mathcal{Z}$ defined in the ith recursive call of the algorithm. We note that $p_i' = p_{i+1}$ for every $i \in [\ell - 1]$. Furthermore, the hypergraph (H_0^i, w_0^i) contains exactly one hyperedge for every $i \in [\ell - 1]$. For $i \in [\ell - 1]$, let (H_1^i, w_1^i) be the hypergraph obtained from (H_0^i, w_0^i) by adding the vertices $\cup_{j=1}^{i-1} \mathcal{Z}_j$.

Let $i \in [\ell]$. Let the input to p_i-`max-sc-Oracle` be a hypergraph $(G = (V_i, E_G), c : E_G \to \mathbb{Z}_+)$ such that $|E_G| = O(|E_i|)$ and disjoint subsets $S_0, T_0 \subseteq V_i$. We note that $p_i = p_{i-1}'$ and hence, it suffices to answer p_{i-1}'-`max-sc-Oracle`$((G, c), S_0, T_0)$. Construct the hypergraph (G_1, c_1) to be the hypergraph obtained from (G, c) by adding vertices $\cup_{j=1}^{i-1} \mathcal{Z}_j$. Construct the hypergraph $(\tilde{H}, \tilde{w}) = (\sum_{j=1}^{i-1} H_1^j, \sum_{j=1}^{i} w_1^j)$. The hypergraph $(\tilde{H}, \tilde{w})$ contains at most $i - 1 \leq \ell = O(|E| + |V|)$ hyperedges, and can be constructed in

time $O(|V|+|E|)$. Next, construct the hypergraph $(G,c) = (G_1 + \tilde{H}, c_1 + \tilde{w})$. The hypergraph (G,c) contains $O(|E|+|V|)$ hyperedges and can be constructed in time $O(|V|+|E|)$. Obtain $(Z, p(Z))$ by querying p-`max-sc-Oracle`$((G,c), S_0, T_0)$ and return $(Z - \cup_{j=1}^{i-1} \mathcal{Z}_j, p(Z))$. □

By Claim 6.6, each query to p_i-`max-wc-Oracle` takes as input a hypergraph with $|V_i| \leq |V|$ vertices and $O(|E_i|) = O(|E|+|V|)$ hyperedges. By Claim 6.7, a query to p_i-`max-wc-Oracle` on a input hypergraph with at most $|V_i|$ vertices and at most $O(|E_i|)$ hyperedges can be implemented in $O(|V|(|V|+|E|))$ time using $O(|V|)$ queries to p-`max-sc-Oracle`, where each query to p-`max-sc-Oracle` is on a input hypergraph (G_0, c_0) that has $O(|V|)$ vertices and $O(|E|+|V|)$ hyperedges. Thus, all steps except for Steps 10, 13, and 14 of Algorithm 1 in the ith recursive call can be implemented to run in $O(|V|^3(|V|+|E|))$ time using $O(|V|^3)$ queries to p-`max-sc-Oracle`, where each query to p-`max-sc-Oracle` is on a input hypergraph (G_0, c_0) that has $O(|V|)$ vertices and $O(|E|+|V|)$ hyperedges.

We note that Steps 10 and 13 need not be implemented explicitly for the purposes of the algorithm. Instead, p_{i+1}-`max-wc-Oracle` on input hypergraphs with $|V_{i+1}| \leq |V|$ vertices and at most $|E_{i+1}|+1$ hyperedges can be used to execute all steps except for Steps 10, 13, and 14 in the $(i+1)^{th}$ recursive call. We have already seen in Claim 6.7 that a query to p_{i+1}-`max-wc-Oracle` on input hypergraphs with at most $|V|$ vertices and $O(|E_{i+1}|) = O(|V|+|E|)$ hyperedges can be implemented in $O(|V|(|V|+|E|))$ time using $O(|V|)$ queries to p-`max-sc-Oracle`. Step 14 is the recursive call. We note that the hypergraph returned in Step 14 contains at most $|V|$ vertices and at most $|E_i| + O(|V_i|) = O(|V|+|E|)$ hyperedges.

Thus, we have shown that each recursive call can be implemented to run in time $O(|V|^3(|V|+|E|))$ using $O(|V|^3)$ queries to p-`max-sc-Oracle`, where each query to p-`max-sc-Oracle` is on an input hypergraph (G_0, c_0) that has $O(|V|)$ vertices and $O(|E|+|V|)$ hyperedges. The number of recursive calls is $O(|V|+|E|)$. Hence, the algorithm can be implemented to run in $O(|V|^3(|V|+|E|)^2)$ time using $O(|V|^3(|V|+|E|))$ queries to p-`max-sc-Oracle`, where each query to p-`max-sc-Oracle` is on a input hypergraph (G_0, c_0) that has $O(|V|)$ vertices and $O(|E|+|V|)$ hyperedges. □

7 Conclusion

We introduced a splitting-off operation in (weighted) hypergraphs. Our contribution on this front is conceptualizing an appropriate notion of splitting-off in hypergraphs. Next, we proved that for every hypergraph there exists a local-connectivity preserving complete h-splitting-off at an arbitrary vertex from the hypergraph. Although our proof of existence follows from previously known existential results of Bernáth and Király [7] for an abstract function cover problem, our main contribution is identifying that our newly introduced notion of hypergraph splitting-off falls within the framework of their abstract function cover problem. Next, we designed a strongly polynomial-time algorithm to find a local-connectivity preserving complete splitting-off at a vertex. This involved substantial technical challenges to overcome. In particular, our main contribution towards the strongly polynomial-time algorithm is a strengthening of the above-mentioned existential result: we showed that there exists a local connectivity preserving complete h-splitting-off at an arbitrary vertex from the hypergraph in which the number of additional hyperedges is polynomial in the number of vertices. The strongly polynomial-time algorithm follows from our techniques to achieve this stronger existential result via standard algorithmic tools in submodularity. Finally, we illustrated the usefulness of the existence of local connectivity preserving complete h-splitting-off at an arbitrary vertex from a hypergraph by presenting two applications. Our first application is to give a constructive characterization of k-hyperedge-connected hypergraphs. Our second application is to give an alternative proof of an approximate min-max relation for the max Steiner rooted-connected orientation problem in graphs and hypergraphs. Two notable features of our proof of the approximate min-max relation for graphs are that (1) it avoids the strong orientation theorem of Nash-Williams and (2) it proves a result for graphs using tools developed for hypergraphs.

Local and global connectivity preserving complete splitting-off at a vertex from a graph is a powerful operation for graphs. It finds applications in a variety of graph problems including graph orientation [25, 28, 40, 50], connectivity augmentation [3, 24, 26, 27, 51], minimum cuts enumeration [32, 34, 52], network design

[14,19,37,48], tree packing [9,35,42,43], congruency-constrained cuts [53], and approximation algorithms for TSP [10,31]. We believe that our local connectivity preserving complete splitting-off results for hypergraphs is likely to find future applications akin to its counterpart in graphs. We mention some of the open questions raised by our work:

1. Our work focused on local connectivity preserving complete h-splitting-off at a vertex from a hypergraph. We gave an example showing that local/global connectivity preserving complete g-splitting-off at a vertex from a hypergraph may not exist (Figure 2). Are there sufficient conditions to guarantee local/global connectivity preserving complete g-splitting-off at a vertex from a hypergraph? We recall that Lovász's [47,49] and Mader's [50] results give sufficient conditions to guarantee local and global connectivity preserving complete g-splitting-off at a vertex from a graph.

2. As one of the applications of our splitting-off result, we presented an alternative proof of an approximate min-max relation for the max Steiner rooted-connected orientation problem in hypergraphs. The computational complexity of a closely related hypergraph orientation problem is open: In max Steiner connected orientation problem in hypergraphs, the input is a hypergraph $G = (V, E)$ and a subset T of terminals. The goal is to find the maximum k and an orientation $\overrightarrow{G}$ of G such that $\overrightarrow{G}$ contains k hyperarc-disjoint paths from u to v for every pair of distinct terminals $u, v \in T$. Max Steiner connected orientation problem in graphs is solvable in polynomial time via the Nash-Williams' strong orientation theorem. Is max Steiner connected orientation problem in hypergraphs solvable in polynomial time?

Acknowledgements. Karthekeyan thanks Eklavya Sharma for engaging in preliminary discussions on hypergraph splitting-off. Karthekeyan and Shubhang were supported in part by NSF grants CCF-1814613 and CCF-1907937. Karthekeyan was supported in part by the Distinguished Guest Scientist Fellowship of the Hungarian Academy of Sciences – grant number VK-6/1/2022. Kristóf and Tamás were supported in part by the Lendület Programme of the Hungarian Academy of Sciences – grant number LP2021-1/2021, by the Ministry of Innovation and Technology of Hungary from the National Research, Development and Innovation Fund – grant number ELTE TKP 2021-NKTA-62 funding scheme, and by the Dynasnet European Research Council Synergy project – grant number ERC-2018-SYG 810115.

References

[1] Head-disjoint strongly connected orientations, http://lemon.cs.elte.hu/egres/open/Head-disjoint_strongly_connected_orientations.

[2] O. Alrabiah, V. Guruswami, and R. Li. Randomly punctured Reed–Solomon codes achieve list-decoding capacity over linear-sized fields. Preprint on arXiv: 2304.09445, 2023.

[3] J. Bang-Jensen, A. Frank, and B. Jackson. Preserving and increasing local edge-connectivity in mixed graphs. SIAM Journal on Discrete Mathematics, 8(2):155–178, 1995.

[4] J. Bang-Jenson and B. Jackson. Augmenting hypergraphs by edges of size two. Mathematical Programming, 84:467–481, 1999.

[5] N. Bansal, O. Svensson, and L. Trevisan. New notions and constructions of sparsification for graphs and hypergraphs. In IEEE 60th Annual Symposium on Foundations of Computer Science, FOCS, pages 910–928, 2019.

[6] K. Bérczi, K. Chandrasekaran, C. Chekuri, T. Király, and S. Kulkarni. Private communication.

[7] A. Bernáth and T. Király. Covering skew-supermodular functions by hypergraphs of minimum total size. Operations Research Letters, 37(5):345–350, 2009.

[8] A. Bernáth and T. Király. A unifying approach to splitting-off. Combinatorica, 32:373—401, 2012.

[9] A. Bhalgat, R. Hariharan, T. Kavitha, and D. Panigrahi. Fast Edge Splitting and Edmonds' Arborescence Construction for Unweighted Graphs. In Proceedings of the 19th Annual ACM-SIAM Symposium on Discrete Algorithms, SODA, pages 455–464, 2008.

[10] J. Blauth and M. Nägele. An improved approximation guarantee for Prize-Collecting TSP. Preprint on arXiv: 2212.03776, 2023.

[11] R. Cen, W. He, J. Li, and D. Panigrahi. Steiner connectivity augmentation and splitting-off in poly-logarithmic maximum flows. In Proceedings of the 34th Annual ACM-SIAM Symposium on Discrete Algorithms, SODA, pages 2449–2488, 2023.

[12] R. Cen, J. Li, and D. Panigrahi. Augmenting edge connectivity via isolating cuts. In Proceedings of the 33rd Annual ACM-SIAM Symposium on Discrete Algorithms, SODA, pages 3237–3252, 2022.

[13] R. Cen, J. Li, and D. Panigrahi. Edge connectivity augmentation in near-linear time. In Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing, STOC, pages 137–150, 2022.

[14] Y. H. Chan, W. S. Fung, L. C. Lau, and C. K. Yung. Degree Bounded Network Design with Metric Costs. SIAM Journal on Computing, 40(4):953–980, 2011.

[15] K. Chandrasekaran and C. Chekuri. Hypergraph k-cut for fixed k in deterministic polynomial time. Mathematics of Operations Research, 47(4), 2022. Prelim. version in FOCS 2020.

[16] K. Chandrasekaran and C. Chekuri. Min-max partitioning of hypergraphs and symmetric submodular functions. Combinatorica, 43:455–477, 2023. Prelim. version in SODA 2021.

[17] K. Chandrasekaran, C. Xu, and X. Yu. Hypergraph k-Cut in randomized polynomial time. Mathematical Programming, 186:85–113, 2021. Prelim. version in SODA 2018.

[18] C. Chekuri and N. Korula. A Graph Reduction Step Preserving Element-Connectivity and Packing Steiner Trees and Forests. SIAM Journal on Discrete Mathematics, 28(2):577–597, 2014.

[19] C. Chekuri and B. Shepherd. Approximate Integer Decompositions for Undirected Network Design Problems. SIAM J. Discrete Math., 23:163–177, 2008.

[20] C. Chekuri and C. Xu. Minimum cuts and sparsification in hypergraphs. SIAM Journal on Computing, 47(6):2118–2156, 2018.

[21] Y. Chen, S. Khanna, and A. Nagda. Near-linear size hypergraph cut sparsifiers. In IEEE 61st Annual Symposium on Foundations of Computer Science, FOCS, pages 61–72, 2020.

[22] B. Cosh. Vertex splitting and connectivity augmentation in hypergraphs. PhD thesis, University of London, 2000.

[23] K. Fox, D. Panigrahi, and F. Zhang. Minimum cut and minimum k-cut in hypergraphs via branching contractions. ACM Trans. Algorithms, 19(2), 2023. Prelim. version in SODA 2019.

[24] A. Frank. Augmenting graphs to meet edge-connectivity requirements. SIAM Journal on Discrete Mathematics, 5(1):25–53, 1992.

[25] A. Frank. Applications of submodular functions. Surveys in Combinatorics, 1993 (Keele), pages 85–136, 1993.

[26] A. Frank. Connectivity augmentation problems in network design. Mathematical Programming: State of the Art 1994, pages 34–63, 1994.

[27] A. Frank. Connections in combinatorial optimization. Oxford University Press, Oxford, 2011.

[28] A. Frank and Z. Király. Graph orientations with edge-connection and parity constraints. Combinatorica, 22:47–70, 2002.

[29] H. N. Gabow. Efficient splitting off algorithms for graphs. In Proceedings of the 26th Annual ACM Symposium on Theory of Computing, STOC, pages 696—705, 1994.

[30] M. Ghaffari, D. Karger, and D. Panigrahi. Random Contractions and Sampling for Hypergraph and Hedge Connectivity. In Proceedings of the 28th annual ACM-SIAM Symposium on Discrete Algorithms, SODA, pages 1101–1114, 2017.

[31] M. Goemans and D. Bertsimas. Survivable networks, linear programming relaxations and the parsimonious property. Mathematical Programming, 60:145–166, 1993.

[32] M. X. Goemans. Approximate Edge Splitting. SIAM Journal on Discrete Mathematics, 14(1):138–141, 2001.

[33] Z. Guo, R. Li, C. Shangguan, I. Tamo, and M. Wootters. Improved List-Decodability and List-Recoverability of Reed-Solomon Codes via Tree Packings. In IEEE 62nd Annual Symposium on Foundations of Computer Science, FOCS, pages 708–719, 2022.

[34] M. Henzinger and D. Williamson. On the number of small cuts in a graph. Information Processing Letters, 59:41–44, 1996.

[35] K. Jain, M. Mahdian, and M. R. Salavatipour. Packing Steiner Trees. In Proceedings of the 14th Annual ACM-SIAM Symposium on Discrete Algorithms, SODA, pages 266–274, 2003.

[36] A. Jambulapati, Y. P. Liu, and A. Sidford. Chaining, group leverage score overestimates, and fast spectral hypergraph sparsification. In Proceedings of the 55th Annual ACM Symposium on Theory of Computing, STOC, pages 196–206, 2023.

[37] T. Jordán. On minimally k-edge-connected graphs and shortest k-edge-connected Steiner networks. Discrete Applied Mathematics, 131(2):421–432, 2003.

[38] M. Kapralov, R. Krauthgamer, J. Tardos, and Y. Yoshida. Towards tight bounds for spectral sparsification of hypergraphs. In Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing, STOC, pages 598–611, 2021.

[39] M. Kapralov, R. Krauthgamer, J. Tardos, and Y. Yoshida. Spectral Hypergraph Sparsifiers of Nearly Linear Size. In IEEE 62nd Annual Symposium on Foundations of Computer Science, FOCS, pages 1159–1170, 2022.

[40] T. Király and L. C. Lau. Approximate min–max theorems for Steiner rooted-orientations of graphs and hypergraphs. Journal of Combinatorial Theory, Series B, 98:1233–1252, 11 2008. Preliminary version in FOCS 2006.

[41] D. Kogan and R. Krauthgamer. Sketching cuts in graphs and hypergraphs. In Proceedings of the 2015 Conference on Innovations in Theoretical Computer Science, ITCS, pages 367–376, 2015.

[42] M. Kriesell. Edge-disjoint trees containing some given vertices in a graph. J. Comb. Theory Ser. B, 88:53–63, 2003.

[43] L. C. Lau. An Approximate Max-Steiner-Tree-Packing Min-Steiner-Cut Theorem. Combinatorica, 27:71–90, 2007.

[44] L. C. Lau and C. K. Yung. Efficient Edge Splitting-Off Algorithms Maintaining All-Pairs Edge-Connectivities. SIAM Journal on Computing, 42(3):1185–1200, 2013.

[45] J. R. Lee. Spectral hypergraph sparsification via chaining. In Proceedings of the 55th Annual ACM Symposium on Theory of Computing, STOC, pages 207–218, 2023.

[46] P. Li and O. Milenkovic. Inhomogeneous hypergraph clustering with applications. Advances in neural information processing systems, 30, 2017.

[47] L. Lovász. Lecture. Presented in a Conference on Graph Theory, Prague, 1974.

[48] L. Lovász. On some connectivity properties of Eulerian graphs. Acta Math. Hungarica, 28:129–138, 1976.

[49] L. Lovász. Combinatorial Problems and Exercises, Second Edition. American Mathematical Soc., 1993. First Edition: North-Holland, Amsterdam, 1979.

[50] W. Mader. A reduction method for edge-connectivity in graphs. In Annals of Discrete Mathematics, volume 3, pages 145–164. Elsevier, 1978.

[51] H. Nagamochi and T. Ibaraki. Algorithmic Aspects of Graph Connectivity. Encyclopedia of Mathematics and its Applications. Cambridge University Press, 2008.

[52] H. Nagamochi, K. Nishimura, and T. Ibaraki. Computing all small cuts in an undirected network. SIAM Journal on Discrete Mathematics, 10(3):469–481, 1997.

[53] M. Nägele and R. Zenklusen. A new contraction technique with applications to congruency-constrained cuts. Mathematical Programming, 183(2):455–481, 2020. Prelim. version in IPCO 2019.

[54] C. S. J. A. Nash-Williams. On orientations, connectivity and odd vertex pairings in finite graphs. Canad. J. Math, 12:555–567, 1960.

[55] C. S. J. A. Nash-Williams. Edge disjoint spanning trees of finite graphs. J. London Math. Soc., 36:445–450, 1961.

[56] S. Ornes. How big data carried graph theory into new dimensions. Quanta Magazine, 2021. https://www.quantamagazine.org/how-big-data-carried-graph-theory-into-new-dimensions-20210819/.

[57] K. Quanrud. Quotient sparsification for submodular functions. Manuscript available at kentquanrud.com, November 2022.

[58] H. E. Robbins. A Theorem on Graphs with an Application to a Problem of Traffic Control. Amer. Math. Monthly, 46:281–283, 1939.

[59] S. Schlag, T. Heuer, L. Gottesbüren, Y. Akhremtsev, C. Schulz, and P. Sanders. High-quality hypergraph partitioning. ACM Journal of Experimental Algorithmics, 27:1–39, 2023.

[60] T. Soma and Y. Yoshida. Spectral sparsification of hypergraphs. In Proceedings of the 30th Annual ACM-SIAM Symposium on Discrete Algorithms, SODA, pages 2570–2581, 2019.

[61] W. T. Tutte. On the problem of decomposing a graph into n connected factors. J. London Math. Soc., 36:221–230, 1961.

[62] N. Veldt, A. R. Benson, and J. Kleinberg. Hypergraph cuts with general splitting functions. SIAM Review, 64(3):650–685, 2022.

A An Example for Weak To Strong Cover with Quadratic New Hyperedges

Lemma A.1. There exists a symmetric skew-supermodular function $p : 2^V \to \mathbb{Z}$ and a hypergraph $\left(H = (V, E), w : E \to \mathbb{Z}_+\right)$ satisfying $b_{(H,w)}(X) \geq p(X)$ for every $X \subseteq V$ such that for every hypergraph $\left(H^* = (V, E^*), w^* : E^* \to \mathbb{Z}_+\right)$ obtained by merging hyperedges of the hypergraph (H, w) such that $d_{(H^*,w^*)}(X) \geq p(X)$ for every $X \subseteq V$, we have that

$$|E^* - E| = \Omega(|V|^2).$$

Proof. Let $u \in V$ be a fixed vertex. Let $p : 2^V \to \mathbb{Z} \cup \{-\infty\}$ be a symmetric skew-supermodular function defined as follows,

$$p(X) \coloneqq \begin{cases} \binom{|V|-1}{2} & \text{if } X \in \left\{\{u\}, V - u\right\}, \\ -\infty & \text{otherwise.} \end{cases}$$

Furthermore, let $(H = (V, E), w : E \to \mathbb{Z}_+)$ be a hypergraph such that $E \coloneqq \binom{V-u}{2} \cup \{\{u\}\}$, and the weight function w is defined as

$$w(e) \coloneqq \begin{cases} 1 & \text{if } e \in \binom{V-u}{2} \\ \binom{|V|-1}{2} & \text{otherwise (i.e., if } e = \{u\}). \end{cases}$$

We note that $b_{H,w}(X) \geq p(X)$ for every $X \subseteq V$. Next, consider the hypergraph $(H^* = (V, E^*), w^*)$, where the hyperedge set $E^* \coloneqq \left\{\{u, a, b\} : ab \in \binom{V-u}{2}\right\}$, and the weight function $w^*(e) \coloneqq 1$ for every $e \in E^*$. Then, the hypergraph (H^*, w^*) strongly covers the function p because $d_{H^*,w^*}(X) \geq p(X)$ for every $X \subseteq V$. Furthermore, the hypergraph (H^*, w^*) is the unique hypergraph that can be obtained by merging hyperedges of (H, w) to strongly cover the function p. We observe that $|E^* - E| = \Omega(|V|^2)$. □

B Projection of Laminar Family

We recall that a family $\mathcal{L} \subseteq 2^V$ is laminar if for every pair of sets $X, Y\mathcal{L}$, we have that either $X \subseteq Y$ or $Y \subseteq X$ or $X \cap Y = \emptyset$ – we use family to refer to a set (and not a multiset) of subsets of the ground set V. For a set $\mathcal{Z} \subseteq V$, and a family $\mathcal{L} \subseteq 2^V$, the projection of the family $\mathcal{L}$ to the ground set $V - \mathcal{Z}$ is the family formed by restricting each set of $\mathcal{L}$ to the ground set $V - \mathcal{Z}$, i.e. $\{X - \mathcal{Z} : X \in \mathcal{L}\}$. The next lemma says that if the family $\mathcal{L}$ is laminar, then the size of the projection is comparable to that of the original family $\mathcal{L}$.

Lemma 2.1. Let $\mathcal{L} \subseteq 2^V$ be a laminar family and $\mathcal{Z} \subseteq V$ be a subset of elements. Let $\mathcal{L}' \coloneqq \mathcal{L}|_{V-\mathcal{Z}}$. Then, the family $\mathcal{L}'$ is a laminar family and $|\mathcal{L}| \leq |\mathcal{L}'| + 3|\mathcal{Z}|$.

Proof. We first show that the family $\mathcal{L}'$ is a laminar family. By way of contradiction, let $X, Y \in \mathcal{L}'$ be two distinct sets such that $X - Y, Y - X, X \cap Y \neq \emptyset$. Then, there exist distinct sets $A, B \in \mathcal{L}$ such that $A - \mathcal{Z} = X$ and $B - \mathcal{Z} = Y$. Since the family $\mathcal{L}$ is laminar, we have that either $A \cap B = \emptyset$, $A \subseteq B$ or $B \subseteq A$. We note that if $A \subseteq B$, then we have that $X = A - \mathcal{Z} \subseteq B - \mathcal{Z} = Y$, which contradicts our choice of sets X and Y. Similarly, if $B \subseteq A$, then we obtain that $Y \subseteq X$, contradicting our choice of sets X and Y. Thus, we have that $A \cap B = \emptyset$, and so $X \cap Y = (A - \mathcal{Z}) \cap (B - \mathcal{Z}) = \emptyset$, once again contradicting our choice of sets X and Y.

Next, we show the second part of the claim, i.e. $|\mathcal{L}| \leq |\mathcal{L}'| + 3|\mathcal{Z}|$. For this, we first partition the family $\mathcal{L}$ into two subfamilies $\mathcal{L}_1 \coloneqq \{A \in \mathcal{L} : A \subseteq \mathcal{Z}\}$ and $\mathcal{L}_2 \coloneqq \mathcal{L} - \mathcal{L}_1$. Furthermore, for every $X \in \mathcal{L}'$, we let $S_X \coloneqq \{A \in \mathcal{L}_2 : A - \mathcal{Z} = X\}$ denote the family of sets in $\mathcal{L}_2$ that map to the (non-empty) set X under projection with the set $\mathcal{Z}$. We note that for every $A \in \mathcal{L}_2$, we have that $\emptyset \neq A - \mathcal{Z} \in \mathcal{L}'$ and hence $A \in S_{A-\mathcal{Z}}$. Consequently, the subfamilies S_X for $X \in \mathcal{L}'$ partition the family $\mathcal{L}_2$. In particular, we have that $\mathcal{L} = \mathcal{L}_1 \uplus \left(\biguplus_{X \in \mathcal{L}'} S_X\right)$. Claim B.1 below shows that the family S_X is a chain family for every $X \in \mathcal{L}'$. Next, for every $X \in \mathcal{L}'$, we define three sets: we let $A_X \coloneqq \arg\max\{|A| : A \in S_X\}$, $B_X \coloneqq \arg\min\{|B| : B \in S_X\}$, and $\mathcal{Z}_X \coloneqq A_X - B_X$. We note that $X \subseteq B_X \subseteq A_X$, where the second containment is because the family S_X is a chain family. Since $A_X - B_X \subseteq A_X - X \subseteq \mathcal{Z}$, we have that $\mathcal{Z}_X \subseteq \mathcal{Z}$. Claim B.2 below shows that

$|S_X| \leq |\mathcal{Z}_X| + 1$. Furthermore, Claim B.3 below shows that $\mathcal{Z}_X$ and $\mathcal{Z}_Y$ are disjoint for distinct $X, Y \in \mathcal{L}'$. Then, we have the following:

$$\begin{aligned}|\mathcal{L}| &= |\mathcal{L}_1| + \sum_{X \in \mathcal{L}'} |S_X| \\ &\leq 2|\mathcal{Z}| - 1 + \sum_{X \in \mathcal{L}'} \big(|\mathcal{Z}_X| + 1\big) \\ &\leq 3|\mathcal{Z}| + |\mathcal{L}'| - 1.\end{aligned}$$

Here, the first equality is because $\mathcal{L} = \mathcal{L}_1 \uplus \left(\biguplus_{X \in \mathcal{L}'} S_X\right)$. The first inequality is because the family $\mathcal{L}_1$ is a laminar family on the ground set $\mathcal{Z}$, and by Claim B.2. The final inequality is because the sets $\mathcal{Z}_X$ for $X \in \mathcal{L}'$ form a subpartition of the set $\mathcal{Z}$ by Claim B.3. □

Claim B.1. For every $X \in \mathcal{L}'$, the family S_X is a chain family.

Proof. By way of contradiction, let $X \in \mathcal{L}'$ be such that the family S_X is not a chain. Then, there exist sets $A, B \in S_X$ such that $A - B, B - A \neq \emptyset$. Since A and B are sets in the laminar family $\mathcal{L}$, we have that the sets A and B are disjoint. However, both the sets A and B contain the set X, and so $A \cap B \neq \emptyset$, a contradiction to the fact that A and B are sets in the laminar family $\mathcal{L}$. □

Claim B.2. For every $X \in \mathcal{L}'$, we have that $|S_X| \leq |\mathcal{Z}_X| + 1$.

Proof. We may assume that $|S_X| \geq 2$, i.e., $A_X \neq B_X$, as otherwise the claim holds. Then, we have the following: $|S_X| \leq |A_X| - |B_X| + 1 \leq |\mathcal{Z}_X| + 1$, where the first inequality is by Claim B.1. □

Claim B.3. For distinct non-empty sets $X, Y \in \mathcal{L}'$, we have that $\mathcal{Z}_X \cap \mathcal{Z}_Y = \emptyset$.

Proof. Let $X, Y \in \mathcal{L}'$ be distinct sets. Since the family $\mathcal{L}'$ is laminar, we have that either $X \subsetneq Y$ or $Y \subsetneq X$ or $X \cap Y = \emptyset$.

First, suppose that $X \subsetneq Y$ (we note that the case of $Y \subsetneq X$ can be show by a similar argument). Here, we note that $X \subseteq A_X \cap B_Y$ since $X \subseteq A_X$ and $X \subseteq Y \subseteq B_Y$. Consequently, $A_X \cap B_Y \neq \emptyset$. Since the family $\mathcal{L}$ is laminar and $A_X, B_Y \in \mathcal{L}$, we have that either $A_X \subseteq B_Y$ or $B_Y \subseteq A_X$. We note that if $A_X \subseteq B_Y$, then the claim holds since $\mathcal{Z}_X \subseteq A_X \subseteq B_Y \subseteq V - \mathcal{Z}_Y$. We now show that $B_Y - A_X \neq \emptyset$, i.e. the scenario $B_Y \subseteq A_X$ cannot happen. Consider a vertex $v \in Y - X$. We note that $v \notin \mathcal{Z}$ since $v \in Y$. Furthermore, $v \notin A_X$ since $v \notin X$ and $A_X - X \subseteq \mathcal{Z}$. Thus, we have that $v \in B_Y - A_X \neq \emptyset$.

Next, suppose that the sets X and Y are disjoint. We also suppose that $A_X \cap A_Y \neq \emptyset$ as otherwise, we have that $\mathcal{Z}_X \cap \mathcal{Z}_Y \subseteq A_X \cap A_Y = \emptyset$ and the claim holds. Since the family $\mathcal{L}$ is laminar and $A_X, A_Y \in \mathcal{L}$, either $A_X \subseteq A_Y$ or $A_Y \subseteq A_X$. Suppose that $A_X \subseteq A_Y$ (we note that the case of $A_Y \subseteq A_X$ can be show by a similar argument). We also suppose that $A_X - B_Y \neq \emptyset$, as otherwise $\mathcal{Z}_X = A_X - B_X \subseteq B_Y \subseteq V - (A_Y - B_Y) = V - \mathcal{Z}_Y$, and the claim holds. The rest of the proof is devoted to showing that this case is impossible. In particular, we show that the sets A_X and B_Y are such that $A_X - B_Y, B_Y - A_X, A_X \cap B_Y \neq \emptyset$, contradicting the laminarity of the family $\mathcal{L}$. First, we show that $B_Y - A_X \neq \emptyset$. By way of contradiction, suppose that $B_Y \subseteq A_X$. Then, we have that $Y \subseteq A_X$ since $Y \subseteq B_Y$. Furthermore, we recall that $Y \cap \mathcal{Z} = \emptyset$. Thus, we have that $\emptyset = Y \cap X = Y \cap (A_X - \mathcal{Z}) = Y \cap A_X$. Thus, $Y \subseteq A_X$ and $Y \cap A_X = \emptyset$ together imply that $Y = \emptyset$, contradicting $Y \in \mathcal{L}'$. Second, we show that $B_Y \cap A_X \neq \emptyset$. By way of contradiction, suppose that $B_Y \cap A_X = \emptyset$. We recall that $A_X \subseteq A_Y$. Consequently, we have that $X \subseteq A_X \subseteq A_Y - B_Y \subseteq \mathcal{Z}$, contradicting $X \in \mathcal{L}'$. Thus the sets A_X and B_Y are such that $A_X - B_Y, B_Y - A_X, A_X \cap B_Y \neq \emptyset$, giving us the required contradiction. □

C Optimization Problems using Function Maximization Oracle

In this section, we prove Lemmas 2.2, 3.1, and 6.13. The proofs of these lemmas follow from known tools in the area of submodular functions. We present their proofs for the sake of completeness.

C.1 p-max-wc-Oracle using p-max-sc-Oracle

In this section, we prove Lemma 2.2.

Lemma 2.2. Let $p : 2^V \to \mathbb{Z}$ be a set function, $(G = (V, E), w : E \to \mathbb{Z}_+)$ be a hypergraph, and $S, T \subseteq V$ be disjoint sets. Then, p-max-wc-Oracle$((G, w), S, T)$ can be implemented to run in $O(|V|(|V|+|E|))$ time using at most $|V| + 1$ queries to p-max-sc-Oracle. The run-time includes the time to construct the hypergraphs used as input to the queries to p-max-sc-Oracle. Moreover, each query to p-max-sc-Oracle is on an input hypergraph (G_0, c_0) that has at most $|V|$ vertices and $|E|$ hyperedges.

Proof. Let $u \in V$ be a vertex. We define the hypergraph $G^u := (V, E^u := \{e \cup \{u\} : e \in E\})$ and the weight function $w^u : E^u \to \mathbb{Z}_+$ as $w^u(e \cup \{u\}) := w(e)$ for every $e \in E$. We observe that $b_{(G,w)}(X) = d_{(G^u,w^u)}(X)$ for every set $X \subseteq V - \{u\}$. We also let $(Z^u, p(Z^u))$ denote the tuple obtained by querying p-max-sc-Oracle $((G_0, c_0) := (G^u, w^u), S_0 := S, T_0 := T \cup \{u\})$.

We now describe a procedure to compute an answer $(Z, p(Z))$ to the query p-max-wc-Oracle$((G, w), S, T)$. We consider two cases to define our set Z. If $T \neq \emptyset$, then we choose an arbitrary vertex $t \in T$ and define $Z := Z^t$. Otherwise, $T = \emptyset$ and we define the set Z to be an arbitrary optimum solution to $\max\{p(Z) - b_{(G,w)}(Z) : Z \in \mathcal{Z}\}$, where $\mathcal{Z} := \{Z^u : u \in V\} \cup \{V\}$. We return the tuple $(Z, p(Z))$ as the answer to the query p-max-wc-Oracle$((G, w), S, T)$.

We now prove correctness of the above procedure. Let Z be the set returned by the above procedure. Then, by definition $S \subseteq V \subseteq V - T$. Our goal is to show that Z is an optimum solution to the following problem:

$$\max \Big\{ p(X) - b_{(G,w)}(X) : S \subseteq X \subseteq V - T \Big\} \tag{1}$$

Let Y be an optimum solution to (1). It suffices to show that $p(Z) - b_{(G,w)}(Z) \geq p(Y) - b_{(G,w)}(Y)$. We consider two cases: suppose that $T \neq \emptyset$. Then, we have that

$$\begin{aligned} p(Z) - b_{(G,w)}(Z) &= p(Z^t) - d_{(G^t,w^t)}(Z^t) \\ &\geq p(Y) - d_{(G^t,w^t)}(Y) \\ &= p(Y) - b_{(G,w)}(Y). \end{aligned}$$

In the above, the first equality is because $Z = Z^t$, the inequality is because $Y \subseteq V - \{v\}$ and by definition of Z^t, and the last equality is because $b_{(G,w)}(Y) = d_{(G^v,w^v)}(Y)$ since $Y \subseteq V - \{t\}$. Next, suppose that $T = \emptyset$. We consider two subcases. Firstly, suppose that V is the unique optimum solution to (1). Then, $Z = Y = V$ by definition and we are done. Secondly, suppose that $Y \neq V$. Let $v \in V - Y$. Then, we have the following:

$$\begin{aligned} p(Z) - b_{(G,w)}(Z) &\geq p(Z^v) - b_{(G,w)}(Z^v) \\ &= p(Z^v) - d_{(G^v,w^v)}(Z^v) \\ &\geq p(Y) - d_{(G^v,w^v)}(Y) \\ &= p(Y) - b_{(G,w)}(Y). \end{aligned}$$

In the above sequence, the first inequality is because of the choice of the set Z (i.e., Z is an optimum solution to $\max\{p(Z) - b_{(G,w)}(Z) : Z \in \mathcal{Z}\}$ and $Z^v \in \mathcal{Z}$), the first equality is because $b_{(G,w)}(Z^v) = d_{(G^v,w^v)}(Z^v)$ since $Z^v \subseteq V - \{v\}$, the second inequality is by definition of Z^v, and the second equality is because $b_{(G,w)}(Y) = d_{(G^v,w^v)}(Y)$ since $Y \subseteq V - \{v\}$. Thus, all inequalities in the above sequence are equalities, and we have that $p(Z) - b_{(G,w)}(Z) = p(Y) - b_{(G,w)}(Y)$. Thus, in both cases, the set Z returned by our procedure is an optimum solution to (1).

We now analyze the runtime of the procedure and the number of p-max-sc-Oracle queries. Let $u \in V$ be a vertex. Constructing the hypergraph (G^u, w^u) takes $O(|V| + |E|)$. Thus, a tuple $(Z^u, p(Z^u))$ takes $O(|V| + |E|)$ time and one query to p-max-sc-Oracle. The runtime includes the time to construct the inputs to the query. Moreover, the input hypergraph to the query has $|V|$ vertices and at most $|E|$ hyperedges. Thus,

if $T \neq \emptyset$, the procedure makes a single p-`max-sc-Oracle` query and runs in $O(|V| + |E|)$ time. Alternatively, if $T = \emptyset$, then the procedure makes $|V| + 1$ p-`max-sc-Oracle` queries and runs in $O(|V|(|V| + |E|))$ time. In both cases, the runtime includes the time to construct the inputs for the p-`max-sc-Oracle` queries. Moreover, the input hypergraph to each query has $|V|$ vertices and at most $|E|$ hyperedges. □

C.2 Skew-supermodularity of a function p

In this section, we prove Lemma 3.1.

Lemma 3.1. Let $(G = (V, E), c : E \to \mathbb{Z}_+)$ be a hypergraph. Let $r : \binom{V}{2} \to \mathbb{Z}_{\geq 0}$ be a function on pairs of elements of V and $R, p_{(G,c,r)} : 2^V \to \mathbb{Z}$ be functions defined by $R(X) \coloneqq \max\{r(\{u, v\}) : u \in X, v \in V \setminus X\}$ for every $X \subseteq V$, $R(\emptyset) \coloneqq 0$, $R(V) \coloneqq 0$, and $p_{(G,c,r)}(X) \coloneqq R(X) - d_{(G,c)}(X)$ for every $X \subseteq V$.

1. The function $p_{(G,c,r)}$ is symmetric skew-supermodular.
2. Given hypergraph (G, c), function r, hypergraph $(G_0 = (V, E_0), c_0 : E_0 \to \mathbb{Z}_+)$, and disjoint sets $S_0, T_0 \subseteq V$, the oracle $p_{(G,c,r)}$-`max-sc-Oracle` $((G_0, c_0), S_0, T_0)$ can be computed in $O(|V|^3(|V| + |E_0| + |E|)(|E_0| + |E|))$ time.

Proof. For ease of notation, let $p = p_{(G,c,r)}$. The function $R : 2^V \to \mathbb{Z}_{\geq 0}$ is symmetric skew-supermodular (see Lemma 8.1.9 of [27]) and the function $d_{(G,c)} : 2^V \to \mathbb{Z}_{\geq 0}$ is symmetric submodular. The function p is obtained by subtracting a symmetric submodular function from a symmetric skew-supermodular and is hence, symmetric skew-supermodular.

Next, we show that p-`max-sc-Oracle` can be implemented in strongly polynomial-time. Let the input to the oracle be the hypergraph $(G_0 = (V, E_0), c_0 : E_0 \to \mathbb{Z}_+)$ and disjoint subsets $S_0, T_0 \subseteq V$. We construct the hypergraph $(G + G_0, c + c_0)$. Moreover, for distinct vertices $u, v \in V$ with $u \in V - T_0$ and $v \in V - S_0$, we compute

$$\lambda_{(G+G_0,c+c_0)}(u, v) \coloneqq \min\left\{d_{(G+G_0,c+c_0)}(Z) : S_0 \cup \{u\} \subseteq Z \subseteq V - (T_0 \cup \{v\})\right\}$$

and the set $Z(u, v)$ that achieves the minimum for $\lambda_{(G+G_0,c+c_0)}(u, v)$. We note that $\lambda_{(G+G_0,c+c_0)}(u, v)$ and $Z(u, v)$ can be computed using a single call to a min (s, t)-cut oracle on the hypergraph $(G + G_0, c + c_0)$. This hypergraph has $|V|$ vertices and at most $|E_0| + |E|$ hyperedges.

To implement the p-`max-sc-Oracle`, we observe that

$$\begin{aligned}
&\max\left\{p(Z) - d_{(G_0,c_0)}(Z) : S_0 \subseteq Z \subseteq V - T_0\right\} \\
&\quad = \max\left\{R(Z) - d_{(G,c)}(Z) - d_{(G_0,c_0)}(Z) : S_0 \subseteq Z \subseteq V - T_0\right\} \\
&\quad = \max\left\{r(u, v) - \lambda_{((G,c),(G_0,c_0))}(u, v) : u \in V - T_0, v \in V - S_0, u \neq v\right\}.
\end{aligned}$$

The RHS problem can be solved by iterating over all distinct vertices $u \in V - T_0$ and $v \in V - S_0$ and returning a pair for which the objective function $r(u, v) - \lambda_{((G,c),(G_0,c_0))}(u, v)$ is maximum. Let $u \in V - T_0$ and $v \in V - S_0$ be a pair of vertices that achieves the maximum in the RHS problem. Then, we observe that the set $Z(u, v)$ is a maximizer for the LHS problem. Thus, it suffices to return $Z(u, v)$ and the value $p(Z(u, v))$. We observe that the value $p(Z(u, v))$ can be computed by subtracting $d_{(G_0,c_0)}(Z(u, v))$ from the objective value of the RHS problem.

The above procedure can be implemented in $O(|V|^2(|V| + |E| + |E_0|))$ time and $O(|V|^2)$ calls to a min (s, t)-cut oracle on hypergraphs with $|V|$ vertices and $|E| + |E_0|$ hyperedges. Then, the claimed runtime is obtained by using a crude upper bound on the run-time of standard deterministic min (s, t)-cut algorithms for hypergraphs—we recall that min (s, t)-cut in a n-vertex m-hyperedge hypergraph can be solved in $O((m + n)mn)$. □

C.3 Computing the Family of Maximal (p, H, w)-Tight Sets

In this section, we prove Lemma 6.13. We recall that for a set function $p : 2^V \to \mathbb{Z}$ and hypergraph $(H = (V, E), w)$, a set $X \subseteq V$ is (p, H, w)-tight if $p(X) = b_{(H,w)}(X)$. The family $\mathcal{T}_{p,H,w}$ denotes the family of all inclusion-wise maximal (p, H, w)-tight sets. In this section, we show that the family $\mathcal{T}_{p,H,w}$ can be computed in $O(|V|^2(|V|+|E|))$ time using $O(|V|^2)$ queries to p-`max-Oracle` if the function p and hypergraph (H, w) satisfy certain properties.

Lemma 6.13. Let $p : 2^V \to \mathbb{Z}$ be a skew-supermodular function and let $\big(H = (V, E), w : E \to \mathbb{Z}_+\big)$ be a hypergraph such that every hyperedge $e \in E$ is contained in some (p, H, w)-tight set, $b_{(H,w)}(u) \neq 0$ for every $u \in V$, and $b_{(H,w)}(X) \geq p(X)$ for every $X \subseteq V$. Then, the family $\mathcal{T}_{p,H,w}$ can be computed in $O(|V|^2(|V| + |E|))$ time using $O(|V|^2)$ queries to p-`max-wc-Oracle`. The run-time includes the time to construct the hypergraphs used as input to the queries to p-`max-wc-Oracle`. Moreover, for each query to p-`max-wc-Oracle`, the hypergraph (G_0, c_0) used as input to the query has at most $|V|$ vertices and at most $|E| + 1$ hyperedges.

Proof. We describe a procedure that recursively constructs $\mathcal{T}_{p,H,w}$. The procedure finds a (p, H, w)-tight set $X \subseteq V$, and then searches for a maximal (p, H, w)-tight set Y containing the set X — we describe this computation in the paragraph below. The procedure is recursive and it reaches the base case if the computed set X is not a (p, H, w)-tight set or $X, Y = \emptyset$. If X is not a (p, H, w)-tight set, then the procedure returns the empty set, and if $X, Y = \emptyset$, then the procedure returns $\{\emptyset\}$. If both of the base case conditions are not satisfied, the procedure recurses on the function $p' : 2^{V'} \to \mathbb{Z}$ defined on the ground set $V' := V - Y$ as $p'(Z) = p(Z)$ for every $Z \subseteq V'$, and the hypergraph $(H' = (V', E'), w')$ where $E' := \{e \cap V' : e \in E\}$ and $w'(e') := \sum_{e \in E} w(e) \cdot \mathbb{1}_{e \cap V' = e'}$ for every $e' \in E'$, to obtain the family $\mathcal{T}_{p',H',w'}$. It then returns the family $\{Y\} \cup (\mathcal{T}_{p',H',w'} - \{\emptyset\})$. The procedure terminates within $|V|$ recursive calls because the size of the ground set decreases during every recursive call (i.e. $|V'| < |V|$). Furthermore, the correctness of the procedure follows from disjointness of the family $\mathcal{T}_{p,H,w}$ by Lemma 6.2 and because $b_{(H,w)}(X) = b_{(H',w')}(X)$ for each $X \subseteq V'$, where (H', w') is the hypergraph constructed as input to the subsequent recursive call.

We now bound the run-time and the number of queries required by the procedure described above. Consider a recursive call of the procedure. We note that a (p, H, w)-tight set exists if and only if the set returned by the query p-`max-wc-Oracle`$((G_0, c_0) := (H, w), S_0 := \emptyset, T_0 := \emptyset)$ is a (p, H, w)-tight set. Furthermore, verifying whether the returned set X is (p, H, w)-tight can be done in $O(|V| + |E|)$ time by checking whether $p(X) = b_{(H,w)}(X)$. Thus, a (p, H, w)-tight set $X \subseteq V$ can be found in $O(|V| + |E|)$ time using one query to p-`max-wc-Oracle`. Moreover, the input hypergraph to p-`max-wc-Oracle` query has $|V|$ vertices and $|E|$ hyperedges. Next, Claim C.1 below shows that we can find a maximal (p, H, w)-tight set Y containing the set X in $O(|V|(|V| + |E|))$ time using $O(|V|)$ queries to p-`max-wc-Oracle` — here, the runtime includes the time required to construct the inputs to p-`max-wc-Oracle`; moreover, each query to p-`max-wc-Oracle` takes the hypergraph (H, w) as input. Let $p' : 2^{V'} \to \mathbb{Z}$ be the input function to the subsequent recursive call. Let $(G = (V', E_G), c)$ be a hypergraph such that $|E_G| \leq |E|$ and $S, T \subseteq V'$ be disjoints sets. Then, Claim C.2 below shows that a query to p'-`max-wc-Oracle`$((G, c), S, T)$ can be implemented in $O(|V|+|E|)$ time using one query to p-`max-wc-Oracle` — here, the runtime includes the time required to construct the input to p-`max-wc-Oracle` query; moreover the hypergraph input to p-`max-wc-Oracle` query has $|V|$ vertices and $|E_G|$ hyperedges. We note that all hypergraphs $(G = (V', E_G), c)$ that are used as inputs to p'-`max-wc-Oracle` during subsequent recursive calls satisfy $|E_G| \leq |E|$. Thus, every recursive call of the procedure can be implemented to run in $O(|V|(|V|+|E|))$ time and make $O(|V|)$ calls to p-`max-wc-Oracle`, where the runtime includes the time to construct the inputs to each query to p-`max-wc-Oracle`. Consequently, the procedure runs in time $O(|V|^2(|V| + |E|))$ and makes $O(|V|^2)$ queries to p-`max-wc-Oracle`, where the runtime includes the time to construct the inputs to the p-`max-wc-Oracle` queries. □

Claim C.1. Let $X \subseteq V$ be a (p, H, w)-tight set. Then, a maximal (p, H, w)-tight set containing the set X can be computed in $O(|V|(|V| + |E|))$ time using $O(|V|)$ queries to p-`max-wc-Oracle`. The runtime includes the time required to construct the inputs to p-`max-wc-Oracle`. Moreover, each query to p-`max-wc-Oracle` takes the hypergraph (H, w) as input.

Proof. We describe a recursive procedure to compute a maximal (p, H, w)-tight set containing the set X. For every $u \in V - X$, let Z^u be the set returned by querying p-`max-wc-Oracle`$((H_0, w_0) := (H, w), S_0 := X \cup \{u\}, T_0 := \emptyset)$. If there exists a vertex $u \in X$ such that the set Z^u is a (p, H, w)-tight set, then recursively compute a (p, H, w)-tight set containing the set $X' := Z^u$. Otherwise, return the set X.

The correctness of the above procedure is because of the following. The above procedure terminates (in at most $|V|$ recursive calls) because the cardinality of the input set X increases during each recursive call (i.e., $|X'| > |X|$). Let $Y \subseteq V$ be a maximal (p, H, w)-tight set such that $X \subseteq Y$. If $Y = X$, then there are no (p, H, w)-tight sets that strictly contain the set X and the procedure returns the set X. Suppose that $X \subsetneq Y$ and let $u \in Y - X$. Then, there exists a (p, H, w)-tight set Z such that $X \cup \{u\} \subsetneq Z \subseteq Y$. Thus, the set Z^u computed by the procedure will be a (p, H, w)-tight set that strictly contains the set X. Using this fact, the procedure can be shown to return a maximal (p, H, w)-tight set containing the set X by induction on $|X|$.

We now describe a slight modification of the above procedure that leads to an improved runtime and a smaller number of queries to p-`max-wc-Oracle`. Let $u \in V - X$ be a vertex. If the set Z^u is (p, H, w)-tight, then we include u in the input set S_0 to all p-`max-wc-Oracle` queries in subsequent recursive calls of the procedure. Alternatively, if the set Z^u is not (p, H, w)-tight, then we include u in the input set T_0 to all p-`max-wc-Oracle` queries in subsequent recursive calls procedure. Consequently, the number of queries to p-`max-wc-Oracle` (and number of recursive calls of the procedure) is at most $|V|$. We note that constructing the input to each query and verifying whether the set returned by the query is (p, H, w)-tight can be done in $O(|V| + |E|)$ time. Thus, the procedure terminates in $O(|V|(|V| + |E|))$ time. Here, the runtime includes the time to construct the inputs to the p-`max-wc-Oracle` queries. Moreover, each query takes the hypergraph (H, w) as input. □

Claim C.2. Let $p : 2^V \to \mathbb{Z}$ be a function. Let $Y \subseteq V$ and let p' denote the function p restricted to the ground set $V' := V - Y$. Then, for a given hypergraph $(G = (V', E_G), c : E_G \to \mathbb{Z}_+)$ such that $|E_G| \leq |E|$ and disjoint subsets $S, T \subseteq V'$, p'-`max-wc-Oracle`$((G, c), S, T, y)$ can be computed in $O(|V| + |E|)$ time using one query to p-`max-wc-Oracle`. The runtime includes the time required to construct the input to p-`max-wc-Oracle` query. Moreover, the hypergraph input to p-`max-wc-Oracle` query has $|V|$ vertices and $|E_G|$ hyperedges.

Proof. Let (G_1, c_1) be the hypergraph obtained from (G, c) by adding the vertices Y. Then, the answer to the query p'-`max-wc-Oracle`$((G, c), S, T)$ is the same as the answer to the query p-`max-wc-Oracle`$((G_1, c_1), S, T \cup Y)$. □